

UNIVERSIDADE ESTADUAL DO NORTE FLUMINENSE  
LABORATÓRIO DE ENGENHARIA E EXPLORAÇÃO DE PETRÓLEO  
CENTRO DE CIÊNCIA E TECNOLOGIA

GeoLogViewer - Ferramenta de Leitura e Visualização de Dados Geofísicos  
de Arquivos - LAS

DISCIPLINA LEP - LEP01348 : Introdução ao Projeto de Engenharia  
Setor de Modelagem Matemática Computacional

Versão 1:  
Pedro Daneluci Cardozo  
Rafael Moreira Silva  
Prof. André Duarte Bueno

MACAÉ - RJ  
NOVEMBRO 2025

# Sumário

|          |                                                          |           |
|----------|----------------------------------------------------------|-----------|
| <b>1</b> | <b>Introdução</b>                                        | <b>1</b>  |
| 1.1      | Escopo do problema . . . . .                             | 1         |
| 1.2      | Objetivos . . . . .                                      | 2         |
| 1.3      | Metodologia utilizada . . . . .                          | 3         |
| <b>2</b> | <b>Concepção</b>                                         | <b>6</b>  |
| 2.1      | Nome do sistema/produto . . . . .                        | 7         |
| 2.2      | Especificação . . . . .                                  | 7         |
| 2.3      | Requisitos . . . . .                                     | 8         |
| 2.3.1    | Requisitos funcionais . . . . .                          | 8         |
| 2.3.2    | Requisitos não funcionais . . . . .                      | 8         |
| 2.4      | Casos de uso . . . . .                                   | 9         |
| 2.4.1    | Diagrama de caso de uso geral . . . . .                  | 9         |
| 2.4.2    | Diagrama de caso de uso específico . . . . .             | 10        |
| <b>3</b> | <b>Elaboração</b>                                        | <b>13</b> |
| 3.1      | Análise de domínio . . . . .                             | 13        |
| 3.2      | Formulação teórica . . . . .                             | 14        |
| 3.2.1    | Fundamentos da Perfilagem de Poço . . . . .              | 14        |
| 3.2.2    | Propriedades Físicas Medidas por Perfilagem . . . . .    | 15        |
| 3.2.3    | Interpretação Geológica a partir de Perfis . . . . .     | 16        |
| 3.2.4    | Técnicas de Aquisição e Formato de Arquivo LAS . . . . . | 18        |
| 3.2.5    | Estimativas Petríficas . . . . .                         | 18        |
| 3.3      | Identificação de pacotes – assuntos . . . . .            | 21        |
| 3.4      | Diagrama de pacotes – assuntos . . . . .                 | 22        |
| <b>4</b> | <b>AOO – Análise Orientada a Objeto</b>                  | <b>23</b> |
| 4.1      | Diagramas de classes . . . . .                           | 23        |
| 4.1.1    | Dicionário de classes . . . . .                          | 23        |
| 4.2      | Diagrama de sequência – eventos e mensagens . . . . .    | 25        |
| 4.2.1    | Diagrama de sequência geral . . . . .                    | 25        |
| 4.3      | Diagrama de comunicação – colaboração . . . . .          | 26        |

|          |                                                                                               |            |
|----------|-----------------------------------------------------------------------------------------------|------------|
| 4.4      | Diagrama de estado . . . . .                                                                  | 27         |
| 4.5      | Diagrama de atividades . . . . .                                                              | 28         |
| <b>5</b> | <b>Projeto</b>                                                                                | <b>30</b>  |
| 5.1      | Projeto do Sistema . . . . .                                                                  | 30         |
| 5.2      | Projeto orientado a objeto – POO . . . . .                                                    | 32         |
| 5.3      | Diagrama de componentes . . . . .                                                             | 35         |
| 5.4      | Diagrama de implantação . . . . .                                                             | 36         |
| 5.4.1    | Lista de características <<features>> . . . . .                                               | 37         |
| 5.4.2    | Tabela classificação sistema . . . . .                                                        | 39         |
| <b>6</b> | <b>Ciclos de Planejamento/Detalhamento</b>                                                    | <b>42</b>  |
| 6.1      | Versão 0.1 - Núcleo de Leitura e Estruturação dos Dados LAS . . . . .                         | 42         |
| 6.2      | Versão 0.2 - Módulo de Plotagem Inicial e Integração com Gnuplot . . . . .                    | 43         |
| 6.3      | Versão 0.3 - Interpretação Petrofísica e Visualização Avançada . . . . .                      | 43         |
| <b>7</b> | <b>Ciclos Construção- Implementação</b>                                                       | <b>45</b>  |
| 7.1      | Código fonte . . . . .                                                                        | 45         |
| <b>8</b> | <b>Teste</b>                                                                                  | <b>161</b> |
| 8.1      | Teste 1: Leitura do arquivo LAS e verificação dos perfis . . . . .                            | 161        |
| 8.2      | Teste 2: Plotagem dos perfis do poço . . . . .                                                | 163        |
| 8.3      | Teste 3: Interpretação automática, exportação de dados e encerramento do aplicativo . . . . . | 167        |
| <b>9</b> | <b>Documentação para o Desenvolvedor</b>                                                      | <b>171</b> |
| 9.1      | Dependências para compilar o software . . . . .                                               | 171        |

# Lista de Figuras

|     |                                                                                    |     |
|-----|------------------------------------------------------------------------------------|-----|
| 1.1 | Etapas para o desenvolvimento do software - <i>projeto de engenharia</i> . . . . . | 5   |
| 2.1 | Diagrama de caso de uso . . . . .                                                  | 10  |
| 2.2 | Diagrama de caso de uso específico de comparação entre curvas . . . . .            | 11  |
| 2.3 | Diagrama de caso de uso específico de tipos de plotagem . . . . .                  | 12  |
| 3.1 | Representação esquemática de uma descida de uma ferramenta de perfilagem           | 15  |
| 3.2 | Exemplo de perfil usualmente utilizado na indústria . . . . .                      | 17  |
| 3.3 | Diagrama de Pacotes . . . . .                                                      | 22  |
| 4.1 | Diagrama de classes . . . . .                                                      | 24  |
| 4.2 | Diagrama de seqüência . . . . .                                                    | 26  |
| 4.3 | Diagrama de comunicação . . . . .                                                  | 27  |
| 4.4 | Diagrama de máquina de estado . . . . .                                            | 28  |
| 4.5 | Diagrama de atividades . . . . .                                                   | 29  |
| 5.1 | Diagrama de componentes . . . . .                                                  | 36  |
| 5.2 | Diagrama de implantação . . . . .                                                  | 37  |
| 8.1 | Tela do prompt executando a leitura do arquivo LAS . . . . .                       | 162 |
| 8.2 | Tela do prompt listando curvas do arquivo LAS . . . . .                            | 163 |
| 8.3 | Tela do prompt ilustrando os perfis do arquivo . . . . .                           | 164 |
| 8.4 | Tela do prompt ilustrando os perfis com overlay e normalização . . . . .           | 165 |
| 8.5 | Tela do prompt ilustrando perfis com overlay e sem normalização . . . . .          | 166 |
| 8.6 | Tela do prompt ilustrando perfis sem overlay . . . . .                             | 167 |
| 8.7 | Tela do prompt ilustrando a interação do reservatório . . . . .                    | 168 |
| 8.8 | Tela do prompt ilustrando o arquivo CSV gerado . . . . .                           | 169 |
| 8.9 | Tela do prompt ilustrando a saída do aplicativo . . . . .                          | 170 |

# Lista de Tabelas

|     |                         |   |
|-----|-------------------------|---|
| 2.1 | Caso de uso 1 . . . . . | 9 |
|-----|-------------------------|---|

# Listagens

|      |                                               |     |
|------|-----------------------------------------------|-----|
| 7.1  | Arquivo main.cpp . . . . .                    | 45  |
| 7.2  | Arquivo CGeoLogViewerApp.cpp . . . . .        | 45  |
| 7.3  | Arquivo CGeoLogViewerApp.h . . . . .          | 50  |
| 7.4  | Arquivo CLASReader.h . . . . .                | 51  |
| 7.5  | Arquivo CLASReader.cpp . . . . .              | 51  |
| 7.6  | Arquivo CLogCurve.h . . . . .                 | 56  |
| 7.7  | Arquivo CLogCurve.cpp . . . . .               | 57  |
| 7.8  | Arquivo CWellLog.h . . . . .                  | 57  |
| 7.9  | Arquivo CWellLog.cpp . . . . .                | 58  |
| 7.10 | Arquivo CFormationZone.h . . . . .            | 59  |
| 7.11 | Arquivo CInterpreter.h . . . . .              | 60  |
| 7.12 | Arquivo CInterpreter.cpp . . . . .            | 63  |
| 7.13 | Arquivo CPetrophysicsCalculator.h . . . . .   | 70  |
| 7.14 | Arquivo CPetrophysicsCalculator.cpp . . . . . | 71  |
| 7.15 | Arquivo CGnuplot.h . . . . .                  | 74  |
| 7.16 | Arquivo CGnuplot.cpp . . . . .                | 91  |
| 7.17 | Arquivo CPlotter.h . . . . .                  | 130 |
| 7.18 | Arquivo CPlotter.cpp . . . . .                | 131 |
| 7.19 | Arquivo CLogCurveUtilities.h . . . . .        | 157 |
| 7.20 | Arquivo CLogCurveUtilities.cpp . . . . .      | 157 |

# Capítulo 1

## Introdução

A elaboração de projetos constitui uma das atribuições fundamentais que diferencia o engenheiro do tecnólogo, exigindo a aplicação integrada de bases científicas, técnicas e computacionais na resolução de problemas reais. Nesse cenário, propõe-se o desenvolvimento de um projeto de engenharia de software direcionado à área de Engenharia de Petróleo, com foco na manipulação e interpretação de dados geofísicos empregados na caracterização de formações e reservatórios. Este projeto propõe o desenvolvimento do GeoLogViewer, um software educacional interativo destinado à leitura, interpretação e visualização de perfis de poços registrados em arquivos LAS (Log ASCII Standard). A ferramenta tem como finalidade facilitar o aprendizado de conceitos de petrofísica e geologia aplicada, permitindo que estudantes interajam diretamente com dados reais utilizados na indústria, visualizando curvas como Gamma Ray (GR), densidade (RHOB) e porosidade (NPHI), de forma simples, leve e acessível.

Com base no diagnóstico da realidade acadêmica, observa-se que os softwares atualmente disponíveis para essa finalidade, como Techlog, Petra e WellCAD, são caros, complexos e restritos ao ambiente corporativo, tornando-se inviáveis para o uso didático em instituições de ensino. O GeoLogViewer busca preencher essa lacuna ao oferecer uma plataforma gratuita e de código aberto aproximando o estudante das práticas profissionais da indústria do petróleo e gás. Em síntese, o GeoLogViewer visa democratizar o acesso à análise de perfis de poços no ambiente acadêmico, contribuindo para o desenvolvimento técnico e científico dos futuros engenheiros de petróleo, fortalecendo a integração entre teoria, prática e tecnologia.

### 1.1 Escopo do problema

O GeoLogViewer é um software acadêmico desenvolvido para auxiliar estudantes e professores de Engenharia de Petróleo na análise e visualização de dados geofísicos provenientes de arquivos LAS (Log ASCII Standard). O sistema foi concebido com o objetivo de tornar acessível o estudo de perfis de poços, promovendo o entendimento de parâmetros

petrofísicos e geológicos essenciais à caracterização de reservatórios e à formação técnica dos futuros engenheiros.

Diferentemente de softwares comerciais complexos e de alto custo, o GeoLogViewer oferece uma solução gratuita, leve e didática, especialmente voltada para o ambiente acadêmico. Sua interface intuitiva e suas funcionalidades permitem que o usuário compreenda e interprete dados de forma prática e interativa, fortalecendo o vínculo entre teoria e aplicação profissional.

Entre suas principais funcionalidades, destacam-se:

- Leitura estruturada de arquivos LAS, com identificação automática de seções e parâmetros geofísicos;
- Visualização gráfica interativa de curvas logarítmicas, como radiação gama (GR), densidade (RHOB) e porosidade neutrônica (NPHI);
- Ferramentas de cálculo, incluindo normalização de curvas, volume de argila, porosidade e saturação de água;
- Interface de uso simples, voltada para o aprendizado em disciplinas como Perfilagem e Petrofísica;
- Exportação de gráficos e arquivos CSV e PNG;
- Arquitetura modular e de código aberto, permitindo futuras expansões e adaptações por outras instituições.

O escopo do sistema abrange, portanto, a leitura, interpretação, cálculo e visualização de dados de poços, proporcionando um ambiente integrado de aprendizado técnico e computacional. O GeoLogViewer visa democratizar o acesso a ferramentas de análise geofísica, tornando o ensino de Engenharia de Petróleo mais interativo, prático e alinhado às tecnologias utilizadas na indústria.

## 1.2 Objetivos

Os objetivos deste projeto de engenharia são:

- Objetivo geral:
  - Desenvolver um software educacional completo para leitura, análise e visualização de dados geofísicos provenientes de arquivos LAS, proporcionando aos estudantes de Engenharia de Petróleo um ambiente interativo que favoreça a compreensão prática da interpretação de perfis de poços, relacionando parâmetros petrofísicos e suas implicações na caracterização de reservatórios, de forma acessível, intuitiva e tecnicamente consistente.
- Objetivos específicos:
  - Implementar a leitura estruturada de arquivos LAS.



- Garantir a plotagem de curvas de diferentes perfis de poços de petróleo.
- Sobreposição e comparação de registros para identificação de padrões e anomalias.
- Interpretar as curvas dos perfis de poços, indicando possíveis regiões de interesse.
- Destaque visual de intervalos de interesse;
- Ser capaz de exportar as imagens geradas dos perfis.
- Garantir o entendimento do estudante sobre a leitura e interpretação de perfis de poços de petróleo.
- Elaborar documentação técnica e manual de uso simplificado, com foco didático, explicando o funcionamento, a instalação e as principais funcionalidades do software.

### 1.3 Metodologia utilizada

A metodologia adotada para o desenvolvimento do GeoLogViewer foi estruturada com o objetivo de solucionar o problema central identificado: a falta de ferramentas acessíveis e didáticas para leitura e interpretação de perfis de poços em ambiente acadêmico. Assim, o processo metodológico articulou fundamentos conceituais da Engenharia de Petróleo com práticas de engenharia de software, de modo a garantir tanto rigor técnico quanto usabilidade.

Inicialmente, realizou-se uma análise conceitual do problema, identificando quais parâmetros geofísicos são mais relevantes no ensino de petrofísica e perfilagem de poços (como GR, RHOB e NPHI) e como eles se relacionam com processos físicos reais no interior do reservatório. Essa etapa incluiu o estudo da estrutura dos arquivos LAS e das interpretações geológicas associadas aos perfis, assegurando que a ferramenta desenvolvida refletisse corretamente o significado técnico das curvas representadas.

Em seguida, passou-se à modelagem da solução, definindo uma arquitetura modular capaz de separar claramente três dimensões essenciais:

1. Entrada e leitura dos dados (parser do arquivo LAS),
2. Visualização gráfica (representação interativa e intuitiva das informações),
3. Interpretação quantitativa e qualitativa das curvas.

Essa modelagem permitiu isolar responsabilidades, facilitando futuras melhorias e garantindo a clareza estrutural do código. A visualização das curvas foi planejada de modo a reforçar a compreensão dos conceitos subjacentes, permitindo ao usuário observar tendências, contrastes e variações geológicas a partir das representações gráficas. A fase de

implementação seguiu um ciclo incremental, no qual funcionalidades eram adicionadas gradualmente, testadas e ajustadas conforme objetivos pedagógicos. Essa estratégia garantiu que decisões de desenvolvimento fossem guiadas não apenas por viabilidade técnica, mas também pela clareza didática — ou seja, por como o aluno interage com os dados e compreende o fenômeno físico representado.

Por fim, a metodologia incorporou etapas de validação prática, envolvendo testes com estudantes em exercícios supervisionados, permitindo observar como a ferramenta era utilizada e quais elementos poderiam dificultar ou facilitar o aprendizado. A partir desse retorno, ajustes foram feitos tanto na interface quanto na forma de apresentação dos dados.

Em síntese, a metodologia combinou:

- Entendimento conceitual do fenômeno geofísico,
- Modelagem computacional clara e modular,
- Desenvolvimento incremental orientado ao usuário, e
- Validação pedagógica e prática,

garantindo que o software não apenas funcionasse tecnicamente, mas servisse efetivamente como instrumento de aprendizado e formação em Engenharia de Petróleo.

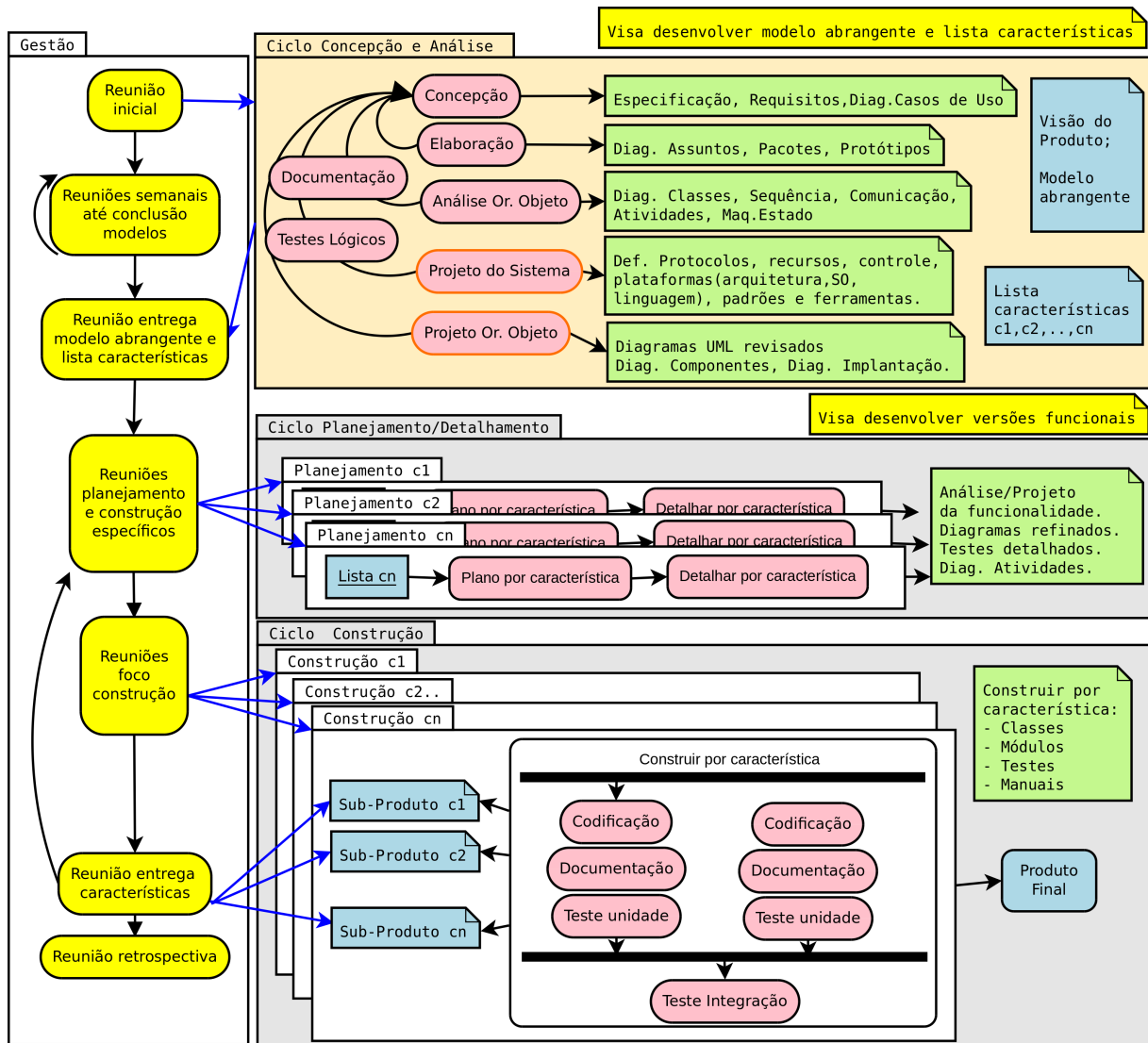


Figura 1.1: Etapas para o desenvolvimento do software - *projeto de engenharia*

# Capítulo 2

## Concepção

Apresenta-se neste capítulo a concepção e a especificação do GeoLogViewer, um sistema computacional destinado à leitura, interpretação e visualização de dados geofísicos provenientes de arquivos LAS (Log ASCII Standard). O projeto surge da necessidade significativa observada no contexto acadêmico da Engenharia de Petróleo: a ausência de ferramentas acessíveis, intuitivas e adequadas ao ensino para a análise de perfis de poços, recurso essencial na formação de engenheiros de reservatórios, engenheiros de poço e geofísicos.

Durante a graduação, é comum que estudantes tenham contato com dados reais de perfis de poços, mas enfrentem dificuldades ao tentar analisá-los devido ao uso restrito de softwares comerciais como Techlog, WellCAD e Petra — ferramentas robustas, porém caras, complexas e pouco acessíveis no ambiente universitário. Muitas vezes, o aprendizado prático torna-se limitado a exemplos estáticos, capturas de tela ou atividades teóricas que não proporcionam ao aluno a experiência de manipular e visualizar curvas em tempo real. O GeoLogViewer busca suprir essa lacuna por meio de uma solução leve, gratuita e voltada exclusivamente ao uso didático.

A concepção do sistema considerou os seguintes pontos fundamentais:

- Leitura estruturada de arquivos LAS, identificando metadados, curvas logarítmicas e unidades de medida;
- Visualização gráfica clara e interativa das curvas geofísicas, como GR, RHOB, NPHI e outras disponíveis no arquivo;
- Ferramentas de navegação e interpretação, incluindo zoom vertical, destaque de regiões de interesse e análise básica de tendências;
- Exportação de gráficos e captura das curvas visualizadas em formatos de imagem para uso em relatórios e atividades acadêmicas;
- Interface intuitiva, adequada para estudantes que ainda estão iniciando seu contato com softwares geológicos ou petrofísicos;

- Estrutura modular, permitindo que o sistema seja expandido com novos logs, algoritmos de interpretação ou cálculos petrofísicos.

A especificação do sistema engloba funcionalidades integradas que respondem em tempo real às ações do usuário, trabalhando a partir dos dados brutos presentes nos arquivos LAS. A modelagem conceitual e computacional foi realizada por meio de diagramas UML — incluindo casos de uso, classes, pacotes e interações — que definem a lógica interna do software, seus módulos principais e a comunicação entre eles. A implementação utiliza C++, garantindo desempenho elevado na manipulação de grandes volumes de dados e proporcionando uma interface gráfica estruturada, responsiva e visualmente clara.

O projeto foi concebido de forma flexível e expandível, permitindo sua futura adaptação a outras disciplinas, cursos ou contextos acadêmicos que utilizem dados de perfilagem, bem como sua evolução para incluir cálculos petrofísicos mais avançados, integração com bancos de dados e ferramentas de análise estatística. Dessa forma, o GeoLogViewer não se limita a uma solução momentânea, mas configura-se como uma plataforma educacional contínua, com potencial de crescimento e adoção em diferentes instituições de ensino.

## 2.1 Nome do sistema/produto

| Nome                          | GeoLogViewer                                                                                                                                                                                                 |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Componentes principais</b> | Módulo de Leitura de Arquivos LAS, Módulo de Processamento e Análise, Módulo de Visualização Gráfica, Módulo de Interpretação e Módulo de Exportação.                                                        |
| <b>Missão</b>                 | A missão do GeoLogViewer é democratizar o acesso ao estudo e à interpretação de perfis de poços no ambiente acadêmico, oferecendo uma ferramenta educacional gratuita, intuitiva e tecnicamente consistente. |

## 2.2 Especificação

O GeoLogViewer é um software educacional interativo desenvolvido para auxiliar estudantes de Engenharia de Petróleo, Geofísica e áreas correlatas na leitura, análise e visualização de perfis de poços registrados em arquivos LAS (Log ASCII Standard). O sistema oferece uma plataforma acessível e intuitiva para manipulação de dados geofísicos reais, promovendo o aprendizado prático de conceitos como radiação gama, densidade, porosidade, litologia e caracterização de formações.

O software permite ao usuário carregar arquivos LAS de diferentes origens, interpretar automaticamente suas seções (versão do arquivo, informações do poço, parâmetros, cur-

vas, dados), e visualizar graficamente os perfis contidos. A leitura é realizada diretamente a partir do arquivo original, respeitando sua estrutura, metadados, unidades e cabeçalhos, garantindo fidelidade e consistência dos dados visualizados.

A interface gráfica possibilita a visualização simultânea de múltiplas curvas, distribuídas em trilhas ajustáveis, com funcionalidades como zoom vertical, deslocamento, comparação lado a lado e personalização dos eixos. O usuário pode destacar regiões de interesse, exportar imagens dos gráficos gerados e alternar entre diferentes conjuntos de curvas disponíveis no arquivo.

O sistema também inclui ferramentas básicas de análise, permitindo ao usuário identificar variações significativas nas curvas, observar tendências e marcar possíveis zonas de reservatório ou camadas selantes, reforçando a compreensão dos fenômenos geológicos e petrofísicos representados pelos dados.

## 2.3 Requisitos

### 2.3.1 Requisitos funcionais

Apresenta-se a seguir os requisitos funcionais.

|              |                                                                                                                             |
|--------------|-----------------------------------------------------------------------------------------------------------------------------|
| <b>RF-01</b> | O sistema deve permitir ao usuário carregar arquivos LAS, reconhecendo automaticamente suas seções.                         |
| <b>RF-02</b> | O sistema deve identificar todas as curvas presentes no arquivo, incluindo nomes, descrições, unidades e valores numéricos. |
| <b>RF-03</b> | O sistema deve plotar as curvas selecionadas em trilhas individuais, permitindo visualização simultânea de múltiplos logs.  |
| <b>RF-04</b> | O sistema deve oferecer ferramentas de zoom, rolagem e ajuste de escalas nas trilhas.                                       |
| <b>RF-05</b> | O usuário deve poder escolher quais curvas deseja visualizar e alterar essa seleção dinamicamente.                          |
| <b>RF-06</b> | O sistema deve exportar os gráficos gerados e permitir salvar os dados processados em formato padrão.                       |

### 2.3.2 Requisitos não funcionais

|               |                                                                                                                                            |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <b>RNF-01</b> | O sistema deve possuir uma interação com o usuário intuitiva e acessível para estudantes com pouca familiaridade com softwares geofísicos. |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------|

|               |                                                                                                                         |
|---------------|-------------------------------------------------------------------------------------------------------------------------|
| <b>RNF-02</b> | O sistema deve ser capaz de carregar e visualizar arquivos LAS de tamanho médio (até alguns MB) em poucos segundos.     |
| <b>RNF-03</b> | O software deve ser compatível com sistemas Windows e Linux.                                                            |
| <b>RNF-04</b> | Os gráficos produzidos devem representar fielmente os dados do arquivo LAS, sem distorção ou alteração não intencional. |
| <b>RNF-05</b> | O sistema deve ser disponibilizado como projeto open-source, com licença que permita modificação e redistribuição.      |

## 2.4 Casos de uso

A Tabela 2.1 mostra a descrição de um caso de uso.

Tabela 2.1: Caso de uso 1

|                        |                                                                                                                                                                                                                                                                                                 |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Nome do caso de uso:   | Carregar Arquivo LAS                                                                                                                                                                                                                                                                            |
| Resumo/descrição:      | Permite ao usuário selecionar e carregar um arquivo LAS para leitura, interpretação e visualização das curvas geofísicas.                                                                                                                                                                       |
| Etapas:                | <ol style="list-style-type: none"> <li>1. Escolher o arquivo LAS desejado</li> <li>2. Realizar a leitura dos dados contidos no arquivo</li> <li>3. Selecionar as curvas de interesse de plot</li> <li>4. Plotar os perfis do poço</li> <li>5. Interpretar os intervalos de interesse</li> </ol> |
| Cenários alternativos: | <ol style="list-style-type: none"> <li>1. Arquivo inválido ou corrompido: o sistema exibe mensagem de erro e solicita nova seleção.</li> <li>2. Formato desconhecido: o sistema alerta que o arquivo não segue o padrão LAS suportado.</li> </ol>                                               |

### 2.4.1 Diagrama de caso de uso geral

O diagrama de caso de uso geral da Figura 2.1 representa uma visão global das interações fundamentais entre o usuário e o sistema GeoLogViewer, destacando as principais funcionalidades necessárias para leitura, seleção, interpretação e plotagem de curvas provenientes de arquivos LAS. Trata-se de um diagrama de caso de uso de alto nível, cujo objetivo é ilustrar as ações essenciais que permitem ao usuário manipular dados geofísicos de forma estruturada dentro do software. No centro do diagrama encontra-se o ator Usuário, representando estudantes, professores ou qualquer indivíduo interessado em visualizar perfis de poços. Um ator secundário, Gnuplot, aparece no diagrama representando uma possível ferramenta ou biblioteca gráfica que o sistema utiliza para gerar os gráficos. Ele

participa especificamente do caso de uso Plotagem das curvas, indicando que a geração visual depende de componentes externos especializados.

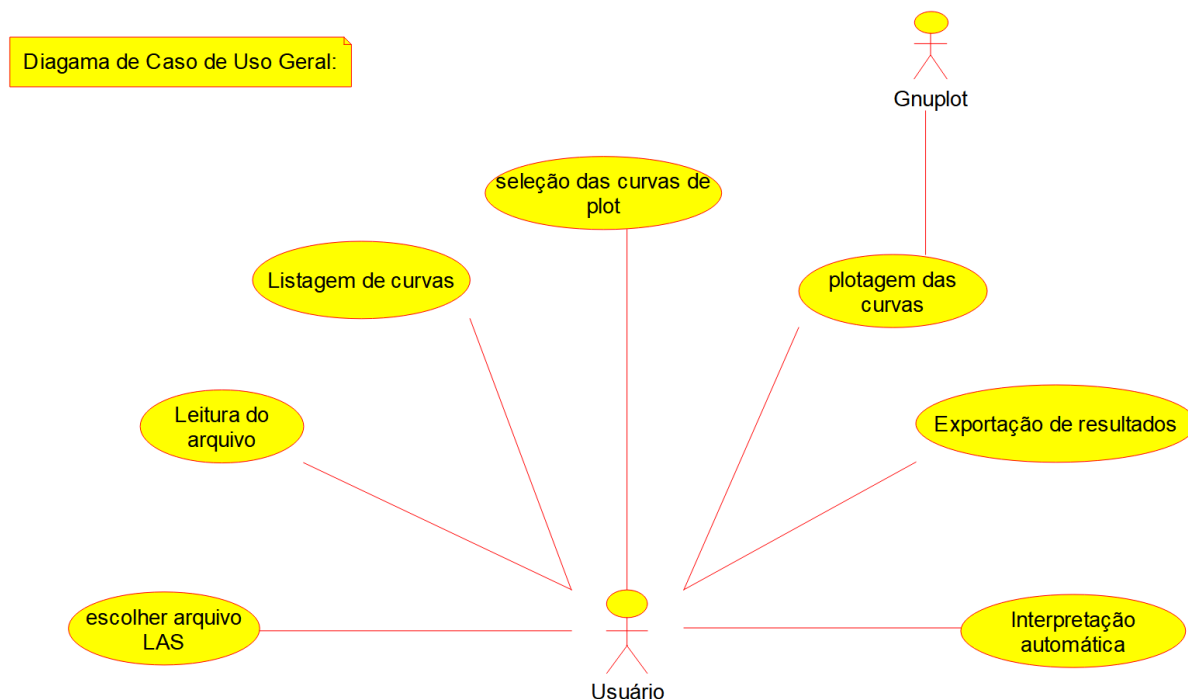


Figura 2.1: Diagrama de caso de uso

## 2.4.2 Diagrama de caso de uso específico

### Diagrama de Caso de Uso Específico de Comparação entre as Curvas

O diagrama de caso de uso específico da Figura 2.2 apresenta as opções que o usuário possui dentro do GeoLogViewer para analisar curvas geofísicas de maneiras distintas, conforme o objetivo da interpretação. No centro do diagrama encontra-se o ator Usuário, representando o estudante ou profissional que manipula o software. A partir dele, ramificam-se três casos de uso principais, cada um correspondendo a uma forma diferente de realizar comparações visuais entre as curvas registradas no arquivo LAS.

O primeiro caso de uso, Curvas Sobrepostas e Normalizadas, refere-se à funcionalidade na qual o sistema exibe duas ou mais curvas simultaneamente em uma mesma trilha, após normalização dos valores. Esse método é particularmente útil quando as curvas possuem escalas muito distintas, pois a normalização coloca todas em uma escala comum, permitindo uma visualização comparativa mais clara e facilitando a identificação de padrões conjuntos, correlações e tendências geológicas.

O segundo caso de uso, Curvas Sobrepostas e Não-Normalizadas, mostra que o usuário pode também comparar curvas sobrepostas mantendo suas unidades e escalas originais. Nessa forma de visualização, preserva-se a magnitude real dos logs, o que é importante para análises que dependem diretamente dos valores absolutos das curvas, como identifi-



cação de limites petrofísicos, avaliação de cutoffs ou observação de anomalias específicas dentro dos registros.

Por fim, o caso de uso Curvas Não-Sobrepostas e Não-Normalizadas representa a forma clássica de apresentação de perfis de poços, na qual cada curva é exibida em sua trilha individual, com sua escala original e sem qualquer tipo de ajuste ou sobreposição. Esse modo de visualização é amplamente utilizado em ambientes acadêmicos e profissionais, pois permite a leitura isolada de cada parâmetro geofísico, facilitando a identificação de variações verticais, mudanças de litologia e interpretação estratigráfica.

De maneira geral, o diagrama evidencia que o GeoLogViewer oferece diferentes categorias de comparação que atendem tanto a análises qualitativas quanto quantitativas, possibilitando ao usuário escolher o tipo de visualização mais adequado às necessidades de estudo ou interpretação. A diversidade de opções demonstra o caráter didático e flexível do software, permitindo que estudantes explorem as curvas de poço sob várias perspectivas e compreendam melhor a relação entre diferentes logs utilizados na Engenharia de Petróleo e na Geofísica.

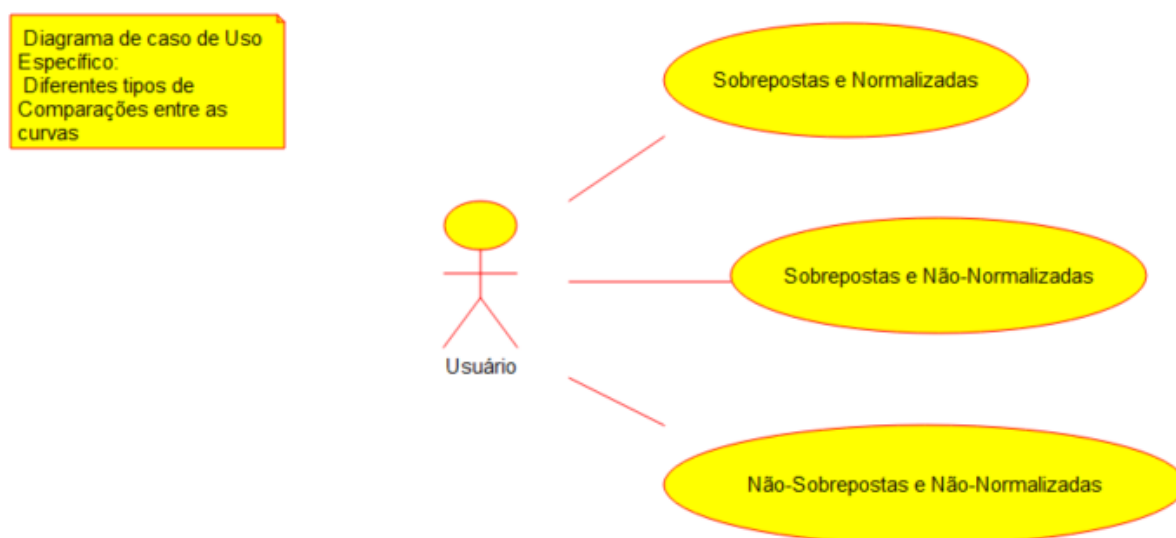


Figura 2.2: Diagrama de caso de uso específico de comparação entre curvas

### Diagrama de Caso de Uso Específico de tipos de plotagem

O diagrama de caso de uso específico da Figura 2.3 as formas pelas quais o usuário pode interagir com o GeoLogViewer para gerar diferentes representações gráficas das curvas geofísicas contidas em arquivos LAS. No centro do diagrama encontra-se o ator Usuário, responsável por iniciar cada uma das ações de plotagem disponíveis no sistema. A partir desse ator, quatro casos de uso são destacados, cada um representando um tipo particular de visualização que o software oferece.

O primeiro caso de uso, Plot Curve, refere-se à funcionalidade que permite ao usuário selecionar e visualizar uma única curva de forma isolada, representada individualmente em uma trilha gráfica. Essa opção é útil quando se deseja realizar uma análise mais detalhada

de um log específico, como radiação gama ou densidade. Em seguida, o caso de uso Plot Multiple representa a capacidade do sistema de permitir a visualização simultânea de múltiplas curvas, possibilitando ao usuário observar a relação entre diferentes logs no mesmo intervalo de profundidade, recurso essencial para interpretações petrofísicas e correlações litológicas.

O caso de uso Plot Selected Tracks indica que o sistema também permite ao usuário definir quais trilhas deseja visualizar, compondo um layout personalizado de análise. Essa funcionalidade aproxima o GeoLogViewer de softwares profissionais usados na indústria, nos quais o intérprete organiza manualmente as trilhas de interesse conforme os objetivos da avaliação. Por fim, o caso de uso Plot Compare Curves representa a funcionalidade destinada à comparação formal entre curvas, permitindo ao usuário avaliar semelhanças, contrastes, padrões comuns e divergências entre logs distintos, seja por sobreposição visual, ajustes de escala ou outros mecanismos internos de comparação.

De forma geral, o diagrama evidencia que o GeoLogViewer oferece um conjunto diversificado e flexível de modos de plotagem, permitindo que o usuário escolha a abordagem mais adequada à sua análise. Assim, o diagrama demonstra que o sistema atende às necessidades típicas do ambiente acadêmico de Engenharia de Petróleo, fornecendo múltiplas ferramentas para visualização gráfica e interpretação preliminar de dados geofísicos de perfis de poço.

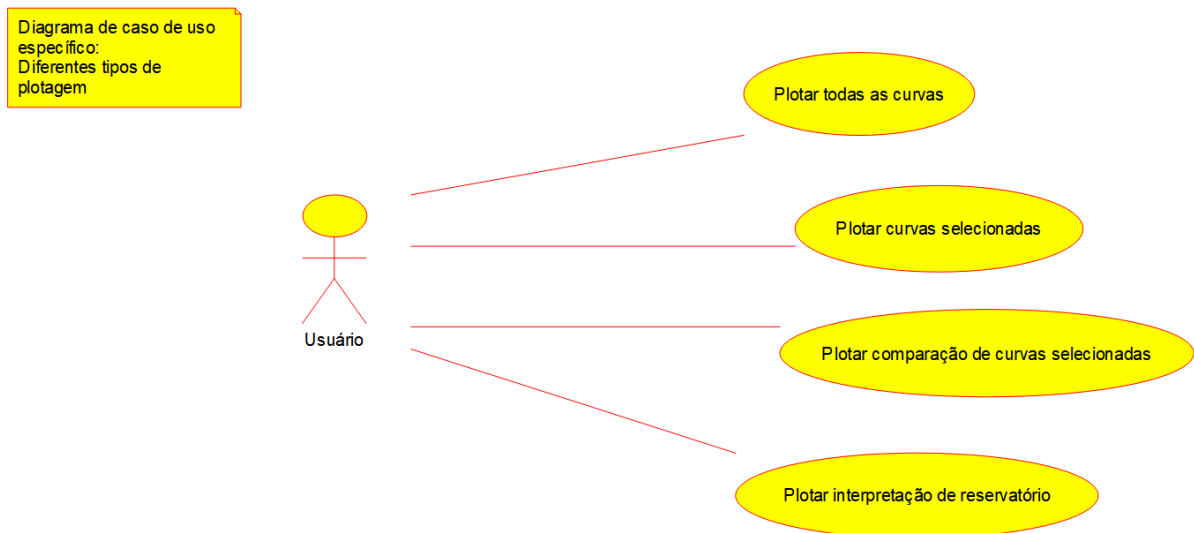


Figura 2.3: Diagrama de caso de uso específico de tipos de plotagem

# Capítulo 3

## Elaboração

Neste capítulo será apresentada a elaboração do software, que envolve a definição teórica, os conceitos de domínio, a organização dos pacotes e os algoritmos adicionais relacionados ao desenvolvimento do sistema proposto.

### 3.1 Análise de domínio

A análise de domínio é uma etapa fundamental da elaboração de um projeto, na qual se faz necessário entender e delimitar os conceitos principais que orientam a construção do sistema.

O presente projeto está relacionado a 5 conceitos fundamentais:

1. Geofísica e perfis de poço:

O software está orientado à manipulação de perfis de poço em formato LAS, amplamente utilizados na indústria do petróleo. Esses arquivos contêm curvas de propriedades físicas (como radiação gama, densidade de formação e porosidade neutrônica), fundamentais para caracterizar reservatórios e avaliar potencial produtivo.

2. Geologia de reservatórios:

A compreensão dos perfis de poço é indissociável da análise geológica. As curvas obtidas a partir dos arquivos LAS permitem interpretar propriedades das rochas, como litologia, porosidade e saturação, essenciais para estudos de correlação estratigráfica e para a modelagem de reservatórios.

3. Estruturas de dados e processamento de arquivos:

O formato LAS (Log ASCII Standard) é padronizado, mas pode apresentar variações dependendo da ferramenta de aquisição e da empresa responsável pelo levantamento. O software deve ser capaz de ler, validar e organizar as informações em estruturas de dados consistentes, permitindo seu processamento e manipulação de forma eficiente.

#### 4. Programação:

O paradigma orientado a objetos (POO) é adotado no projeto para modularizar o código em classes e objetos responsáveis por diferentes partes do sistema (entrada de dados, processamento, visualização, exportação). Esse paradigma facilita a manutenção, reutilização de código e futuras expansões, como a incorporação de novos formatos de dados além do LAS.

#### 5. Visualização gráfica:

A etapa final do processamento consiste em disponibilizar ao usuário representações gráficas claras e interativas dos perfis de poço. Isso envolve o uso de bibliotecas gráficas capazes de gerar trilhos, sobrepor curvas, ajustar escalas e permitir zoom em intervalos específicos. A visualização intuitiva é essencial para que engenheiros e estudantes possam interpretar os dados de forma rápida e confiável.

## 3.2 Formulação teórica

### 3.2.1 Fundamentos da Perfilagem de Poço

A perfilagem geofísica de poços é a prática de efetuar um registro detalhado das formações geológicas atravessadas por uma perfuração. Nesse método, sondas equipadas com diversos sensores são deslocadas verticalmente no poço (geralmente por cabo ou integradas à sonda de perfuração – LWD), registrando parâmetros físicos das rochas em função da profundidade. Esses parâmetros podem incluir resistividade, radioatividade natural (raios gama), densidade, porosidade, tempo de trânsito sônico, entre outros (Rider,1996).

Tradicionalmente, as medições são obtidas após a perfuração de um trecho, baixando-se uma sonda com guincho até o fundo e depois recolhendo-a lentamente, de modo que as leituras sejam registradas continuamente enquanto a sonda ascende pela coluna de sondagem. Atualmente, é comum o uso de LWD (Logging While Drilling), em que sensores integrados à ferramenta de perfuração coletam dados em tempo real durante a própria perfuração. Esses dados são então armazenados em arquivos digitais (tipicamente no padrão LAS) para análise posterior. A combinação de múltiplos perfis de poço possibilita uma caracterização detalhada das camadas perfuradas, muito além do que seria obtido apenas com amostras físicas (Schlumberger,1991)

A alta resolução dos perfis de poço é crucial na engenharia de petróleo, pois permite identificar com precisão a profundidade e espessura de camadas porosas ou impermeáveis no reservatório. Por exemplo, variações bruscas nos perfis de resistividade e porosidade podem indicar zonas saturadas por hidrocarbonetos ou água, enquanto níveis elevados de raios gama sinalizam camadas argilosas indesejáveis. Essas informações auxiliam geocientistas e engenheiros a recomendar completações de poço adequadas (como intervalos produtivos versus selos) ou mesmo o abandono seguro de trechos sem potencial produtivo.

Em suma, a perfilagem de poços é ferramenta central na caracterização de reservatórios, complementando outros métodos geofísicos e geológicos para delineamento estratigráfico e avaliação de reservas (Asquith,2004).

### 3.2.2 Propriedades Físicas Medidas por Perfilagem

Os perfis de poço registram diversas propriedades físicas das rochas sedimentares. Dentre os perfis mais comuns estão: perfil de raios gama (mede a radioatividade natural da rocha), perfil de densidade (gama-gama), perfil de porosidade (em geral nêutron, ou densidade calibrada para porosidade) e perfil sônico (tempo de trânsito de ondas acústicas). Cada perfil baseia-se em um princípio físico distinto: por exemplo, o perfil de raios gama detecta as emissões de elementos radioativos naturais (como K, U e Th) na formação, usado sobretudo para distinguir camadas argilosas de arenitos mais “limpos” (Rider,1996). Já perfis de densidade baseados na atenuação de fótons gama permitem estimar a massa por unidade de volume, e perfis nêutron avaliam a quantidade de hidrogênio (e portanto de porosidade) na rocha. O perfil sônico mede o tempo de propagação de um pulso acústico através da formação, sendo útil para inferir porosidade e elasticidade. Além destes, perfis de resistividade elétrica e de potencial espontâneo (SP) são frequentemente coletados: a resistividade indica a capacidade da rocha conduzir corrente (alta em arenitos saturados com óleo), enquanto o SP reflete diferenças de salinidade entre lama e fluido de formação. A Figura abaixo exemplifica o trabalho realizado para se adquirir os perfis geofísicos de um poço de petróleo.

Figura 3.1: Representação esquemática de uma descida de uma ferramenta de perfilagem



Fonte: (Rider,1996)

O perfil de raios gama é quase sempre o primeiro analisado, pois sinaliza rapidamente

variações litológicas. Equipamentos de contagem gama na sonda detectam fótons de alta energia liberados pela desintegração radioativa natural. Valores altos no perfil de raios gama geralmente indicam folhelhos ricos em argila, enquanto valores baixos sugerem arenitos ou carbonatos de baixa radioatividade.

Os perfis de densidade e porosidade são complementares: sondas de densidade (gama-gama) emitem radiação gama e medem sua atenuação para calcular a densidade da rocha. Perfis de porosidade (por exemplo, sondas nêutron ou combinação de densidade-nêutron) estimam a fração de vazio no volume rochoso. Em rochas carbonáticas calcárias, por exemplo, a diferença entre a densidade lítica esperada e a densidade medida dá a porosidade; em arenitos, sondas nêutron são calibradas para saturação de hidrocarboneto. Esses registros permitem avaliar diretamente a porosidade efetiva e inferir saturação de fluidos no reservatório (Ellis, 2007).

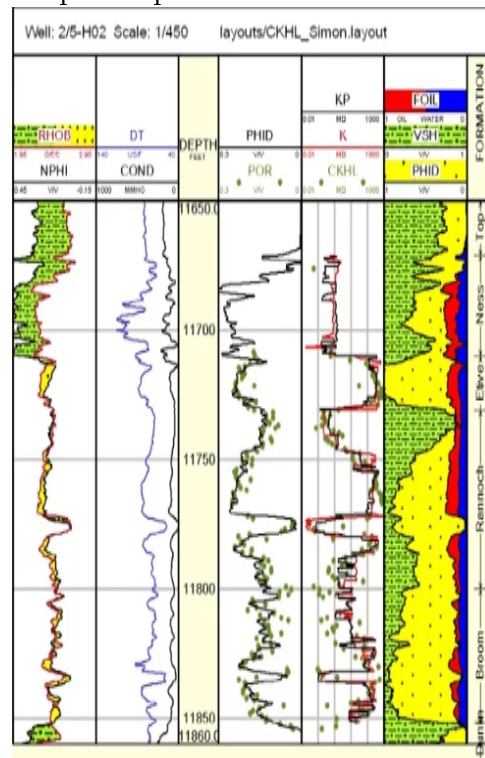
O perfil sônico (acústico) mede o tempo de trânsito de ondas de pressão geradas pela sonda. Rochas consolidadas (arenitos cimentados, calcários duros) apresentam tempos de trânsito menores (ondas viajam mais rápido) do que rochas macias ou porosas. A partir do tempo de trânsito sônico e da densidade, pode-se inferir porosidade acústica e mesmo parâmetros mecânicos da rocha. Já os perfis de resistividade são obtidos com eletrodos elétricos ou bobinas indutivas na sonda: rochas saturadas com óleo são mais resistivas que as saturadas por água salina. O perfil de potencial espontâneo (SP), por fim, mede uma diferença de potencial natural gerada quando lama de perfuração (diluída) encontra água de formação (geralmente salina), ajudando a delimitar camadas permeáveis (que geram maior sinal SP).

Além desses, há ainda perfis de calibre (que registram o diâmetro do poço, indicando cavernas ou incrustações), de imagem (resistiva ou acústica, que geram imagens 360° das paredes do poço) e dipmeter (para orientação das camadas). Em conjunto, os múltiplos perfis permitem calcular frações de argila, saturação de água (curva de saturação) e outros parâmetros petrofísicos essenciais à avaliação do reservatório.

### 3.2.3 Interpretação Geológica a partir de Perfis

A interpretação de perfis de poço visa identificar litologias, zonas potenciais de produção e correlações estratigráficas. Por exemplo, altos valores de raios gama associados a resistividades baixas podem revelar camadas argilosas impermeáveis, enquanto declínios abruptos de resistividade combinados com picos de porosidade sugerem arenitos saturados de hidrocarbonetos. A análise integrada dos perfis – por exemplo, cruzando curvas de densidade e nêutron – permite estimar a porosidade efetiva e a saturação de fluidos de cada camada, diferenciando here arenitos porosos de camadas compactas (Asquith, 2004). A Figura abaixo exemplifica perfis típicos de poços, incluindo raios gama, porosidade e resistividade, ilustrando como múltiplas curvas são alinhadas versus profundidade.

Figura 3.2: Exemplo de perfil usualmente utilizado na indústria



Fonte: (Petrowiki,2014)

A correlação estratigráfica entre poços é uma aplicação fundamental dos logs. Como apontado por estudos em campos petrolíferos, “a correlação estratigráfica de poços é de suma importância na indústria do petróleo”, pois define a continuidade lateral das camadas produtivas. Isso significa que intervalos identificados como reservatório em um poço podem ser localizados em outros poços da mesma bacia utilizando padrões semelhantes nos perfis. Assim, a partir da comparação de curvas de raios gama, resistividade, porosidade e densidade de poços adjacentes, geólogos podem construir um modelo estratigráfico contínuo. No caso ilustrado abaixo, os dados de raios gama, resistividade e porosidade de três poços foram correlacionados para demonstrar a continuidade do reservatório do Campo de Namorado.

Ao correlacionar camadas em diferentes poços, também é possível mapear heterogeneidades e orientar estudos sísmicos e de engenharia. Além disso, perfis de poço calibrados com dados de testemunhos ou amostras geológicas reforçam a interpretação litológica: por exemplo, uma mudança súbita no perfil sônico pode corresponder a uma mudança de facies sedimentar, enquanto uma zona de alta densidade e baixa porosidade sugere preenchimento calcário ou compactação. Em síntese, a interpretação de perfis de poço fornece um detalhamento petrofísico que, integrado com outros dados de reservatório, permite caracterizar unidades de interesse econômico e otimizar estratégias de produção (Ellis,2007).

### 3.2.4 Técnicas de Aquisição e Formato de Arquivo LAS

Os dados de perfilagem de poço são coletados por meio de procedimentos bem definidos. Em poços perfurados, as medições em poço aberto normalmente ocorrem após estabilização da lama, usando sondas wireline: após a perfuração, a sonda de perfilagem é inserida pelo poço, registra as medidas em profundidade (track), e é então recolhida. Complementarmente, o logging while drilling (LWD) permite adquirir perfis enquanto a perfuração prossegue: sensores instalados no fundo de sonda medem continuamente as propriedades das rochas conforme o furo avança. O LWD é especialmente vantajoso em poços profundos ou desviados, pois dispensa o tempo de espera pela finalização do furo. Em ambos os casos, o resultado da perfilagem é um conjunto de curvas quantitativas associadas a profundidades no poço, que refletem diretamente as propriedades geofísicas calculadas pelos instrumentos de medição (Schlumberger, 1991).

Após a aquisição, os dados de perfilagem são organizados no padrão Log ASCII Standard (LAS) – um formato de arquivo texto amplamente usado na indústria de óleo e gás. O arquivo LAS facilita o intercâmbio de dados entre equipamentos e softwares de interpretação. Ele é composto por seções padronizadas (geralmente iniciadas por ~Version, ~Well, ~Curve, ~Parameter e ~Ascii), onde cada seção reúne informações específicas: por exemplo, ~Well contém dados do poço (profundidade total, nome do poço, informações de localização, etc.), ~Curve lista as curvas registradas (com nome, unidade e descrição), e a seção ~Ascii abriga os valores medidos em formato tabulado (cada linha corresponde a uma profundidade e às medições de todas as curvas naquele ponto). A padronização do LAS garante que diferentes softwares de visualização e análise possam ler os mesmos dados sem ambiguidades.

Em termos práticos, o arquivo LAS armazena cada amostra de dado como um registro de profundidade e valores em todas as curvas, permitindo a sobreposição e comparação precisa de várias curvas de múltiplos poços em um mesmo ambiente de interpretação. Por exemplo, ao carregar um arquivo LAS em um software de log-plot, o interprete vê simultaneamente as curvas de raios gama, resistividade, porosidade, etc., alinhadas por profundidade, facilitando a análise integrada. Dessa forma, o formato LAS é fundamental para preservar a integridade dos perfis medidos e viabilizar a manipulação e visualização dos dados de perfilagem por engenheiros e geocientistas (Asquith, 2004).

### 3.2.5 Estimativas Petrofísicas

#### Volume de Argila a partir do Perfil de Gamma Ray

A interpretação do volume de argila a partir do perfil de Gamma Ray baseia-se no fato de que minerais argilosos apresentam maior radioatividade natural devido à presença de potássio, urânio e tório. Assim, formações mais argilosas emitem mais radiação detectada pela ferramenta de poço.



Para obter um valor adimensional entre 0 e 1, o sinal é inicialmente normalizado:

$$GR_{norm} = \frac{GR - GR_{min}}{GR_{max} - GR_{min}} \quad (3.1)$$

onde:

- $GR$  é o valor lido no perfil
- $GR_{min}$  representa a zona mais limpa
- $GR_{max}$  representa a zona mais argilosa

### Modelo Linear

O modelo linear assume que a radioatividade cresce de forma proporcional à fração de argila na rocha.

Dessa forma:

$$Vsh = GR_{norm} \quad (3.2)$$

Esse modelo é indicado quando a resposta do perfil não apresenta efeitos significativos de cimentação ou compactação, sendo comum em arenitos pouco complexos.

### Modelos de Larionov

Larionov propôs modelos empíricos que corrigem a não linearidade observada entre o Gamma Ray e a fração de argila em rochas consolidadas. Esses modelos foram desenvolvidos a partir de estudos experimentais comparando perfis de poço com análises de testemunhos geológicos.

Larionov para Rochas Terciárias

$$Vsh = 0.083(2^{3,7 \cdot GR_{norm}} - 1) \quad (3.3)$$

Esse modelo considera que formações jovens, como rochas terciárias, apresentam maior influência de minerais com radioatividade dispersa e comportamento não linear entre GR e argilosidade.

Larionov para Rochas Pré-terciárias

$$Vsh = 0.33(2^{2,0 \cdot GR_{norm}} - 1) \quad (3.4)$$

Formações antigas e mais consolidadas têm menor dispersão radioativa, e essa formulação corrige a superestimativa que ocorre quando se usa o modelo linear ou o modelo típico para rochas jovens.

### Porosidade Calculada pelo Perfil de Densidade

A estimativa de porosidade por densidade deriva diretamente do modelo de mistura volumétrica entre matriz sólida e fluido. A densidade total medida pode ser expressa como:

$$\rho_b = \phi \rho_{fluid} + (1 - \phi) \rho_{matrix} \quad (3.5)$$

Rearranjando, obtém-se a forma clássica:

$$\phi_d = \frac{\rho_{matrix} - \rho_b}{\rho_{matrix} - \rho_{fluid}} \quad (3.6)$$

onde:

- $\rho_b$  é a densidade da formação medida pela ferramenta
- $\rho_{matrix}$  é a densidade do grão mineral (quartzo, calcita ect.)
- $\rho_{fluid}$  é a densidade do fluido presente nos poros

Esse modelo vem diretamente da lei das misturas e assume que não há efeitos significativos de gás ou minerais pesados distorcendo a leitura.

### Porosidade Combinada Neutrão-Densidade

O perfil de neutrões responde principalmente à presença de hidrogênio, enquanto o perfil de densidade mede a massa específica da formação. Como cada ferramenta apresenta limitações distintas, especialmente na presença de gás, é comum combinar as duas de forma simples:

$$\phi_{ND} = \frac{\phi_n + \phi_d}{2} \quad (3.7)$$

Essa combinação surge de técnicas clássicas de correlação cruzada entre logs, demonstrando que a média simples fornece um valor mais robusto para arenitos e carbonatos, reduzindo distorções individuais das ferramentas.

### Saturação de Água pelo Modelo de Archie

A equação de Archie foi desenvolvida empiricamente a partir de experimentos realizados por G.E. Archie em amostras de rochas saturadas com água salina. Ela estabelece a relação entre resistividade elétrica da rocha, porosidade e saturação de água em meios porosos limpos, sem argilas condutivas.

O ponto de partida da formulação é:

$$R_t = a \frac{R_w}{\phi^m S_w^n} \quad (3.8)$$

onde:

- $R_t$  é a resistividade da formação
- $R_w$  é a resistividade da água nos poros
- $\phi$  é a porosidade
- $S_w$  é a saturação de água
- $a$  é o fator de tortuosidade
- $m$  é o expoente de cimentação
- $n$  é o expoente de saturação

Isolando  $S_w$ :

$$S_w = \left( \frac{aR_w}{\phi^m R_t} \right)^{\frac{1}{n}} \quad (3.9)$$

O modelo é amplamente utilizado devido à sua base experimental sólida e simplicidade matemática. Ele descreve adequadamente rochas limpas onde a condução elétrica ocorre predominantemente no fluido intersticial.

### 3.3 Identificação de pacotes – assuntos

- Pacote Análise de Perfis: Este pacote é o núcleo principal do sistema, responsável por integrar os demais pacotes relacionados à leitura, plotagem e interpretação dos perfis. É a partir dele que os módulos específicos são organizados para permitir uma análise completa e estruturada dos dados de perfilagem.
- Pacote Leitura de Arquivos: Este pacote é dedicado ao processamento e entrada de dados, especialmente arquivos no formato LAS, que são amplamente utilizados na perfilagem de poços. Ele garante a correta importação das informações brutas para que possam ser posteriormente analisadas.
- Pacote Arquivos LAS: Representa o subconjunto especializado dentro do pacote de leitura de arquivos. É responsável por manipular e estruturar os dados provenientes deste tipo de arquivo, assegurando a padronização e consistência na utilização dos dados de perfilagem.
- Pacote Plotagem: Este pacote agrupa as funcionalidades relacionadas à visualização gráfica dos perfis. Sua função é transformar os dados numéricos em representações visuais claras, permitindo que o usuário compreenda de maneira intuitiva as variações e correlações entre os parâmetros analisados.

- Pacote Gnuplot: É um subpacote associado à plotagem, responsável por fornecer os recursos de geração de gráficos por meio da integração com a ferramenta Gnuplot. Ele automatiza a construção de curvas, tracks compostos e representações visuais necessárias à análise.
- Pacote Interpretação: Este pacote trata da etapa analítica do processo, na qual os perfis já processados e plotados são avaliados em termos de parâmetros petrofísicos e geológicos. Seu objetivo é oferecer informações sobre propriedades como porosidade, saturação e resistividade.
- Pacote Parâmetros de Perfil: Subconjunto do pacote de interpretação, é responsável por armazenar, organizar e manipular os diferentes parâmetros derivados dos perfis. Ele fornece a base quantitativa necessária para a interpretação detalhada e consistente dos dados de perfilagem.

### 3.4 Diagrama de pacotes – assuntos

O diagrama de pacotes é apresentado na Figura 3.3.

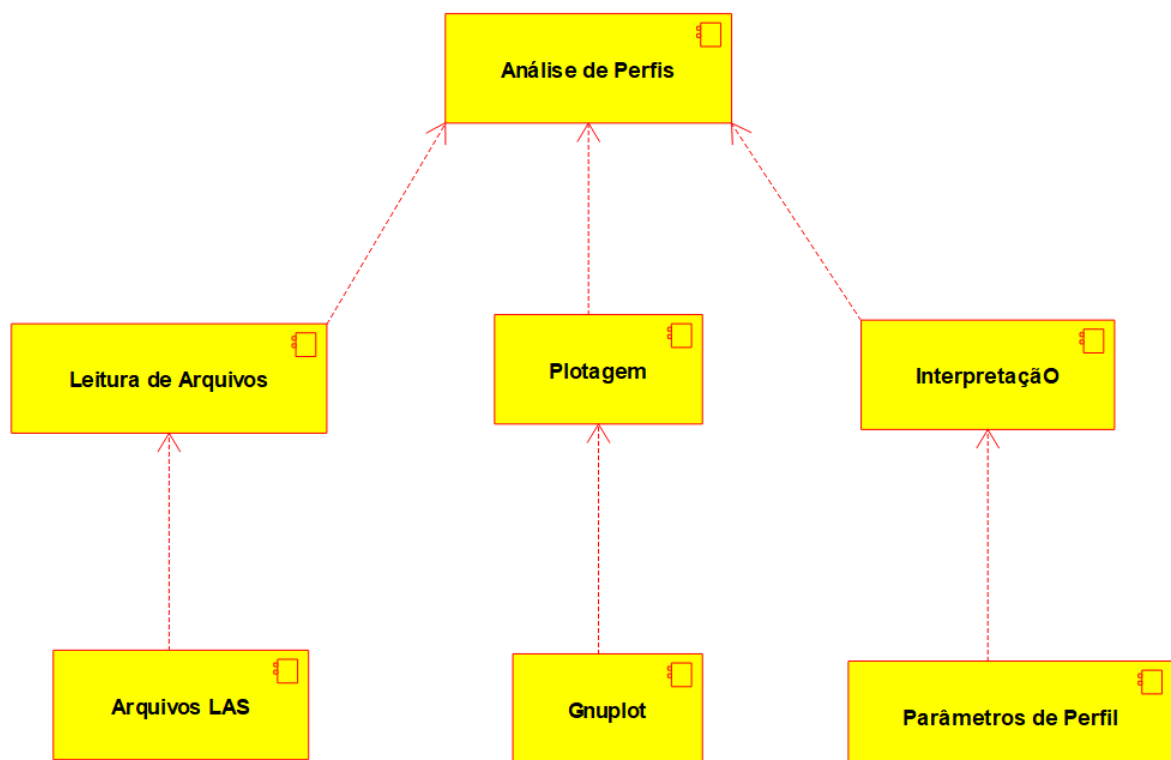


Figura 3.3: Diagrama de Pacotes

# Capítulo 4

## AOO – Análise Orientada a Objeto

Neste capítulo são apresentadas as classes desenvolvidas no projeto, suas relações, atributos e métodos. Ainda teremos um breve conceito de cada classe. Todos os diagramas respeitam a estrutura UML (Linguagem de Modelagem Unificada) para auxiliar na padronização e compreensão. Ainda veremos, além do diagrama de classes, os diagramas de sequência, de comunicação, de máquina de estado e de atividades (Bueno,2003).

### 4.1 Diagramas de classes

O diagrama de classes é apresentado na Figura 4.1.

#### 4.1.1 Dicionário de classes

A seguir apresenta-se o dicionário de classes correspondente ao diagrama de classes fornecido, descrevendo o papel, responsabilidades e principais funcionalidades de cada classe do sistema GeoLogViewer.

- CGeoLogViewerApp: Classe central do software, responsável por integrar todos os módulos, gerenciar fluxo de execução e coordenar ações do usuário.
- CLASReader: Classe responsável pela leitura e carregamento de arquivos LAS.
- CWellLog: Classe que representa um registro completo de poço, contendo profundidades, curvas e informações gerais.
- CLogCurve: Classe que representa cada curva individual presente em um arquivo LAS (GR, RHOB, NPHI, etc.).
- CPlotter: Classe responsável pela geração de gráficos, atuando como intermediária entre os dados do poço e ferramentas externas (como Gnuplot).

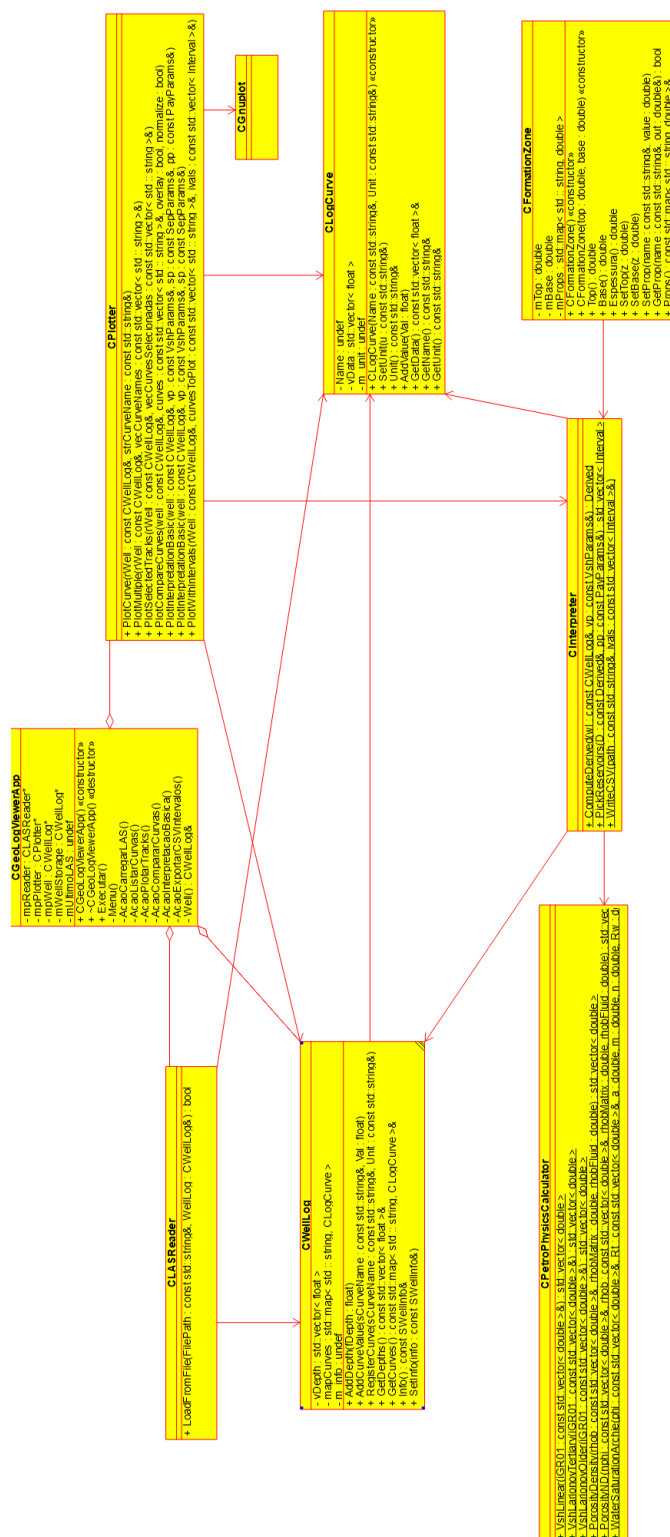


Figura 4.1: Diagrama de classes

- CGnuplot: Classe responsável por realizar a comunicação entre o software e o programa Gnuplot, enviando comandos e dados para gerar e exibir os gráficos das curvas.
- CPetroPhysicsCalculator: Classe dedicada aos cálculos petrofísicos. É um módulo matemático que processa curvas e gera informações complementares.
- CInterpreter: Classe responsável por realizar interpretações automáticas ou auxiliares, aplicando critérios petrofísicos e geológicos.
- CFormationZone: Classe utilizada para representar intervalos interpretados (como zonas de reservatório, contatos litológicos ou intervalos produtivos).

## 4.2 Diagrama de seqüência – eventos e mensagens

O diagrama de seqüência enfatiza a troca de eventos e mensagens e sua ordem temporal. Contém informações sobre o fluxo de controle do software. Costuma ser montado a partir de um diagrama de caso de uso e estabelece o relacionamento dos atores (usuários e sistemas externos) com alguns objetos do sistema.

### 4.2.1 Diagrama de sequência geral

Veja o diagrama de seqüência na Figura 4.2.

- O diagrama de sequência apresenta, de maneira sintetizada, o fluxo completo de interação entre o usuário e os principais componentes do GeoLogViewer durante as etapas de carregamento, interpretação e plotagem de dados LAS. Ele mostra como o usuário inicia o programa, seleciona um arquivo LAS e aciona o processo de leitura realizado pela classe CLASReader, que extrai curvas, profundidades e metadados e os armazena em um objeto CWellLog. Em seguida, ao solicitar a interpretação, o sistema utiliza o CInterpreter em conjunto com o CPetroPhysicsCalculator para gerar parâmetros derivados e curvas interpretadas. Por fim, na etapa de visualização, o CPlotter envia comandos ao Gnuplot para produzir os gráficos interpretados, que são então exibidos ao usuário. O diagrama evidencia a cooperação ordenada entre os módulos e a separação clara entre leitura, análise e visualização.

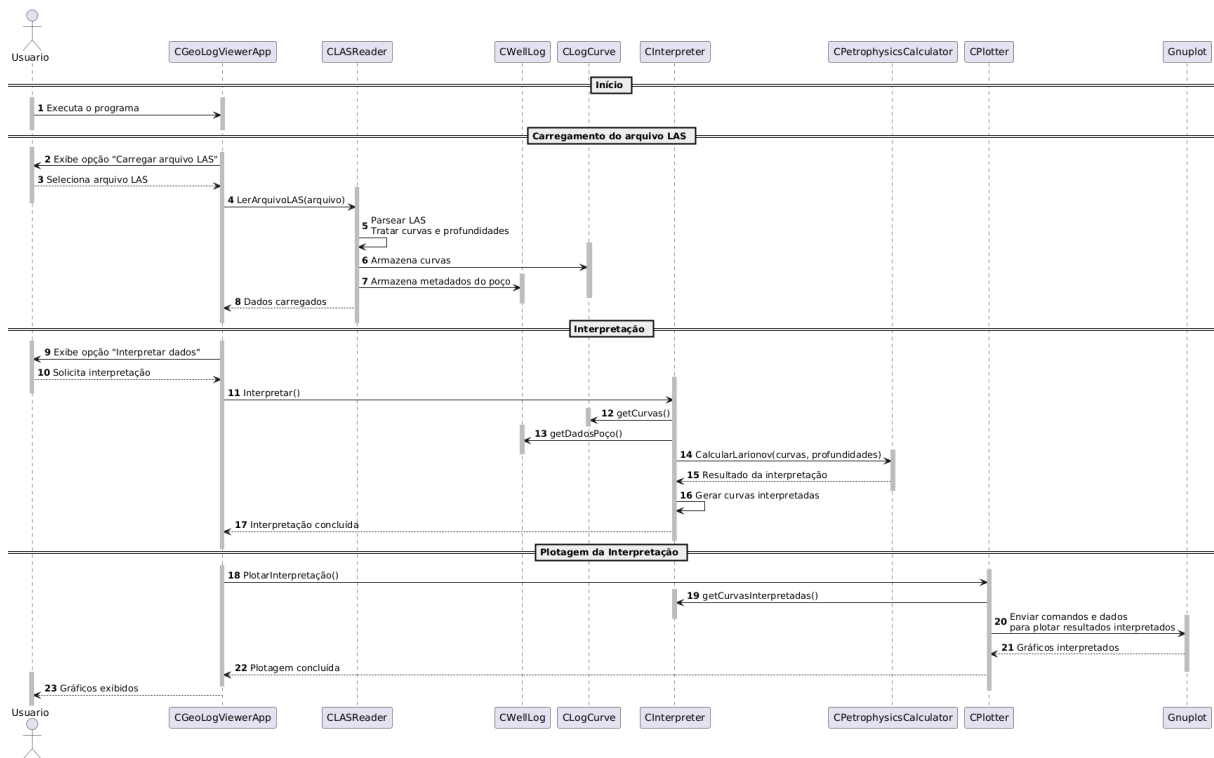


Figura 4.2: Diagrama de seqüência

### 4.3 Diagrama de comunicação – colaboração

Vea o diagrama de comunicação na Figura 4.3

- O diagrama de comunicação apresenta, de forma resumida, como os principais componentes do GeoLogViewer trocam informações entre si durante o processamento de um arquivo LAS. O fluxo inicia com o CGeoLogViewerApp solicitando ao CLASReader a leitura do arquivo, cujos dados são então armazenados em CWellLog e CLogCurve. Para interpretar esses dados, o CInterpreter consulta o poço e utiliza o CPetroPhysicsCalculator para realizar cálculos petrofísicos. Os resultados interpretados são enviados ao CPlotter, que, em conjunto com o CGnuplot, gera os gráficos finais. Assim, o diagrama evidencia uma comunicação estruturada e modular, onde cada classe desempenha um papel específico e contribui para o fluxo completo de leitura, interpretação e visualização dos dados geofísicos.



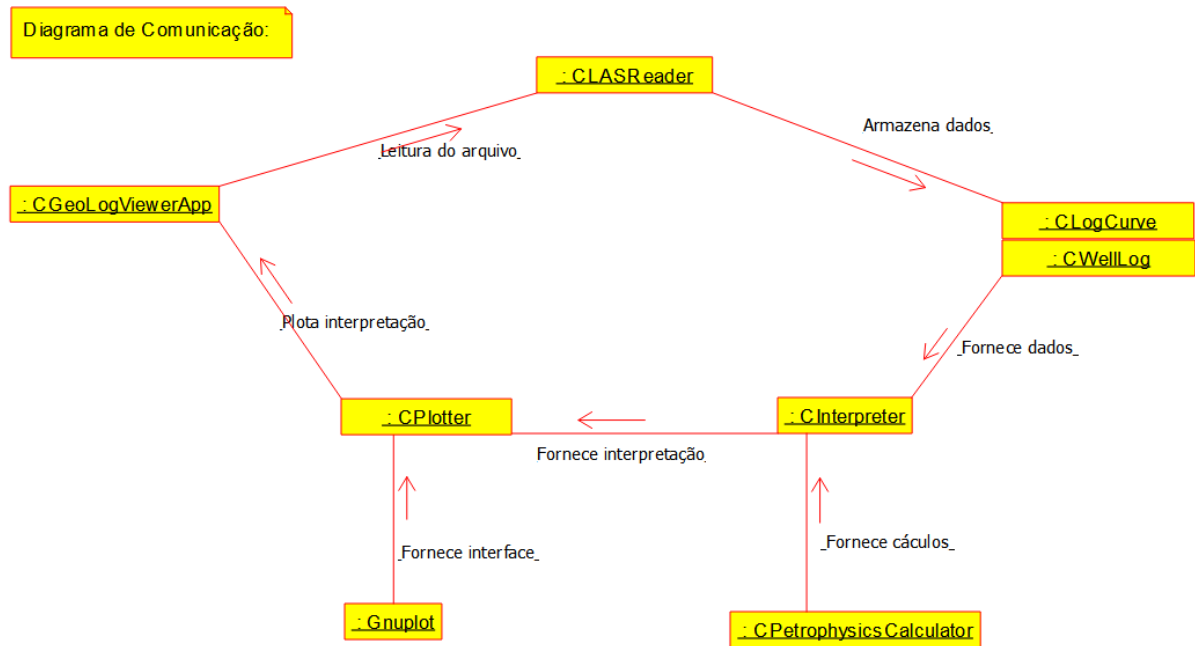


Figura 4.3: Diagrama de comunicação

## 4.4 Diagrama de estado

Um diagrama de máquina de estado representa os diversos estados que o objeto assume e os eventos que ocorrem ao longo de sua vida ou mesmo ao longo de um processo (histórico do objeto). É usado para modelar aspectos dinâmicos do objeto.

Veja na Figura 4.4 o diagrama de máquina de estado..

- O diagrama de máquina de estado representa a evolução de um único objeto responsável pelo gerenciamento do arquivo LAS, destacando como ele muda de condição conforme novos eventos ocorrem. Inicialmente, esse objeto encontra-se em estado ocioso, passando para o estado de recebimento do arquivo quando um LAS é fornecido. A partir daí, ele assume o estado de leitura do conteúdo, no qual se torna capaz de interpretar e reconhecer as informações estruturais do arquivo. Após essa leitura, o objeto evolui para o estado de armazenamento dos dados de poço e curvas, momento em que sua estrutura interna é atualizada para conter esses valores. Com os dados devidamente incorporados ao objeto, ele avança para o estado de processamento interpretativo, no qual mantém informações adicionais derivadas das curvas originais. Em seu estado final, o objeto assume a condição de pronto para visualização, permitindo que suas informações sejam consultadas para fins de plotagem ou análise externa. Assim, o diagrama evidencia a trajetória de transformação interna do objeto ao longo de todo o ciclo de vida, e não uma simples sequência de ações computacionais.

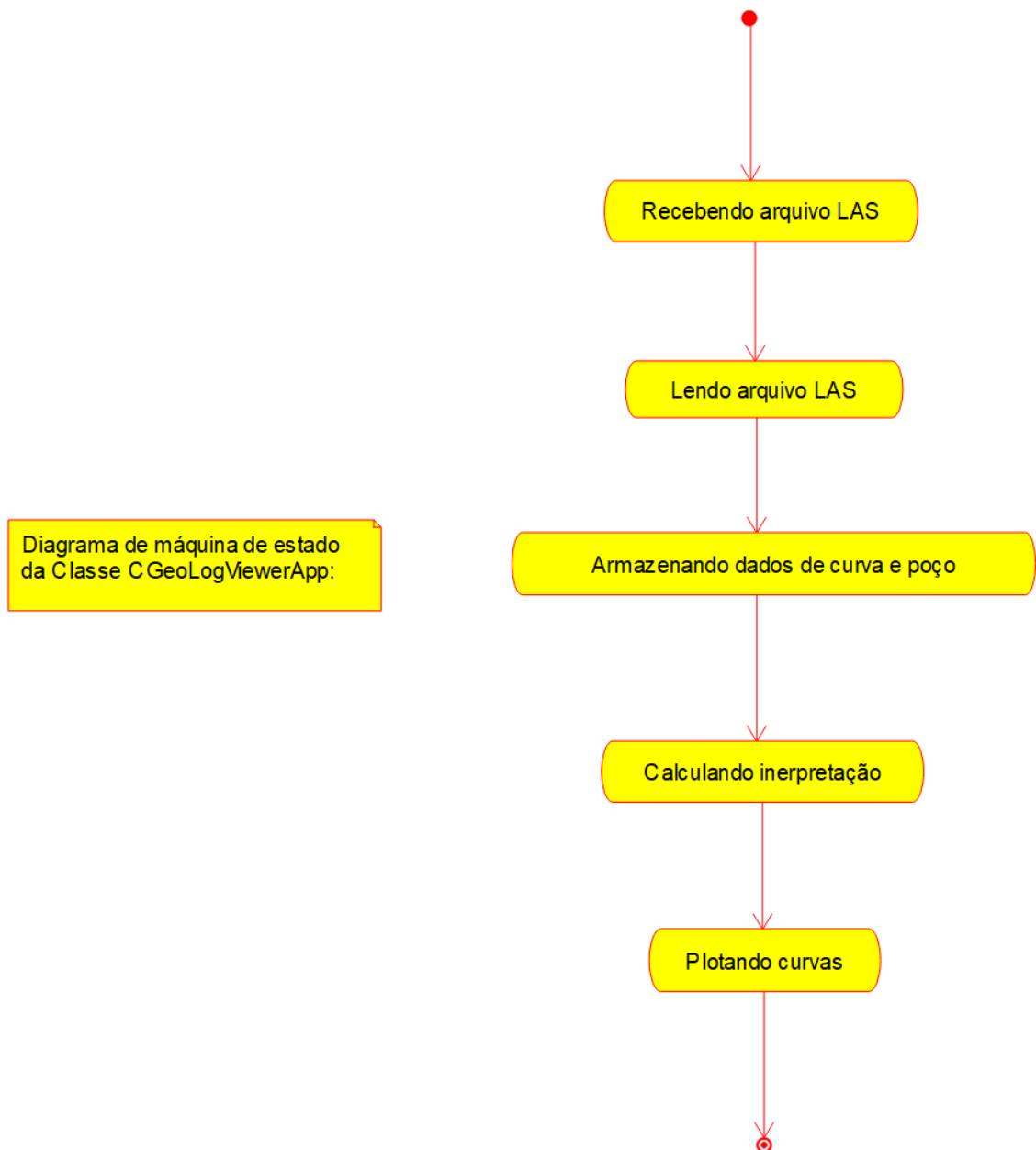


Figura 4.4: Diagrama de máquina de estado

## 4.5 Diagrama de atividades

Veja na Figura 4.5 o diagrama de atividades correspondente a uma atividade específica do diagrama de máquina de estado.

O diagrama de atividades apresentado descreve o fluxo operacional para a atividade específica de seleção e plotagem das curvas a partir de um arquivo fornecido pelo usuário. O processo inicia-se com a obtenção do arquivo, passando imediatamente pela verificação de sua conformidade com o formato LAS. Somente quando essa condição é satisfeita o fluxo segue para o estado em que a atividade de plotagem é preparada. A partir daí, o usuário é conduzido a decidir se deseja visualizar todas as curvas disponíveis ou selecionar

apenas algumas. Caso opte pela seleção parcial, o fluxo direciona a atividade para a entrada manual dos nomes das curvas desejadas.

Uma vez informadas, o sistema verifica a existência dessas curvas, retornando à etapa de entrada sempre que algum nome não corresponda aos dados presentes no arquivo. Quando a verificação é bem-sucedida ou quando o usuário escolhe visualizar todas as curvas, o fluxo avança para a confirmação da ação — representada pela atividade de finalizar a seleção. Com essa confirmação, a atividade conclui-se na plotagem das curvas solicitadas, encerrando o processo específico modelado pelo diagrama.

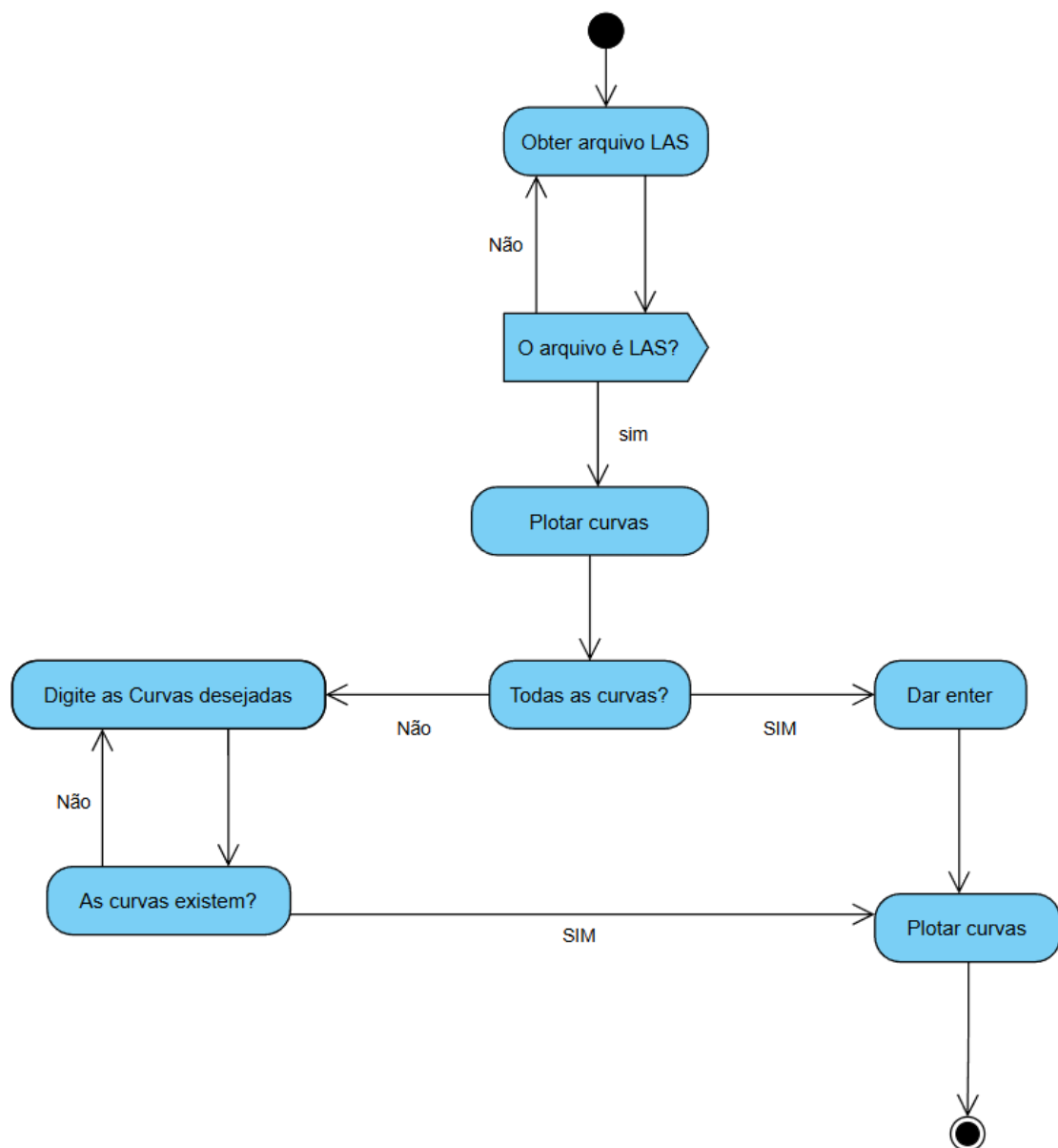


Figura 4.5: Diagrama de atividades

# Capítulo 5

## Projeto

Neste capítulo do projeto de engenharia veremos questões associadas ao projeto do sistema, incluindo protocolos, recursos, plataformas suportadas, implicações nos diagramas feitos anteriormente, diagramas de componentes e implantação. Na segunda parte revisamos os diagramas levando em conta as decisões do projeto do sistema.

### 5.1 Projeto do Sistema

O projeto do sistema GeoLogViewer foi estruturado com base na análise orientada a objeto e nas demandas específicas do processamento de dados geofísicos em Engenharia de Petróleo. A aplicação foi concebida como um software desktop leve e autônomo, desenvolvido em C++ e integrado ao Gnuplot para a visualização gráfica das curvas e interpretações petrofísicas.

#### 1. Protocolos

- Entre elementos externos: O GeoLogViewer não depende de dispositivos físicos nem APIs remotas. A única interação externa ocorre com o Gnuplot, por meio do envio de comandos e dados através de pipes ou arquivos temporários para geração de gráficos. Fora isso, a aplicação opera de forma totalmente local.
- Entre elementos internos: A comunicação entre os módulos ocorre por chamadas diretas de métodos entre objetos C++. Cada classe expõe sua funcionalidade principal por meio de sua interface pública, e os dados trafegam por referências, ponteiros inteligentes (smart pointers) e containers da STL.
- Interface das classes: As APIs internas são descritas nos arquivos .h, onde são definidos os métodos públicos, atributos essenciais e contratos das classes como CWellLog, CLogCurve, CInterpreter, CPlotter e CLASReader. Essas interfaces documentam o que cada módulo oferece ao restante do sistema.
- Formato de arquivos: O software trabalha principalmente com arquivos LAS (entrada), seguindo o padrão Log ASCII Standard, e gera arquivos CSV con-

tendo curvas interpretadas e intervalos calculados. Para gráficos, são produzidos arquivos PNG ou SVG gerados por Gnuplot. Todos os formatos são abertos e facilmente manipuláveis.

## 2. Recursos

- Uso de memória: O gerenciamento é feito pela STL, utilizando `std::vector`, `std::map` e smart pointers (`std::unique_ptr`, `std::shared_ptr`). Os dados das curvas são mantidos em estruturas contíguas para eficiência em cálculos numéricos.
- Banco de dados: Não é utilizado banco de dados. Todas as informações são carregadas diretamente de arquivos LAS e exportadas para CSV.
- Armazenamento: Todos os dados são salvos localmente pelo usuário. A leveza dos arquivos LAS e CSV garante alta portabilidade e facilidade de versionamento.

## 3. Controle

- Baseado em execução sequencial: O fluxo principal segue uma ordem definida: leitura do arquivo → armazenamento dos dados → interpretação → plotagem. A aplicação não utiliza event loop gráfico, mas possui um controle centralizado pelo `CGeoLogViewerApp`.
- Condições extremas: O sistema trata cenários como arquivos LAS mal-formados, curvas ausentes ou valores inconsistentes. O parser do LAS (`CLASReader`) verifica seções obrigatórias e notifica erros. Funções de interpretação validam tamanhos e unidades das curvas antes dos cálculos.
- Escalas de tempo: O processamento é imediato, com operações realizadas assim que o usuário solicita leitura, interpretação ou plotagem. As curvas interpretadas são geradas e exportadas logo após a execução dos cálculos.
- Concorrência: A aplicação é leve e realiza cálculos relativamente rápidos, portanto a concorrência ainda não é necessária. Contudo, o projeto permite futura implementação com `std::thread` ou `std::async` para paralelizar cálculos petrofísicos.

## 4. Plataformas

- Arquitetura: O software utiliza uma estrutura modular clara, organizada em camadas: entrada, modelo, processamento e visualização.
- Plataformas suportadas: O sistema foi projetado para compilar em Windows e Linux, utilizando apenas recursos padrão do C++ e ferramentas externas simples (Gnuplot), garantindo portabilidade.

- Bibliotecas utilizadas: STL - containers, algoritmos e gestão de memória; Gnuplot - geração dos gráficos interpretados
- IDE: O projeto pode ser desenvolvido em qualquer editor, mas utiliza CMake, permitindo integração com Visual Studio, VSCode, CLion ou qualquer outra IDE moderna.

## 5. Padrões de projeto

- Modularidade: cada classe possui uma responsabilidade única — leitura, armazenamento, cálculo ou plotagem.
- Baixo acoplamento e alta coesão: o fluxo de dados é linear e cada módulo conhece somente o necessário sobre os demais. As interfaces são limpas e bem definidas.
- Padrões usados:
  - – Facade: CGeoLogViewerApp atua como fachada coordenando todas as operações.
  - – Strategy: os algoritmos de interpretação podem ser trocados ou estendidos sem alterar o núcleo do sistema.
  - – Adapter: a interação com o Gnuplot é encapsulada dentro do CPlotter.
  - – Creator/Factory (implícito): criação de curvas e poços é centralizada para manter consistência.

## 5.2 Projeto orientado a objeto – POO

O projeto orientado a objeto do GeoLogViewer foi desenvolvido considerando os recursos da linguagem C++, suas bibliotecas padrão (STL) e a integração com ferramentas externas como o Gnuplot. As classes definidas durante a análise foram refinadas com novos atributos, métodos e estruturas necessárias para atender aos requisitos arquiteturais determinados no projeto do sistema. Nesta etapa, são incorporadas decisões práticas sobre algoritmos, estruturas de dados, manipulação de arquivos, otimização de desempenho e controle do fluxo de execução. O modelo conceitual passa a considerar as particularidades da plataforma escolhida (hardware, SO e compilador), além de acrescentar classes e métodos específicos para resolver problemas não identificados na análise inicial.

### Efeitos do projeto no modelo estrutural

- Atualização dos diagramas de pacotes: O sistema foi organizado em subsistemas claros — Leitura, Modelo, Interpretação, Plotagem e App Central. Essa separação permite modularidade e facilita a manutenção.

- Refinamento dos diagramas de classes: As classes receberam novos atributos como `std::vector<float>`, `std::map<std::string, CLogCurve>` e `std::vector<Interval>`. Também foram acrescentados métodos concretos como `ParseLAS()`, `PlotCurve()`, `ComputeDerivedWI()`, `GetCurvesInterpretadas()` e controladores centrais como `Execute()` dentro do `CGeoLogViewerApp`.
- Inclusão de classes auxiliares: O projeto adicionou estruturas como `CFormationZone`, `Derived`, `Interval`, entre outras, para suportar cálculos petrofísicos e detalhar intervalos e propriedades.

### Efeitos do projeto no modelo dinâmico

- Atualização dos diagramas de sequência: Os diagramas agora refletem o fluxo real de execução, incluindo chamadas aos métodos de parsing, cálculo e plotagem.
- Aprimoramento do diagrama de comunicação: Cada classe passa a interagir conforme seus métodos concretos, evidenciando compartilhamento de dados entre `CWellLog`, `CLogCurve`, `CInterpreter` e `CPlotter`.
- Completar diagramas de atividades: O fluxo agora inclui etapas como validação do arquivo LAS, normalização das curvas, seleção das curvas de interpretação e exportação para CSV.

### Efeitos do projeto nos atributos

- Novos atributos adicionados para implementação real:
  - – Vetores de profundidade (`std::vector<float> vDepth`)
  - – Armazenamento de curvas (`std::map<std::string, CLogCurve> mapCurves`)
  - – Zona de formação (`std::map<std::string, double> mProps`)
  - – Flags internas (`bool mLASCarregado`, `bool mInterpretacaoPronta`)
  - – Ponteiros ou referências para classes centrais (ex: ponteiro para `CLASReader` dentro de `CGeoLogViewerApp`)
- Esses atributos foram acrescentados para otimizar cálculos, melhorar o acesso aos dados e minimizar conversões redundantes.

### Efeitos do projeto nos métodos

- Métodos refinados ou criados especificamente para a implementação:
  - – `bool LoadFromFile()`

- – AddCurveValue()
- – ComputeDerivedWI()
- – PickReservoirs()
- – PlotCurve()
- – WriteCSV()
- – Métodos de controle: Menu(), Execute(), AcaoListarCurvas()
- – Métodos auxiliares de verificação e normalização.
- Os métodos recebem parâmetros específicos (vetores, referências, nomes de curva, parâmetros petrofísicos), algo não detalhado na análise, mas essencial para implementação real.

### Efeitos do projeto nas associações

- Associações implementadas por containers STL:
- – Poço → curvas: `std::map<std::string, CLogCurve>`
- – Curva → dados: `std::vector<float>`
- – Interpretação → resultados: `std::vector<Derived>` ou `std::vector<Interval>`
- Associações diretas entre módulos principais: CGeoLogViewerApp mantém associações diretas com CLASReader, CWellLog, CInterpreter e CPlotter, controlando o fluxo completo.
- Uso de referências ou ponteiros: Permite reduzir cópias e trabalhar com objetos grandes (como curvas com milhares de amostras).

### Efeitos do projeto nas otimizações

- Desempenho:
- – Vetores contíguos (`std::vector`) foram priorizados para cálculos numéricos eficientes.
- – Erros são detectados cedo para evitar desperdício de processamento.
- – Gnuplot é chamado apenas com dados necessários, minimizando overhead.
- Organização:



- – Classes auxiliares foram criadas para separar parsing, armazenamento, cálculo e visualização.
- – O código foi estruturado para minimizar acoplamento e facilitar manutenção.
- Escalabilidade:
  - – O projeto permite adicionar novos algoritmos de interpretação.
  - – A estrutura facilita futura inclusão de interfaces gráficas, novos formatos de arquivo ou paralelização dos cálculos.
- – O núcleo petrofísico pode evoluir para incluir métodos avançados como Neural Logs ou inversão.

### 5.3 Diagrama de componentes

O diagrama de componentes mostra as relações entre todos os componentes do software e é apresentado na Figura 5.1

O diagrama de componentes apresentado evidencia a organização estrutural do sistema e as dependências existentes entre os módulos que o compõem. Ele destaca como cada componente participa da solução, mostrando de que forma as funcionalidades são distribuídas e como o fluxo de informações se estabelece entre eles. No centro da arquitetura encontra-se o componente `CGeoLogViewerApp`, responsável por coordenar as operações principais da aplicação e atuar como ponto de integração entre os demais módulos.

A partir dele, observam-se dependências direcionadas para três componentes essenciais: `CLASReader`, encarregado da leitura e interpretação inicial dos arquivos LAS; `CInterpreter`, que atua como intermediário na preparação e organização dos dados interpretados; e `CPetrophysicsCalculator`, responsável pelo cálculo dos parâmetros derivados utilizados na análise petrofísica. Além disso, o componente `CPlotter` mantém uma relação de dependência com a aplicação principal, sendo ele o responsável pela renderização gráfica das curvas processadas. Assim, o diagrama apresenta claramente uma arquitetura modular, na qual cada componente possui responsabilidades bem definidas e colabora para garantir a funcionalidade completa do sistema.

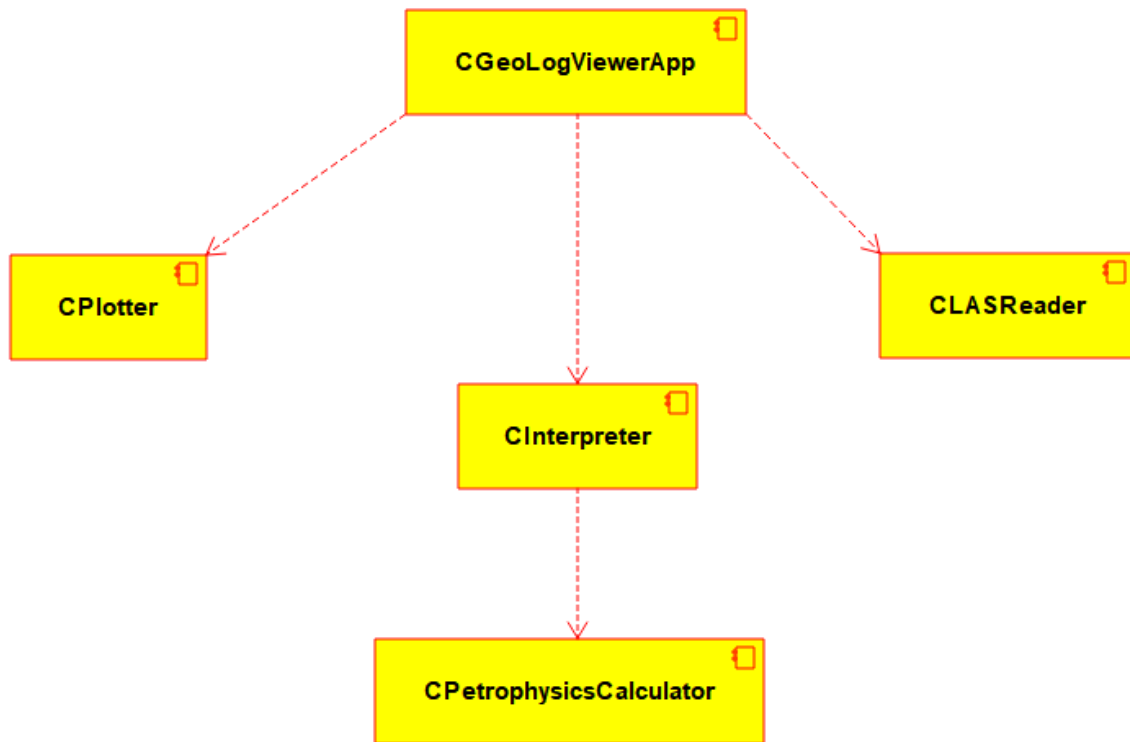


Figura 5.1: Diagrama de componentes

## 5.4 Diagrama de implantação

O diagrama de implantação é apresentado na Figura 5.2

O diagrama de implantação demonstra como o software de perfilagem GeoLogViewer é disponibilizado e executado dentro da infraestrutura física do sistema. Esse tipo de diagrama tem foco na arquitetura computacional em nível físico, evidenciando os nós de hardware envolvidos, a alocação dos componentes de software e as interações que ocorrem entre eles em tempo de execução. Nele, o computador é representado como o nó principal, dentro do qual se encontra o Disco Rígido, responsável por armazenar tanto o Sistema Operacional quanto o software de perfilagem. O sistema operacional exerce papel fundamental, uma vez que fornece o ambiente necessário para a execução do programa, gerenciando os recursos físicos e realizando a intermediação entre hardware e aplicação.

Após ser carregado pelo sistema operacional, o GeoLogViewer inicia o processamento dos dados de perfis, realizando cálculos e gerando curvas para visualização. Esse processamento é efetuado pelo Processador (CPU), que recebe as instruções geradas pelo software e realiza as operações necessárias, enviando os resultados para o monitor, onde são exibidos graficamente ao usuário. Além disso, o diagrama também apresenta o teclado como dispositivo de entrada responsável pela interação do usuário com o sistema, possibilitando a inserção de comandos e dados que são encaminhados ao processador para continuidade do fluxo de execução. Também é representada a relação entre o processador

e o disco rígido, indicando o acesso constante a arquivos e ao sistema conforme a aplicação trabalha.

Dessa forma, o diagrama de implantação evidencia todos os elementos essenciais para o funcionamento do software, incluindo componentes físicos como computador, CPU, teclado, monitor e disco rígido, bem como a organização e comunicação entre eles. Além disso, representa claramente as dependências e associações necessárias para execução do programa, mostrando como o ambiente computacional suporta o processamento e a exibição dos dados visualizados. Assim, este diagrama cumpre seu propósito ao descrever com clareza a arquitetura física do sistema, demonstrando o fluxo de execução, funcionamento dos dispositivos e relação entre hardware e software dentro do ambiente operacional.

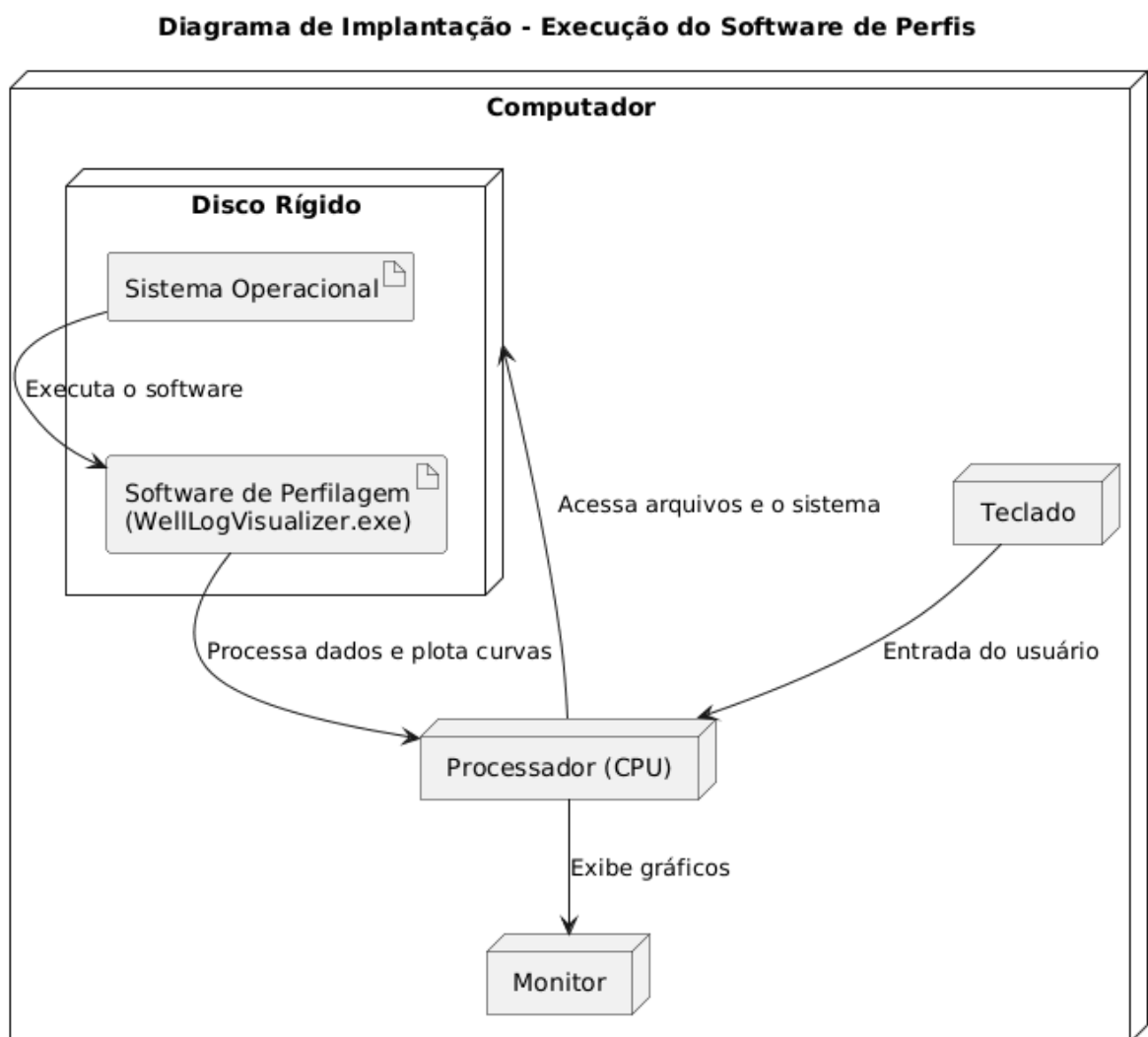


Figura 5.2: Diagrama de implantação

#### 5.4.1 Lista de características <<features>>

##### Versão v0.1 – Núcleo de Leitura e Estruturação dos Dados LAS

Nesta primeira fase, o desenvolvimento concentrou-se na implementação do núcleo de

dados do GeoLogViewer. O objetivo foi estabelecer a base do sistema, com foco na leitura, organização e armazenamento das curvas e metadados contidos em arquivos LAS. Essa etapa fornece o alicerce para as funcionalidades posteriores de interpretação e plotagem.

#### **Features v0.1:**

- Implementação da classe CLASReader, responsável pela leitura estruturada de arquivos LAS.
- Extração e armazenamento de profundidades, curvas geofísicas e metadados do poço.
- Criação da classe CLogCurve, representando uma curva individual com nome, unidade e vetor de valores.
- Implementação da classe CWellLog, contendo todas as curvas e informações gerais do poço.
- Validação inicial do arquivo LAS (checagem de seções obrigatórias e tamanho dos dados).
- Construção das estruturas de dados principais (std::vector, std::map) para armazenar e acessar curvas de forma eficiente.
- Testes iniciais de consistência entre o número de profundidades e o número de amostras das curvas.

#### **Versão v0.2 – Módulo de Plotagem Inicial e Integração com Gnuplot**

Na segunda fase, foram desenvolvidas as funcionalidades essenciais de visualização das curvas lidas. O sistema passou a gerar gráficos utilizando o Gnuplot, integrando o backend de dados com a camada de plotagem, tornando possível ao usuário visualizar perfis completos ou curvas selecionadas.

#### **Features v0.2:**

- Implementação da classe CPlotter, responsável por preparar dados e comandos para o Gnuplot.
- Integração com a ferramenta Gnuplot para geração de gráficos em PNG e SVG.
- Plotagem de curvas individuais selecionadas pelo usuário.
- Plotagem simultânea de múltiplas curvas em um único gráfico.
- Exportação de gráficos para arquivos externos.
- Criação de opções iniciais de plotagem (limites de eixo, cores padrão, espessura de linha).

- Testes de plotagem com arquivos LAS reais, garantindo compatibilidade e estabilidade.

### Versão v0.3 – Interpretação Petrofísica e Visualização Avançada

A etapa final introduziu funcionalidades de interpretação, tornando o GeoLogViewer uma ferramenta não apenas de visualização, mas também analítica. Foram implementados cálculos petrofísicos básicos, intervalos interpretados, curvas derivadas e recursos avançados de visualização.

#### Features v0.3:

- Implementação da classe CInterpreter, responsável pela interpretação automática de curvas.
- Integração com o módulo CPetroPhysicsCalculator, incluindo:
  - – Cálculo de Vshale (Larionov / Linear).
  - – Cálculo de porosidade (densidade, neutrão e densidade-neutrão).
  - – Geração de curvas derivadas.
- Identificação preliminar de zonas de interesse e intervalos potenciais de reservatório.
- Implementação da classe CFormationZone, armazenando topo, base, espessura e propriedades interpretadas.
- Plotagem de interpretações sobrepostas às curvas originais.
- Exportação de resultados interpretativos em arquivo CSV.
- Testes com múltiplos arquivos LAS para validar robustez das interpretações e variações entre curvas.

### 5.4.2 Tabela classificação sistema

A Tabela a seguir é utilizada para classificação do sistema desenvolvido. Deve ser preenchida na etapa de projeto e revisada no final, quando o software for entregue na sua versão final.

|                           |                                                                                                                                      |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Licença:                  | <input checked="" type="checkbox"/> livre GPL-v3 <input type="checkbox"/> proprietária                                               |
| Engenharia de software:   | <input type="checkbox"/> tradicional <input checked="" type="checkbox"/> ágil <input type="checkbox"/> outras                        |
| Paradigma de programação: | <input type="checkbox"/> estruturada <input checked="" type="checkbox"/> orientado a objeto - POO <input type="checkbox"/> funcional |

|                      |                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Modelagem UML:       | [X] básica [X] intermediária [X] avançada                                                                                                                                                                                                                                                                                                                                                                                            |
| Algoritmos:          | [X] alto nível [X] baixo nível                                                                                                                                                                                                                                                                                                                                                                                                       |
| Implementação        | [ ] recursivo ou [X] iterativo; [X] determinístico ou [ ] não-determinístico; [ ] exato ou [X] aproximado                                                                                                                                                                                                                                                                                                                            |
| Concorrências:       | [X] serial [X] concorrente [X] paralelo                                                                                                                                                                                                                                                                                                                                                                                              |
| Paradigma:           | [X] dividir para conquistar [ ] programação linear [ ] transformação/ redução [ ] busca e enumeração [ ] heurístico e probabilístico [ ] baseados em pilhas                                                                                                                                                                                                                                                                          |
| Software:            | [ ] de base [X] aplicativos [ ] de cunho geral [X] específicos para determinada área [X] educativo [X] científico                                                                                                                                                                                                                                                                                                                    |
| Instruções:          | [X] alto nível [ ] baixo nível                                                                                                                                                                                                                                                                                                                                                                                                       |
| Otimização           | [X] serial não otimizado [X] serial otimizado [X] concorrente [X] paralelo [ ] vetorial                                                                                                                                                                                                                                                                                                                                              |
| Interface do usuário | [ ] kernel numérico [ ] linha de comando [ ] modo texto [X] híbrida (texto e saídas gráficas) [ ] modo gráfico (ex: Qt) [ ] navegador                                                                                                                                                                                                                                                                                                |
| Recursos de C++:     | [X] C++ básico (FCC): variáveis padrões da linguagem, estruturas de controle e repetição, estruturas de dados, struct, classes(objetos, atributos, métodos), funções; entrada e saída de dados ( <i>streams</i> ), funções de cmath                                                                                                                                                                                                  |
|                      | [X] C++ intermediário: funções lambda. Ponteiros, referências, herança, herança múltipla, polimorfismo, sobrecarga de funções e de operadores, tipos genéricos (templates), <i>smarth pointers</i> . Diretrizes de pré-processor, classes de armazenamento e modificadores de acesso. Estruturas de dados: enum, uniões. Bibliotecas: entrada e saída acesso com arquivos de disco, redirecionamento. Bibliotecas: <i>filesystem</i> |
|                      | [X] C++ intermediário 2: A biblioteca de gabaritos de C++ (a STL), containers, iteradores, objetos funções e funções genéricas. Noções de processamento paralelo (múltiplas threads, uso de <i>thread</i> , <i>join</i> e <i>mutex</i> ). Bibliotecas: <i>random</i> , <i>threads</i>                                                                                                                                                |
|                      | [ ] C++ avançado: Conversão de tipos do usuário, especializações de templates, excessões. Cluster de computadores, processamento paralelo e concorrente, múltiplos processos (pipes, memória compartilhada, sinais). Bibliotecas: <i>expressões regulares</i> , <i>múltiplos processos</i>                                                                                                                                           |
| Bibliotecas de C++:  | [X] Entrada e saída de dados ( <i>streams</i> ) [X] cmath [X] <i>filesystem</i> [ ] <i>random</i> [X] <i>threads</i> [ ] <i>expressões regulares</i> [ ] <i>múltiplos processos</i>                                                                                                                                                                                                                                                  |

|                                |                                                                                                                                                                                                                              |
|--------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Bibliotecas<br>externas:       | <input checked="" type="checkbox"/> CGnuplot <input checked="" type="checkbox"/> QCustomPlot <input type="checkbox"/> Qt diálogos <input type="checkbox"/> QT<br>Janelas/menus/BT _ _ _ _ _                                  |
| Ferramentas<br>auxiliares:     | Montador: <input type="checkbox"/> make <input checked="" type="checkbox"/> cmake <input type="checkbox"/> qmake                                                                                                             |
| IDE:                           | <input checked="" type="checkbox"/> Editor simples: kate/gedit/emacs <input type="checkbox"/> kdevelop <input type="checkbox"/> QT-Creator                                                                                   |
| SCV:                           | <input type="checkbox"/> cvs <input type="checkbox"/> svn <input checked="" type="checkbox"/> git                                                                                                                            |
| Disciplinas<br>correlacionadas | <input type="checkbox"/> estatística <input checked="" type="checkbox"/> cálculo numérico <input checked="" type="checkbox"/> modelamento numérico <input checked="" type="checkbox"/><br>análise e processamento de imagens |

# Capítulo 6

## Ciclos de Planejamento/Detalhamento

Apresentam-se neste capítulo os ciclos de planejamento e detalhamento das diferentes versões desenvolvidas ao longo do projeto GeoLogViewer. Cada versão corresponde a um conjunto incremental de funcionalidades, evoluindo desde a estruturação inicial do sistema até a incorporação dos mecanismos de análise e visualização das curvas geofísicas extraídas dos arquivos LAS.

### 6.1 Versão 0.1 - Núcleo de Leitura e Estruturação dos Dados LAS

#### **Versão v0.1 – Núcleo de Leitura e Estruturação dos Dados LAS**

Nesta primeira fase, o desenvolvimento concentrou-se na implementação do núcleo de dados do GeoLogViewer. O objetivo foi estabelecer a base do sistema, com foco na leitura, organização e armazenamento das curvas e metadados contidos em arquivos LAS. Essa etapa fornece o alicerce para as funcionalidades posteriores de interpretação e plotagem.

#### **Funcionalidades da v0.1:**

- Implementação da classe CLASReader, responsável pela leitura estruturada de arquivos LAS.
- Extração e armazenamento de profundidades, curvas geofísicas e metadados do poço.
- Criação da classe CLogCurve, representando uma curva individual com nome, unidade e vetor de valores.
- Implementação da classe CWellLog, contendo todas as curvas e informações gerais do poço.
- Validação inicial do arquivo LAS (checagem de seções obrigatórias e tamanho dos dados).



- Construção das estruturas de dados principais (`std::vector`, `std::map`) para armazenar e acessar curvas de forma eficiente.
- Testes iniciais de consistência entre o número de profundidades e o número de amostras das curvas.

## 6.2 Versão 0.2 - Módulo de Plotagem Inicial e Integração com Gnuplot

### Versão v0.2 – Módulo de Plotagem Inicial e Integração com Gnuplot

Na segunda fase, foram desenvolvidas as funcionalidades essenciais de visualização das curvas lidas. O sistema passou a gerar gráficos utilizando o Gnuplot, integrando o backend de dados com a camada de plotagem, tornando possível ao usuário visualizar perfis completos ou curvas selecionadas.

#### Funcionalidades da v0.2:

- Implementação da classe CPlotter, responsável por preparar dados e comandos para o Gnuplot.
- Integração com a ferramenta Gnuplot para geração de gráficos em PNG e SVG.
- Plotagem de curvas individuais selecionadas pelo usuário.
- Plotagem simultânea de múltiplas curvas em um único gráfico.
- Exportação de gráficos para arquivos externos.
- Criação de opções iniciais de plotagem (limites de eixo, cores padrão, espessura de linha).
- Testes de plotagem com arquivos LAS reais, garantindo compatibilidade e estabilidade.

## 6.3 Versão 0.3 - Interpretação Petrofísica e Visualização Avançada

### Versão v0.3 – Expansão das Análises e Exportação dos Resultados

A etapa final introduziu funcionalidades de interpretação, tornando o GeoLogViewer uma ferramenta não apenas de visualização, mas também analítica. Foram implementados cálculos petrofísicos básicos, intervalos interpretados, curvas derivadas e recursos avançados de visualização.

#### Funcionalidades da v0.3:

- Implementação da classe CInterpreter, responsável pela interpretação automática de curvas.
- Integração com o módulo CPetroPhysicsCalculator, incluindo:
  - – Cálculo de Vshale (Larionov / Linear).
  - – Cálculo de porosidade (densidade, neutrão e densidade-neutrão).
  - – Geração de curvas derivadas.
- Identificação preliminar de zonas de interesse e intervalos potenciais de reservatório.
- Implementação da classe CFormationZone, armazenando topo, base, espessura e propriedades interpretadas.
- Plotagem de interpretações sobrepostas às curvas originais.
- Exportação de resultados interpretativos em arquivo CSV.
- Testes com múltiplos arquivos LAS para validar robustez das interpretações e variações entre curvas.

# Capítulo 7

## Ciclos Construção- Implementação

Neste capítulo, são apresentados os códigos fonte implementados.

### 7.1 Código fonte

Como visto na seção anterior, a versão 0.1 foi a última desenvolvida utilizando execução no terminal. Abaixo serão exibidas as classes necessárias para a interação via terminal.

Apresenta-se a seguir um conjunto de classes (arquivos .h e .cpp) além do programa *main*.

Apresenta-se na listagem 7.1 o arquivo com código da função *main*.

Listing 7.1: Arquivo main.cpp

```
#include "App/CGeoLogViewerApp.h"

int main() {
    CGeoLogViewerApp app;
    app.Executar();
    return 0;
}
```

Fonte: produzido pelo autor.

Apresenta-se na listagem 7.2 a implementação da classe CGeoLogViewerApp.

Listing 7.2: Arquivo CGeoLogViewerApp.cpp

```
#include "app/CGeoLogViewerApp.h"

#include "core/CLASReader.h"
#include "core/CWellLog.h"
#include "plotting/CPlotter.h"
#include "interpretation/CInterpreter.h"
```

```

#include <iostream>
#include <algorithm>
#include <limits>

```

```

CGeoLogViewerApp::CGeoLogViewerApp()
    : mpReader(new CLASReader()),
      mpPlotter(new CPlotter()),
      mpWell(nullptr),
      mWellStorage(nullptr) {}

```

```

CGeoLogViewerApp::~CGeoLogViewerApp() {
    delete mWellStorage;
    delete mpPlotter;
    delete mpReader;
}

```

```

static void PrintPrompt() {
    std::cout << "\n=====GeoLogViewer=====\\n"
               << "1) Carregar LAS\\n"
               << "2) Listar curvas\\n\\n"
               << "3) Plotar tracks selecionadas (multiplot)\\n"
               << "4) Comparar curvas (overlay/side-by-side)\\n"
               << "5) Interpretacao basica (GR | VSH | dR/RR) +- shading\\n"
               << "6) Exportar intervalos (CSV)\\n"
               << "0) Sair\\n> ";
}

```

```

void CGeoLogViewerApp::Menu() { PrintPrompt(); }

```

```

CWellLog& CGeoLogViewerApp::Well() {

    if (!mpWell) {
        if (!mWellStorage) mWellStorage = new CWellLog();
        mpWell = mWellStorage;
    }
    return *mpWell;
}

```

```

void CGeoLogViewerApp::Executar() {
    for (;;) {

```

```

    Menu();
    int op = -1;
    if (!(std::cin >> op)) {
        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(),
            '\n');
        continue;
    }
    switch (op) {
        case 1: AcaoCarregarLAS(); break;
        case 2: AcaoListarCurvas(); break;
        case 3: AcaoPlotarTracks(); break;
        case 4: AcaoCompararCurvas(); break;
        case 5: AcaoInterpretacaoBasica(); break;
        case 6: AcaoExportarCSVIntervalos(); break;
        case 0: return;
        default: std::cout << "Opcao_invalida.\n"; break;
    }
}

void CGeoLogViewerApp::AcaoCarregarLAS() {
    std::cout << "Caminho_do_arquivo_LAS:_";
    std::cin >> mUltimoLAS;

    CWellLog& wellRef = Well(); // referencia ao storage

    const bool ok = mpReader->LoadFromFile(mUltimoLAS, wellRef);
    if (!ok) {
        std::cout << "Falha_ao_ler_LAS.\n";
        return;
    }

    if (wellRef.GetDepths().empty() || wellRef.GetCurves().empty()) {
        std::cout << "LAS_lido ,_mas_sem_profundidades/curvas.\n";
        return;
    }

    std::cout << "Arquivo_lido_com_sucesso!\n";
}

```

```

void CGeoLogViewerApp::AcaoListarCurvas() const {
    if (!mpWell) { std::cout << "Carregue_um_LAS_primeiro.\n"; return; }
    std::cout << "Curvas:\n";
    for (const auto& kv : mpWell->GetCurves())
        std::cout << "__" << kv.first << "\n";
}

void CGeoLogViewerApp::AcaoPlotarTracks() {
    if (!mpWell) { std::cout << "Carregue_um_LAS_primeiro.\n"; return; }
    std::cout << "Curvas_separadas_por_virgula_(vazio_=todas):_";
    std::string line;
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    std::getline(std::cin, line);

    std::vector<std::string> curves;
    if (!line.empty()) {
        size_t pos=0;
        while (pos < line.size()) {
            size_t comma = line.find(',', pos);
            std::string c = line.substr(pos, (comma==std::string::npos)?
            c.erase(std::remove_if(c.begin(), c.end(), ::isspace), c.end());
            if (!c.empty()) curves.push_back(c);
            if (comma==std::string::npos) break; else pos = comma+1;
        }
    }
    mpPlotter->PlotSelectedTracks(*mpWell, curves);
}

void CGeoLogViewerApp::AcaoCompararCurvas() {
    if (!mpWell) { std::cout << "Carregue_um_LAS_primeiro.\n"; return; }
    std::cout << "Curvas_para_comparar_(virgulas):_";
    std::string line;
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    std::getline(std::cin, line);

    std::vector<std::string> curves;
    size_t pos=0;
    while (pos < line.size()) {
        size_t comma = line.find(',', pos);
        std::string c = line.substr(pos, (comma==std::string::npos)? std::

```

```

        c.erase(std::remove_if(c.begin(), c.end(), ::isspace), c.end());
        if (!c.empty()) curves.push_back(c);
        if (comma==std::string::npos) break; else pos = comma+1;
    }
    bool overlay=true, normalize=false;
    std::cout << "Overlay?(1=sim,0=nao):_"; std::cin >> overlay;
    if (overlay) { std::cout << "Normalizar_0-1?(1/0):_"; std::cin >> no;

    mpPlotter->PlotCompareCurves(*mpWell, curves, overlay, normalize);
}

void CGeoLogViewerApp::AcaoInterpretacaoBasica() {
    if (!mpWell) {
        std::cout << "Carregue_um_LAS_primeiro.\n";
        return;
    }

    VshParams vp;
    SepParams sp;

    // >>> Ajuste pros nomes reais do LAS <<<
    sp.rdeep_curve = "RPD2"; // deep
    sp.rmed_curve  = "RPM2"; // medium
    sp.rshal_curve = "RPS2"; // shallow

    mpPlotter->PlotInterpretationBasic(*mpWell, vp, sp);
}

void CGeoLogViewerApp::AcaoExportarCSVIntervalos() {
    if (!mpWell) {
        std::cout << "Carregue_um_LAS_primeiro.\n";
        return;
    }

    VshParams vp;
    SepParams sp;
    PayParams pp;

    // >>> Mesmos nomes aqui <<<
    sp.rdeep_curve = "RPD2";

```

```

    sp.rmed_curve = "RPM2";
    sp.rshal_curve = "RPS2";

    auto D      = CInterpreter::ComputeDerived(*mpWell, vp, sp);
    auto picks = CInterpreter::PickReservoirs(D, pp);
    CInterpreter::WriteCSV("./out/intervalos.csv", picks);
    std::cout << "intervalos.csv_gerado_em_./out\n";
}

```

Fonte: produzido pelo autor.

Apresenta-se na listagem 7.3 o arquivo de cabeçalho da classe CGeoLogViewerApp.

Listing 7.3: Arquivo CGeoLogViewerApp.h

```

#ifndef CGEOLOGVIEWERAPP_H
#define CGEOLOGVIEWERAPP_H

#include <string>
#include <vector>

class CLASReader;
class CWellLog;
class CPlotter;

class CGeoLogViewerApp {
public:
    CGeoLogViewerApp();
    ~CGeoLogViewerApp();

    void Executar();    // loop principal

private:
    void Menu();
    void AcaoCarregarLAS();
    void AcaoListarCurvas() const;
    void AcaoPlotarTracks();
    void AcaoCompararCurvas();
    void AcaoInterpretacaoBasica();
    void AcaoExportarCSVIntervalos();

private:
    CLASReader* mpReader;

```



```

    CPlotter*      mpPlotter;

    CWellLog*      mpWell;          // ponteiro para o membro mWell
    CWellLog*      mWellStorage;
    CWellLog&      Well();          // helper para obter referencia valida ao w

    std::string    mUltimoLAS;
};

#endif

```

Fonte: produzido pelo autor.

Apresenta-se na listagem 7.4 o arquivo de cabeçalho da classe CLASReader.

Listing 7.4: Arquivo CLASReader.h

```

#ifndef CLASREADER_H
#define CLASREADER_H

#include <string>
#include "CWellLog.h"

class CLASReader {
public:
    bool LoadFromFile(const std::string& FilePath, CWellLog& WellLog);
};

#endif // CLASREADER_H

```

Fonte: produzido pelo autor.

Apresenta-se na listagem 7.5 a implementação da classe CLASReader.

Listing 7.5: Arquivo CLASReader.cpp

```

// CLASReader.cpp
#include "CLASReader.h"
#include "CWellLog.h"
#include "CLogCurve.h"

#include <algorithm>
#include <cctype>
#include <cstdlib>    // std::strtod
#include <fstream>
#include <iostream>

```

```

#include <limits>      // std::numeric_limits
#include <sstream>
#include <string>
#include <vector>
#include <cmath>      // std::isnan

// Util: trim simples
static inline std::string Trim(const std::string& s) {           //remove espaços
    size_t b = 0, e = s.size();
    while (b < e && std::isspace((unsigned char)s[b])) ++b;
    while (e > b && std::isspace((unsigned char)s[e - 1])) --e;
    return s.substr(b, e - b);
}

// Util: upper sem espaços
static inline std::string UpperNoSpace(std::string s) {         //remove espaços
    s.erase(std::remove_if(s.begin(), s.end(), ::isspace), s.end());
    std::transform(s.begin(), s.end(), s.begin(), ::toupper);
    return s;
}

bool CLASReader::LoadFromFile(const std::string& path, CWellLog& WellLog)
{
    std::ifstream in(path);
    if (!in.is_open()) {
        std::cerr << "[CLASReader]_Nao_foi_possivel_abrir:_ " << path << "
        return false;
    }

    std::string line;
    std::string currentSection;
    std::vector<std::string> curveNames; // ordem das colunas em ASCII

    // zera info do WELL (mantendo NaN por padrao)
    SWellInfo info = WellLog.Info();

    while (std::getline(in, line)) {
        // Normaliza quebras de linha e ignora linhas vazias
        if (!line.empty() && (line.back() == '\r' || line.back() == '\n'))
            line.pop_back();
    }
}

```

```

std::string raw = line;
line = Trim(line);
if (line.empty()) continue;

// Troca de secao (WELL, CURVE, ASCII, etc.)
if (line.size() > 0 && line[0] == '~') {
    currentSection = UpperNoSpace(line); // garante WELL, CURVE,
    continue;
}

// Ignora comentarios iniciando por '#'
if (line[0] == '#') continue;

// =====
// Secao WELL: NULL/STRT/STOP/STEP
// =====
if (currentSection == "~WELL") {
    // Formatos comuns:
    // STRT.M          343.96 : START DEPTH
    // STOP.M          915.00 : STOP DEPTH
    // STEP.M          0.1524 : STEP
    // NULL.           -999.25 : NULL VALUE
    auto posDot = line.find('.');
    auto posColon = line.find(':');

    if (posDot != std::string::npos) {
        std::string key = UpperNoSpace(line.substr(0, posDot));

        // captura o primeiro numero da linha
        double val = std::numeric_limits<double>::quiet_NaN();
        {
            std::istringstream iss(line);
            std::string tok;
            while (iss >> tok) {
                char* endp = nullptr;
                double tmp = std::strtod(tok.c_str(), &endp);
                if (endp && *endp == '\\0') { val = tmp; break; }
            }
        }
    }
}

```

```

        if (!std::isnan(val)) {
            if (key == "NULL") info.null = val;
            else if (key == "STRT") info.strt = val;
            else if (key == "STOP") info.stop = val;
            else if (key == "STEP") info.step = val;
        }
    }
    WellLog.SetInfo(info);
    continue;
}

// =====
// Secao CURVE: mnemonic e unidade
// =====
if (currentSection == "~CURVE") {
    // Exemplos de linha:
    // DEPT.M          : DEPTH
    // GR.API          : GAMMA RAY
    // RPD2.OHM-M      : RES 2MHz (Deep)
    // ROP.M/HR
    // Regra: texto antes do '.' eh o mnemonic; apos o '.' ate es
    std::string mnemonic, unit;

    auto posDot = line.find('.');
    if (posDot != std::string::npos) {
        mnemonic = line.substr(0, posDot);
        auto rest = line.substr(posDot + 1);
        size_t endu = rest.find_first_of("_\\t:");
        unit = (endu == std::string::npos) ? rest : rest.substr(0, endu);
    } else {
        // sem ponto -> sem unidade explicita
        size_t endm = line.find_first_of("_\\t:");
        mnemonic = (endm == std::string::npos) ? line : line.substr(0, endm);
        unit.clear();
    }

    mnemonic = UpperNoSpace(mnemonic);

    // DEPT/DEPTH eh tratado como indice de profundidade
    if (mnemonic == "DEPT" || mnemonic == "DEPTH") {

```

```

        curveNames.push_back(mnemonic);
    } else {
        curveNames.push_back(mnemonic);
        WellLog.RegisterCurve(mnemonic, unit); // <— agora com
    }
    continue;
}

// =====
// Secao ASCII: dados (DEPT + curvas na ordem de curveNames)
// =====
if (currentSection == "~ASCII") {
    // separa tokens por espaco/abas
    std::vector<std::string> toks;
    {
        std::istringstream iss(line);
        std::string t;
        while (iss >> t) toks.push_back(t);
    }
    if (toks.empty()) continue;

    if (toks.size() != curveNames.size()) {
        // linha malformada — loga e continua
        std::cerr << "[CLASReader]_Linha_~ASCII_com_colunas_" <<
            << "_mas_esperado_" << curveNames.size() << "_:" << endl;
        continue;
    }

    // primeiro token -> profundidade
    {
        char* endp = nullptr;
        double d = std::strtod(toks[0].c_str(), &endp);
        if (!(endp && *endp == '\0')) {
            std::cerr << "[CLASReader]_Profundidade_invalida:_" << endl;
            continue;
        }
        WellLog.AddDepth(d);
    }

    // demais tokens -> valores de curvas

```

```

        for (size_t i = 1; i < toks.size(); ++i) {
            const std::string& name = UpperNoSpace(curveNames[i]);
            char* endp = nullptr;
            double v = std::strtod(toks[i].c_str(), &endp);
            if (!(endp && *endp == '\0')) {
                // valor invalido; registra NaN aqui optamos por pula
                continue;
            }
            WellLog.AddCurveValue(name, static_cast<float>(v));
        }
        continue;
    }

    // Outras secoes: ignoradas por enquanto (mantemos o que ja funci

}

return true;
}

```

Fonte: produzido pelo autor.

Apresenta-se na listagem 7.6 o arquivo de cabeçalho da classe CLogCurve.

Listing 7.6: Arquivo CLogCurve.h

```

#ifndef CLOG_CURVE_H
#define CLOG_CURVE_H

#include <string>
#include <vector>

class CLogCurve {
private:
    std::string Name;
    std::vector<float> vData;
    std::string m_unit;

public:
    CLogCurve(const std::string& Name, const std::string& Unit);

    void SetUnit(const std::string& u) { m_unit = u; }
    const std::string& Unit() const { return m_unit; }
    void AddValue(float Val);

```

```

    const std::vector<float>& GetData() const;
    const std::string& GetName() const;
    const std::string& GetUnit() const;
};

#endif

```

Fonte: produzido pelo autor.

Apresenta-se na listagem 7.7 a implementação da classe CLogCurve.

Listing 7.7: Arquivo CLogCurve.cpp

```

#include "CLogCurve.h"

CLogCurve::CLogCurve(const std::string& Name, const std::string& Unit)
    : Name(Name), m_unit(Unit) {}

void CLogCurve::AddValue(float Val) {
    vData.push_back(Val);
}

const std::vector<float>& CLogCurve::GetData() const {
    return vData;
}

const std::string& CLogCurve::GetName() const {
    return Name;
}

const std::string& CLogCurve::GetUnit() const {
    return m_unit;
}

```

Fonte: produzido pelo autor.

Apresenta-se na listagem 7.8 o arquivo de cabeçalho da classe CWellLog.

Listing 7.8: Arquivo CWellLog.h

```

#ifndef CWELL_LOG_H
#define CWELL_LOG_H

#include "CLogCurve.h"
#include <map>
#include <string>

```

```

#include <limits>           // std::numeric_limits
#include <cmath>             // std::isnan

struct SWellInfo {
    double null    = std::numeric_limits<double>::quiet_NaN();
    double strt    = std::numeric_limits<double>::quiet_NaN();
    double stop    = std::numeric_limits<double>::quiet_NaN();
    double step    = std::numeric_limits<double>::quiet_NaN();
};

class CWellLog {
private:
    std::vector<float> vDepth;
    std::map<std::string, CLogCurve> mapCurves;

public:
    void AddDepth(float fDepth);
    void AddCurveValue(const std::string& sCurveName, float Val);
    void RegisterCurve(const std::string& sCurveName, const std::string&

    const std::vector<float>& GetDepths() const;
    const std::map<std::string, CLogCurve>& GetCurves() const;

    const SWellInfo& Info() const { return m_info; }
    void SetInfo(const SWellInfo& info) { m_info = info; }

private:
    SWellInfo m_info;
};

#endif

```

Fonte: produzido pelo autor.

Apresenta-se na listagem 7.9 a implementação da classe CWellLog.

Listing 7.9: Arquivo CWellLog.cpp

```

#include "CWellLog.h"
#include <iostream>

void CWellLog::AddDepth(float fDepth) {

```



```

        vDepth.push_back(fDepth);
    }

    void CWellLog::RegisterCurve(const std::string& sCurveName, const std::string& sUnit) {
        mapCurves.emplace(sCurveName, CLogCurve(sCurveName, Unit));
    }

    void CWellLog::AddCurveValue(const std::string& sCurveName, float Val) {
        auto it = mapCurves.find(sCurveName);
        if (it != mapCurves.end()) {
            it->second.AddValue(Val);
        } else {
            std::cerr << "Curva_n?o_registrada:_" << sCurveName << std::endl;
        }
    }

    const std::vector<float>& CWellLog::GetDepths() const {
        return vDepth;
    }

    const std::map<std::string, CLogCurve>& CWellLog::GetCurves() const {
        return mapCurves;
    }

```

Fonte: produzido pelo autor.

Apresenta-se na listagem 7.10 a implementação da classe CFormationZone.

Listing 7.10: Arquivo CFormationZone.h

```

#ifndef CFORMATIONZONE_H
#define CFORMATIONZONE_H

#include <string>
#include <map>

class CFormationZone {
public:
    CFormationZone() : mTop(0.0), mBase(0.0) {}
    CFormationZone(double top, double base) : mTop(top), mBase(base) {}

    double Top() const { return mTop; }
    double Base() const { return mBase; }

```

```

    double Espessura() const { return mBase - mTop; }

    void SetTop(double z){ mTop = z; }
    void SetBase(double z){ mBase = z; }

    // propriedades medias por nome (ex.: "VSH", "PHIE", "Sw", "Rdeep")
    void SetProp(const std::string& name, double value){ mProps[name]=value; }
    bool GetProp(const std::string& name, double& out) const {
        auto it=mProps.find(name); if (it==mProps.end()) return false; out=*it;
    }
    const std::map<std::string,double>& Props() const { return mProps; }

private:
    double mTop, mBase;
    std::map<std::string,double> mProps;
};

#endif

```

Fonte: produzido pelo autor.

Apresenta-se na listagem 7.11 o arquivo de cabeçalho da classe CInterpreter.

Listing 7.11: Arquivo CInterpreter.h

```

#ifndef CINTERPRETER_H
#define CINTERPRETER_H

#include <string>
#include <vector>

class CWellLog;
class CFormationZone;

// Parametros de calculo de VSH
struct VshParams {
    // Percentis para areia limpa e xisto
    double p_clean = 5.0;      // percentil da areia
    double p_shale = 95.0;     // percentil do xisto

    enum class Method {
        Linear,
        LarionovTertiary,
    };
};

```

```

        LarionovOlder
    } method = Method::LarionovTertiary;

    // Suavizacao opcional na amostra (0 sem suavizacao)
    int smooth_window_samples = 0;

    // Nome da curva de GR
    std::string gr_curve = "GR";
};

// Parametros para resistividades

struct SepParams {
    // Nomes padrao, ajuste conforme seu LAS
    std::string rdeep_curve = "RT";    // resistividade profunda
    std::string rmed_curve  = "RMD";   // media
    std::string rshal_curve = "RS";    // rasa
};

// Parametros de pay / triagem de reservatorio

struct PayParams {
    double vsh_max      = 0.5;    // corte maximo de Vsh
    double RR_min       = 2.0;    // minimo de Rdeep/Rshal
    double dR_min       = 0.0;    // minimo de separacao (Rdeep - Rshal)
    double min_thickness = 1.0;    // espessura minima em metros
};

// Derivados calculados por profundidade

struct Derived {
    std::vector<double> depth;    // md (mesma amostragem do poco)

    std::vector<double> vsh;      // volume de argila
    std::vector<double> rdeep;    // resistividade profunda
    std::vector<double> rmed;     // media
    std::vector<double> rshal;    // rasa

    std::vector<double> dR;       // separacao Rdeep - Rshal
    std::vector<double> RR;       // razao Rdeep / Rshal
};

```

```

};

// Intervalo interpretado

struct Interval {
    double top    = 0.0;    // topo (m)
    double base   = 0.0;    // base (m)

    double vsh_mean    = 0.0;
    double rdeep_mean  = 0.0;
    double dR_mean     = 0.0;
    double RR_mean     = 0.0;

    double h() const { return base - top; }
};

//Classe principal de interpretacao

class CInterpreter {
public:
    // Derivados: VSH Rdeep/med/shal dR RR (alinhados ao mesmo depth)
    static Derived ComputeDerived(const CWellLog& wl,
                                const VshParams& vp = VshParams{} ,
                                const SepParams& sp = SepParams{});

    // Picks automaticos de reservatorio (screening)
    static std::vector<Interval> PickReservoirs(const Derived& D,
                                                const PayParams& pp = PayParams{});

    // Exporta CSV simples de intervalos
    static void WriteCSV(const std::string& path,
                       const std::vector<Interval>& ivals);

    // Converte Interval CFormationZone (para uso posterior)
    static std::vector<CFormationZone> ToFormationZones(const std::vector<Interval>& ivals);
};

#endif

```

Fonte: produzido pelo autor.

Apresenta-se na listagem 7.12 a implementação da classe CInterpreter.

Listing 7.12: Arquivo CInterpreter.cpp

```
#include "CInterpreter.h"
#include "CWellLog.h"
#include "CLogCurve.h"
#include "CPetrophysicsCalculator.h"
#include "CFormationZone.h"

#include <algorithm>
#include <cmath>
#include <fstream>

// ----- helpers -----
namespace {

inline double Clamp01(double x)
{
    if (x < 0.0) return 0.0;
    if (x > 1.0) return 1.0;
    return x;
}

inline bool Finite(double x)
{
    return std::isfinite(x);
}

// percentil simples [0,100]
double Percentile(std::vector<double> v, double p)
{
    if (v.empty())
        return NAN;

    if (p <= 0.0)
        return *std::min_element(v.begin(), v.end());
    if (p >= 100.0)
        return *std::max_element(v.begin(), v.end());

    std::sort(v.begin(), v.end());
```

```

    const double pos  = p / 100.0 * (v.size() - 1);
    const size_t i0    = static_cast<size_t>(pos);
    const double frac  = pos - i0;

    if (i0 + 1 < v.size())
        return v[i0] * (1.0 - frac) + v[i0 + 1] * frac;
    else
        return v[i0];
}

// media movel simples , NaN-safe
void SmoothMA(std::vector<double>& v, int w)
{
    if (w <= 1 || v.empty())
        return;

    const int half = w / 2;
    const size_t N = v.size();
    std::vector<double> out(N, NAN);

    for (size_t i = 0; i < N; ++i) {
        int i0 = static_cast<int>(i) - half;
        int i1 = static_cast<int>(i) + half;
        if (i0 < 0) i0 = 0;
        if (i1 >= static_cast<int>(N)) i1 = static_cast<int>(N) - 1;

        double acc = 0.0;
        int n = 0;
        for (int k = i0; k <= i1; ++k) {
            if (Finite(v[k])) {
                acc += v[k];
                ++n;
            }
        }
        if (n > 0)
            out[i] = acc / n;
    }

    v.swap(out);
}

```

```

} // namespace

// ----- ComputeDerived -----

Derived CInterpreter::ComputeDerived(const CWellLog& wl,
                                     const VshParams& vp,
                                     const SepParams& sp)
{
    Derived D;

    const auto& zf = wl.GetDepths();
    const size_t N = zf.size();
    D.depth.resize(N);
    for (size_t i = 0; i < N; ++i)
        D.depth[i] = static_cast<double>(zf[i]);

    const auto& curves = wl.GetCurves();

    auto CopyCurve = [&](const std::string& curveName,
                        std::vector<double>& out)
    {
        out.assign(N, NAN);
        if (curveName.empty())
            return;

        auto it = curves.find(curveName);
        if (it == curves.end())
            return;

        const auto& data = it->second.GetData();
        const size_t nCopy = std::min(N, data.size());
        for (size_t i = 0; i < nCopy; ++i) {
            const float v = data[i];
            if (std::isfinite(v))
                out[i] = static_cast<double>(v);
        }
    };

    // Resistividades

```

```

CopyCurve(sp.rdeep_curve, D.rdeep);
CopyCurve(sp.rmed_curve, D.rmed);
CopyCurve(sp.rshal_curve, D.rshal);

D.dR.assign(N, NAN);
D.RR.assign(N, NAN);

for (size_t i = 0; i < N; ++i) {
    const double rd = (i < D.rdeep.size()) ? D.rdeep[i] : NAN;
    const double rs = (i < D.rshal.size()) ? D.rshal[i] : NAN;

    if (Finite(rd) && Finite(rs) && rd > 0.0 && rs > 0.0) {
        D.dR[i] = rd - rs;
        D.RR[i] = rd / rs;
    }
}

// GR -> VSH via CPetrophysicsCalculator

std::vector<double> gr;
CopyCurve(vp.gr_curve, gr);

std::vector<double> grValid;
grValid.reserve(N);
for (double g : gr)
    if (Finite(g)) grValid.push_back(g);

D.vsh.assign(N, NAN);

if (!grValid.empty()) {
    const double gr_clean = Percentile(grValid, vp.p_clean);
    double gr_shale = Percentile(grValid, vp.p_shale);

    if (gr_shale == gr_clean)
        gr_shale = gr_clean + 1e-6;

    std::vector<double> igr(N, NAN);
    for (size_t i = 0; i < N; ++i) {
        if (!Finite(gr[i])) continue;
        igr[i] = Clamp01((gr[i] - gr_clean) / (gr_shale - gr_clean));
    }
}

```



```

    }

    switch (vp.method) {
        case VshParams::Method::Linear:
            D.vsh = CPetroPhysicsCalculator::VshLinear(igr);
            break;
        case VshParams::Method::LarionovTertiary:
            D.vsh = CPetroPhysicsCalculator::VshLarionovTertiary(igr);
            break;
        case VshParams::Method::LarionovOlder:
            D.vsh = CPetroPhysicsCalculator::VshLarionovOlder(igr);
            break;
    }

    if (vp.smooth_window_samples > 1)
        SmoothMA(D.vsh, vp.smooth_window_samples);
}

return D;
}

// ----- PickReservoirs -----

std::vector<Interval> CInterpreter::PickReservoirs(const Derived& D,
                                                    const PayParams& pp)
{
    std::vector<Interval> out;

    const size_t N = D.depth.size();
    if (N == 0)
        return out;

    std::vector<bool> isPay(N, false);

    for (size_t i = 0; i < N; ++i) {
        const double vsh = (i < D.vsh.size()) ? D.vsh[i] : NAN;
        const double RR = (i < D.RR.size()) ? D.RR[i] : NAN;
        const double dR = (i < D.dR.size()) ? D.dR[i] : NAN;

        if (!Finite(vsh) || !Finite(RR))

```

```
        continue;

    if (vsh > pp.vsh_max)
        continue;

    if (RR < pp.RR_min)
        continue;

    if (pp.dR_min > 0.0) {
        if (!Finite(dR) || dR < pp.dR_min)
            continue;
    }

    isPay[i] = true;
}

size_t i = 0;
while (i < N) {
    while (i < N && !isPay[i])
        ++i;
    if (i >= N)
        break;

    const size_t iStart = i;
    while (i < N && isPay[i])
        ++i;
    const size_t iEnd = i - 1;

    const double top = D.depth[iStart];
    const double base = D.depth[iEnd];

    if ((base - top) < pp.min_thickness)
        continue;

    double sum_vsh = 0.0; int n_vsh = 0;
    double sum_rdeep = 0.0; int n_rdeep = 0;
    double sum_dR = 0.0; int n_dR = 0;
    double sum_RR = 0.0; int n_RR = 0;

    for (size_t k = iStart; k <= iEnd; ++k) {
```

```

        if (k < D.vsh.size() && Finite(D.vsh[k])) {
            sum_vsh += D.vsh[k];
            ++n_vsh;
        }
        if (k < D.rdeep.size() && Finite(D.rdeep[k])) {
            sum_rdeep += D.rdeep[k];
            ++n_rdeep;
        }
        if (k < D.dR.size() && Finite(D.dR[k])) {
            sum_dR += D.dR[k];
            ++n_dR;
        }
        if (k < D.RR.size() && Finite(D.RR[k])) {
            sum_RR += D.RR[k];
            ++n_RR;
        }
    }

    Interval iv;
    iv.top    = top;
    iv.base   = base;
    iv.vsh_mean    = (n_vsh    > 0) ? (sum_vsh    / n_vsh)    : NAN;
    iv.rdeep_mean  = (n_rdeep > 0) ? (sum_rdeep / n_rdeep) : NAN;
    iv.dR_mean     = (n_dR    > 0) ? (sum_dR     / n_dR)     : NAN;
    iv.RR_mean     = (n_RR    > 0) ? (sum_RR     / n_RR)     : NAN;

    out.push_back(iv);
}

return out;
}

// ----- WriteCSV -----

void CInterpreter::WriteCSV(const std::string& path,
                           const std::vector<Interval>& ivals)
{
    std::ofstream f(path.c_str());
    if (!f) return;

```

```

    f.setf(std::ios::fixed);
    f.precision(3);

    f << "Top_m,Base_m,Thick_m,VSH_mean,Rdeep_mean,dR_mean,RR_mean\n";
    for (const auto& I : ivals) {
        f << I.top << " , "
          << I.base << " , "
          << I.h() << " , "
          << I.vsh_mean << " , "
          << I.rdeep_mean << " , "
          << I.dR_mean << " , "
          << I.RR_mean << "\n";
    }
}

// ----- ToFormationZones -----

std::vector<CFormationZone> CInterpreter::ToFormationZones(const std::vec
{
    std::vector<CFormationZone> out;
    out.reserve(ivals.size());

    for (const auto& iv : ivals) {
        CFormationZone z(iv.top, iv.base);
        z.SetProp("VSH", iv.vsh_mean);
        z.SetProp("RDEEP", iv.rdeep_mean);
        z.SetProp("dR", iv.dR_mean);
        z.SetProp("RR", iv.RR_mean);
        out.push_back(z);
    }

    return out;
}

```

Fonte: produzido pelo autor.

Apresenta-se na listagem 7.13 o arquivo de cabeçalho da classe CPetrophysicsCalculator.

Listing 7.13: Arquivo CPetrophysicsCalculator.h

```

#ifndef CPETROPHYSICSCALCULATOR_H
#define CPETROPHYSICSCALCULATOR_H

```

```

#include <vector>
#include <string>

class CPetroPhysicsCalculator {
public:
    // Vsh a partir de GR ja normalizado (0..1) ou via IGamma opcional
    static std::vector<double> VshLinear(const std::vector<double>& iGR01,
    static std::vector<double> VshLarionovTertiary(const std::vector<double>& iGR01,
    static std::vector<double> VshLarionovOlder(const std::vector<double>& iGR01,

    // Porosidade efetiva
    // Densidade: phi = (RHOBma - RHOB) / (RHOBma - RHOBfl)
    static std::vector<double> PorosityDensity(const std::vector<double>& iGR01,
                                                double rhobMatrix, double rhobfl)

    // Neutron-Density simples: media/combinacao (ex.: arenitos)
    static std::vector<double> PorosityND(const std::vector<double>& nphi,
                                           const std::vector<double>& rhob,
                                           double rhobMatrix, double rhobfl)

    // Archie (agua): Sw(n) = (a*Rw)/(phi(m) * Rt)
    static std::vector<double> WaterSaturationArchie(const std::vector<double>& iGR01,
                                                       const std::vector<double>& iGR02,
                                                       double a, double m,
    };

#endif

```

Fonte: produzido pelo autor.

Apresenta-se na listagem 7.14 a implementação da classe CPetrophysicsCalculator.

Listing 7.14: Arquivo CPetrophysicsCalculator.cpp

```

#include "CPetroPhysicsCalculator.h"
#include <cmath>
#include <algorithm>

static inline bool fin(double v){ return std::isfinite(v); }
static inline double clamp01(double x){ return x<0?0:(x>1?1:x); }

// ----- Vsh -----

```

```
std::vector<double> CPetroPhysicsCalculator::VshLinear(const std::vector<
    std::vector<double> out(iGR.size(), NAN);
    for (size_t i=0;i<iGR.size();++i)
        if (fin(iGR[i])) out[i] = clamp01(iGR[i]);
    return out;
}
```

```
std::vector<double> CPetroPhysicsCalculator::VshLarionovTertiary(const st
    std::vector<double> out(iGR.size(), NAN);
    for (size_t i=0;i<iGR.size();++i)
        if (fin(iGR[i])) out[i] = clamp01(0.083*(std::pow(2.0,3.7*iGR[i]))
    return out;
}
```

```
std::vector<double> CPetroPhysicsCalculator::VshLarionovOlder(const std::
    std::vector<double> out(iGR.size(), NAN);
    for (size_t i=0;i<iGR.size();++i)
        if (fin(iGR[i])) out[i] = clamp01(0.33*(std::pow(2.0,2.0*iGR[i]))
    return out;
}
```

// ————— Porosidade —————

```
std::vector<double> CPetroPhysicsCalculator::PorosityDensity(const std::v
    double rhobMatrix;
    double rhobFluid;
{
    const size_t N = rhob.size();
    std::vector<double> out(N, NAN);
    const double denom = (rhobMatrix - rhobFluid);
    if (!fin(denom) || denom == 0.0)
        return out;

    for (size_t i=0;i<N;++i){
        if (fin(rhob[i])) {
            double phi = (rhobMatrix - rhob[i]) / denom;
            out[i] = phi;
        }
    }
}
```

```

        return out;
    }

std::vector<double> CPetroPhysicsCalculator::PorosityND(const std::vector<double> nphi,
                                                         const std::vector<double> rhob,
                                                         double rhobMatrix,
                                                         double rhobFluid)
{
    const size_t N = std::min(nphi.size(), rhob.size());
    std::vector<double> out(N, NAN);

    // phiD pela densidade
    auto phiD = PorosityDensity(rhob, rhobMatrix, rhobFluid);

    for (size_t i=0; i<N; ++i){
        if (fin(nphi[i]) && fin(phiD[i])) {
            // combinacao simples: media aritmetica
            out[i] = 0.5*(nphi[i] + phiD[i]);
        }
    }
    return out;
}

```

// ————— Archie —————

```

std::vector<double> CPetroPhysicsCalculator::WaterSaturationArchie(const std::vector<double> phi,
                                                                    const std::vector<double> Rt,
                                                                    double m,
                                                                    double a,
                                                                    double Rw)
{
    const size_t N = std::min(phi.size(), Rt.size());
    std::vector<double> out(N, NAN);

    for (size_t i=0; i<N; ++i){
        if (fin(phi[i]) && fin(Rt[i]) && phi[i]>0 && Rt[i]>0 && Rw>0){
            double num = a*Rw;
            double den = std::pow(phi[i], m)*Rt[i];
            if (den>0){
                double sw_n = num/den;
                if (sw_n>=0) out[i] = clamp01(std::pow(sw_n, 1.0/n));
            }
        }
    }
}

```

```

    }
}
return out;
}

```

Fonte: produzido pelo autor.

Apresenta-se na listagem 7.15 a implementação da classe CGnuplot.

Listing 7.15: Arquivo CGnuplot.h

```

////////////////////////////////////
//                               Classe de Interface em C++ para o programa gnuplot.
//
////////////////////////////////////
// Esta interface usa pipes e nao ira funcionar em sistemas que nao suportam
// o padrao POSIX pipe.
// O mesmo foi testado em sistemas Windows (MinGW e Visual C++) e Linux(C)
// Este programa foi originalmente escrito por:
// Historico de versoes:
// 0. Interface para linguagem C
//   por N. Devillard (27/01/03)
// 1. Interface para C++: traducao direta da versao em C
//   por Rajarshi Guha (07/03/03)
// 2. Correcoes para compatibilidadde com Win32
//   por V. Chyzhdenka (20/05/03)
// 3. Novos metodos membros, correcoes para compatibilidade com Win32 e Linux
//   por M. Burgis (10/03/08)
// 4. Traducao para Portugues, documentacao - javadoc/doxygen,
//   e modificacoes na interface (adicao de interface alternativa)
//   por Bueno.A.D. (30/07/08)
// Tarefas:
// (v1)
// Documentar toda classe
// Adicionar novos metodos, criando atributos adicionais se necessario.
// Adotar padrao C++, isto e, usar sobrecarga nas chamadas.
// (v2)
// Criar classe herdeira CGnuplot, que inclui somente a nova interface.
// como e herdeira, o usuario vai poder usar nome antigos.
// Vantagem: preserva classe original, cria nova interface, fica a crit
// qual interface utilizar.
////////////////////////////////////
// Requisitos:

```



```
// - O programa gnuplot deve estar instalado (veja http://www.gnuplot.info)
// - No Windows: setar a Path do Gnuplot (i.e. C:/program files/gnuplot/bin
//           ou setar a path usando: Gnuplot::set_GNUPlotPath(const std::string& path)
//           Gnuplot::set_GNUPlotPath("C:/program files/gnuplot/bin");
// - Para um melhor uso, consulte o manual do gnuplot,
//   no GNU/Linux digite: man gnuplot ou info gnuplot.
//
// - Veja aula em http://www.lenep.uenf.br/bueno/DisciplinaSL/
//
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
```

```
#ifndef CGnuplot_h
#define CGnuplot_h
#include <iostream>           // Para teste
#include <string>
#include <vector>
#include <stdexcept>         // Heranca da classe std::runtime_error e
#include <cstdio>             // Para acesso a arquivos FILE
```

```
/**
```

```
@brief Erros em tempo de execucao
```

```
@class GnuplotException
```

```
@file GnuplotException.h
```

```
*/
```

```
class GnuplotException : public std::runtime_error
```

```
{
```

```
public:
```

```
    /// Construtor
```

```
    GnuplotException (const std::string & msg):std::runtime_error (msg) {}
```

```
};
```

```
/**
```

```
@brief Classe de interface para acesso ao programa gnuplot.
```

```
@class Gnuplot
```

```
@file gnuplot_i.hpp
```

```
*/
```

```
class Gnuplot
```

```
{
```

```
private:
```

```

//-----
FILE *   gnuCMD;          ///< Ponteiro para stream que escreve no pipe.
bool     valid;           ///< Flag que indica se a sessao do gnuplot esta
bool     two_dim;         ///< true = verdadeiro = 2d, false = falso = 3d.
int      nplots;          ///< Numero de graficos (plots) na sessao.
std::string pstyle;       ///< Estilo utilizado para visualizacao das funcoes
std::string smooth;       ///< interpolate and approximate data in defined
std::vector<std::string> tmpfile_list; ///< Lista com nome dos arquivos

//-----
bool fgrid;               ///< 0 sem grid,          1 com grid
bool fhidden3d;           ///< 0 nao oculta,          1 oculta
bool fcontour;            ///< 0 sem contorno,        1 com contorno
bool fsurface;            ///< 0 sem superficie,      1 com superficie
bool flegend;             ///< 0 sem legendad,        1 com legenda
bool ftitle;              ///< 0 sem titulo,          1 com titulo
bool fxlogscale;          ///< 0 desativa escala log, 1 ativa escala log
bool fylogscale;          ///< 0 desativa escala log, 1 ativa escala log
bool fzlogscale;          ///< 0 desativa escala log, 1 ativa escala log
bool fsmooth;             ///< 0 desativa,            1 ativa

//-----
// Atributos estaticos (compartilhados por todos os objetos)
static int tmpfile_num;    ///< Numero total de arquivos temporarios
static std::string m_sGNUPlotFileName; ///< Nome do arquivo executavel do gnuplot
static std::string m_sGNUPlotPath;    ///< Caminho para executavel do gnuplot
static std::string terminal_std;      ///< Terminal padrao (standart),

//-----
// Funcoes membro (metodos membro) (funcoes auxiliares)
/// @brief Cria arquivo temporario e retorna seu nome.
/// Usa get_program_path(); e popen();
void init ();

/// @brief Cria arquivo temporario e retorna seu nome.
/// Usa get_program_path(); e popen();
void Init() { init(); }

/// @brief Cria arquivo temporario.
std::string create_tmpfile (std::ofstream & tmp);

```

```

/// @brief Cria arquivo temporario.
std::string CreateTmpFile (std::ofstream & tmp) { return create_tmpf

//-----
// Funcoes estaticas (static functions)
/// @brief Retorna verdadeiro se a path esta presente.
static bool get_program_path ();

/// @brief Retorna verdadeiro se a path esta presente.
static bool Path() { return get_program_path(); }

/// @brief Checa se o arquivo existe.
static bool file_exists (const std::string & filename , int mode = 0);

/// @brief Checa se o arquivo existe.
static bool FileExists (const std::string & filename , int mode = 0)
    { return file_exists( filename , mode ); }

//-----
public:
// Opcional: Seta path do gnuplot manualmente
// No windows: a path (caminho) deve ser dada usando '/' e nao backslash
/// @brief Seta caminho para path do gnuplot.
//ex: CGnuplot::set_GNUPlotPath ("\"C:/program files/gnuplot/bin/\");

static bool set_GNUPlotPath (const std::string & path);

/// @brief Seta caminho para path do gnuplot.
static bool Path(const std::string & path) { return set_GNUPlotPath(path)
//
/// @brief Opcional: Seta terminal padrao (standart), usado para visual
/// Valores padroes (default): Windows - win, Linux - x11, Mac - aqua
static void set_terminal_std (const std::string & type);

/// @brief Opcional: Seta terminal padrao (standart), usado para visual
/// Para retornar para terminal janela precisa chamar ShowOnScreen().
/// Valores padroes (default): Windows - win, Linux - x11 ou wxt (fedora)
static void Terminal (const std::string & type) { set_terminal_std(type)

```

---

```

//
/// @brief Construtor, seta o estilo do grafico na construcao.
Gnuplot (const std::string & style = "points");

/// @brief Construtor, plota um grafico a partir de um vector, diretamente
Gnuplot (const std::vector< double >&x,
         const std::string & title = "",
         const std::string & style = "points",
         const std::string & labelx = "x",
         const std::string & labely = "y");

/// @brief Construtor, plota um grafico do tipo x_y a partir de vetores
Gnuplot (const std::vector< double >&x,
         const std::vector< double >&y,
         const std::string & title = "",
         const std::string & style = "points",
         const std::string & labelx = "x",
         const std::string & labely = "y");

/// @brief Construtor, plota um grafico de x_y_z a partir de vetores,
Gnuplot (const std::vector< double >&x,
         const std::vector< double >&y,
         const std::vector< double >&z,
         const std::string & title = "",
         const std::string & style = "points",
         const std::string & labelx = "x",
         const std::string & labely = "y",
         const std::string & labelz = "z");

/// @brief Destrutor, necessario para deletar arquivos temporarios.
~Gnuplot ();

//
/// @brief Envia comando para o gnuplot.
Gnuplot & cmd (const std::string & cmdstr);

/// @brief Envia comando para o gnuplot.
Gnuplot & Cmd (const std::string & cmdstr) { return cmd(cmdstr); }

/// @brief Envia comando para o gnuplot.

```

```

Gnuplot & Command (const std::string & cmdstr) { return cmd(cmdstr); }

/// @brief Sobrecarga operador <<, funciona como Comando.
Gnuplot & operator<< (const std::string & cmdstr);

///
/// @brief Mostrar na tela ou escrever no arquivo, seta o tipo de term
Gnuplot & showonscreen (); // Janela de saida e setada como

/// @brief Mostrar na tela ou escrever no arquivo, seta o tipo de term
Gnuplot & ShowOnScreen () { return showonscreen (); }
};

/// @brief Salva sessao do gnuplot para um arquivo postscript, informe
/// Depois retorna para modo terminal
Gnuplot & savetops (const std::string & filename = "gnuplot_output");

/// @brief Salva sessao do gnuplot para um arquivo postscript, informe
/// Depois retorna para modo terminal
Gnuplot & SaveTops (const std::string & filename = "gnuplot_output")
{ return savetops(filename); }

/// @brief Salva sessao do gnuplot para um arquivo png, nome do arquivo
/// Depois retorna para modo terminal
Gnuplot & savetopng (const std::string & filename = "gnuplot_output");

/// @brief Salva sessao do gnuplot para um arquivo png, nome do arquivo
/// Depois retorna para modo terminal
Gnuplot & SaveTopng (const std::string & filename = "gnuplot_output")
{ return savetopng(filename); }

/// @brief Salva sessao do gnuplot para um arquivo jpg, nome do arquivo
/// Depois retorna para modo terminal
Gnuplot & savetojpeg (const std::string & filename = "gnuplot_output");

/// @brief Salva sessao do gnuplot para um arquivo jpg, nome do arquivo
/// Depois retorna para modo terminal
Gnuplot & SaveTojpeg (const std::string & filename = "gnuplot_output")
{ return savetojpeg(filename); }

```

```

/// @brief Salva sessao do gnuplot para um arquivo filename, usando o t
/// Ex:
/// grafico.SaveTo("pressao_X_temperatura","png", "enhanced size 1280,9
/// Para melhor uso dos flags adicionais consulte o manual do gnuplot (
Gnuplot & SaveTo(const std::string &filename, const std::string &terminal

//-----
/// @brief Seta estilos de linhas (em alguns casos sao necessarias inf
/// lines, points, linespoints, impulses, dots, steps, fsteps, histeps,
/// boxes, histograms, filledcurves
Gnuplot & set_style (const std::string & stylestr = "points");

/// @brief Seta estilos de linhas (em alguns casos sao necessarias inf
/// lines, points, linespoints, impulses, dots, steps, fsteps, histeps,
/// boxes, histograms, filledcurves
Gnuplot & Style (const std::string & stylestr = "points")
{ return set_style(stylestr); }

/// @brief Ativa suavizacao.
/// Argumentos para interpolacoes e aproximacoes.
/// csplines, bezier, acsplines (para dados com valor > 0), sbezier, un
/// frequency (funciona somente com plot_x, plot_xy, plotfile_x,
/// plotfile_xy (se a suavizacao esta ativa, set_style nao tem efeito n
Gnuplot & set_smooth (const std::string & stylestr = "csplines");

/// @brief Desativa suavizacao.
Gnuplot & unset_smooth (); // A suavizacao nao e setada por

/// @brief Ativa suavizacao.
/// Argumentos para interpolacoes e aproximacoes.
/// csplines, bezier, acsplines (para dados com valor > 0), sbezier, un
/// frequency (funciona somente com plot_x, plot_xy, plotfile_x,
/// plotfile_xy (se a suavizacao esta ativa, set_style nao tem efeito n
Gnuplot & Smooth(const std::string & stylestr = "csplines")
{ return set_smooth(stylestr); }

Gnuplot & Smooth( int _fsmooth )
{ if( fsmooth == _fsmooth )
    return set_contour( _fsmooth );
else

```

```

                                return unset_contour
                                }

    /// @brief Desativa suavizacao.
    //Gnuplot & UnsetSmooth() { return unset_smooth ();

    /// @brief Escala o tamanho do ponto usado na plotagem.
    Gnuplot & set_pointsize (const double pointsize = 1.0);

    /// @brief Escala o tamanho do ponto usado na plotagem.
    Gnuplot & PointSize (const double pointsize = 1.0)
                                { return set_pointsize(p

    /// @brief Ativa o grid (padrao = desativado).
    Gnuplot & set_grid ();

    /// @brief Desativa o grid (padrao = desativado).
    Gnuplot & unset_grid ();

    /// @brief Ativa/Desativa o grid (padrao = desativado).
    Gnuplot & Grid(bool _fgrid = 1)
                                { if(fgrid == _fgrid)
                                    return set_grid();
                                else
                                    return unset_grid();

    /// @brief Seta taxa de amostragem das funcoes, ou dos dados de interpo
    Gnuplot & set_samples (const int samples = 100);

    /// @brief Seta taxa de amostragem das funcoes, ou dos dados de interpo
    Gnuplot & Samples(const int samples = 100) { return set_samples(sam

    /// @brief Seta densidade de isolinhas para plotagem de funcoes como su
    Gnuplot & set_isosamples (const int isolines = 10);

    /// @brief Seta densidade de isolinhas para plotagem de funcoes como su
    Gnuplot & IsoSamples (const int isolines = 10){ return set_isosamples(

    /// @brief Ativa remocao de linhas ocultas na plotagem de superficies (
    Gnuplot & set_hidden3d ();

```

```

/// @brief Desativa remocao de linhas ocultas na plotagem de superficie
Gnuplot & unset_hidden3d (); // hidden3d nao e setado por padrao

/// @brief Ativa/Desativa remocao de linhas ocultas na plotagem de superficie
Gnuplot & Hidden3d(bool _fhidden3d = 1)
{
    if(fhidden3d == _fhidden3d)
        return set_hidden3d();
    else
        return unset_hidden3d();
}

/// @brief Ativa desenho do contorno em superficies (para plotagem 3d).
/// @param base, surface, both.
Gnuplot & set_contour (const std::string & position = "base");

/// @brief Desativa desenho do contorno em superficies (para plotagem 3d).
Gnuplot & unset_contour (); // contour nao e setado por padrao

/// @brief Ativa/Desativa desenho do contorno em superficies (para plotagem 3d).
/// @param base, surface, both.
Gnuplot & Contour(const std::string & position = "base")
{
    return set_contour(position);
}

Gnuplot & Contour( int _fcontour )
{
    if( fcontour == _fcontour )
        return set_contour();
    else
        return unset_contour();
}

/// @brief Ativa a visualizacao da superficie (para plotagem 3d).
Gnuplot & set_surface (); // surface e setado por padrao (0)

/// @brief Desativa a visualizacao da superficie (para plotagem 3d).
Gnuplot & unset_surface ();

/// @brief Ativa/Desativa a visualizacao da superficie (para plotagem 3d).
Gnuplot & Surface( int _fsurface = 1 )
{
    if( fsurface == _fsurface )
        return set_surface();
    else
        return unset_surface();
}

```



```

                                return unset_surfa
                                }

    /// @brief Ativa a legenda (a legenda eh setada por padrao).
    /// Posicao: inside/outside, left/center/right, top/center/bottom, nob
    Gnuplot & set_legend (const std::string & position = "default");

    /// @brief Desativa a legenda (a legenda eh setada por padrao).
    Gnuplot & unset_legend ();

    /// @brief Ativa/Desativa a legenda (a legenda eh setada por padrao).
    Gnuplot & Legend(const std::string & position = "default")
                                { return set_legend(position); }

    /// @brief Ativa/Desativa a legenda (a legenda eh setada por padrao).
    Gnuplot & Legend(int _flegend)
                                { if(flegend == _flegend)
                                    return set_legend();
                                    else
                                        return unset_legend();
                                    }

    /// @brief Ativa o titulo da secao do gnuplot.
    Gnuplot & set_title (const std::string & title = "");

    /// @brief Desativa o titulo da secao do gnuplot.
    Gnuplot & unset_title (); // O title nao e setado por padrao

    /// @brief Ativa/Desativa o titulo da secao do gnuplot.
    Gnuplot & Title(const std::string & title = "")
                                {
                                    return set_title(title);
                                }

    Gnuplot & Title(int _ftitle)
                                {
                                    if(ftitle == _ftitle)
                                        return set_title();
                                    else
                                        return unset_title();
                                    }

```

```

/// @brief Seta o rotulo (nome) do eixo y.
Gnuplot & set_ylabel (const std::string & label = "y");

/// @brief Seta o rotulo (nome) do eixo y.
/// Ex: set ylabel "{/Symbol s}[MPa]" font "Times Italic , 10"
Gnuplot & YLabel(const std::string & label = "y")
{ return set_ylabel(label); }

/// @brief Seta o rotulo (nome) do eixo x.
Gnuplot & set_xlabel (const std::string & label = "x");

/// @brief Seta o rotulo (nome) do eixo x.
Gnuplot & XLabel(const std::string & label = "x")
{ return set_xlabel(label); }

/// @brief Seta o rotulo (nome) do eixo z.
Gnuplot & set_zlabel (const std::string & label = "z");

/// @brief Seta o rotulo (nome) do eixo z.
Gnuplot & ZLabel(const std::string & label = "z")
{ return set_zlabel(label); }

/// @brief Seta intervalo do eixo x.
Gnuplot & set_xrange (const int iFrom, const int iTo);

/// @brief Seta intervalo do eixo x.
Gnuplot & XRange (const int iFrom, const int iTo)
{ return set_xrange(iFrom, iTo); }

/// @brief Seta intervalo do eixo y.
Gnuplot & set_yrange (const int iFrom, const int iTo);

/// @brief Seta intervalo do eixo y.
Gnuplot & YRange (const int iFrom, const int iTo)
{ return set_yrange(iFrom, iTo); }

/// @brief Seta intervalo do eixo z.
Gnuplot & set_zrange (const int iFrom, const int iTo);

/// @brief Seta intervalo do eixo z.

```

```

Gnuplot & ZRange (const int iFrom, const int iTo)
{ return set_zrange(iFrom, iTo); }

/// @brief Seta escalonamento automatico do eixo x (default).
Gnuplot & set_xautoscale ();

/// @brief Seta escalonamento automatico do eixo x (default).
Gnuplot & XAutoscale() { return set_xautoscale (); }

/// @brief Seta escalonamento automatico do eixo y (default).
Gnuplot & set_yautoscale ();

/// @brief Seta escalonamento automatico do eixo y (default).
Gnuplot & YAutoscale() { return set_yautoscale (); }

/// @brief Seta escalonamento automatico do eixo z (default).
Gnuplot & set_zautoscale ();

/// @brief Seta escalonamento automatico do eixo z (default).
Gnuplot & ZAutoscale() { return set_zautoscale (); }

/// @brief Ativa escala logaritma do eixo x (logscale nao e setado por default).
Gnuplot & set_xlogscale (const double base = 10);

/// @brief Desativa escala logaritma do eixo x (logscale nao e setado por default).
Gnuplot & unset_xlogscale ();

/// @brief Ativa escala logaritma do eixo x (logscale nao e setado por default).
Gnuplot & XLogscale (const double base = 10) { //if(base)
{
    return set_xlogscale(base);
}
//else
//return unset_xlogscale();
}

/// @brief Ativa/Desativa escala logaritma do eixo x (logscale nao e setado por default).
Gnuplot & XLogscale(bool _fxlogscale)
{
    if(fxlogscale == _fxlogscale)
        return set_xlogscale(_fxlogscale);
    else
        return unset_xlogscale();
}

```

```

    }

    /// @brief Ativa escala logaritma do eixo y (logscale nao e setado por
    Gnuplot & set_ylogscale (const double base = 10);

    /// @brief Ativa escala logaritma do eixo y (logscale nao e setado por
    Gnuplot & YLogscale (const double base = 10) { return set_ylogscale (base); }

    /// @brief Desativa escala logaritma do eixo y (logscale nao e setado por
    Gnuplot & unset_ylogscale ();

    /// @brief Ativa/Desativa escala logaritma do eixo y (logscale nao e setado por
    Gnuplot & YLogscale(bool _fylogscale)
    {
        if(fylogscale == _fylogscale)
            return set_ylogscale (base);
        else
            return unset_ylogscale ();
    }

    /// @brief Ativa escala logaritma do eixo y (logscale nao e setado por
    Gnuplot & set_zlogscale (const double base = 10);

    /// @brief Ativa escala logaritma do eixo y (logscale nao e setado por
    Gnuplot & ZLogscale (const double base = 10) { return set_zlogscale (base); }

    /// @brief Desativa escala logaritma do eixo z (logscale nao e setado por
    Gnuplot & unset_zlogscale ();

    /// @brief Ativa/Desativa escala logaritma do eixo y (logscale nao e setado por
    Gnuplot & ZLogscale(bool _fzlogscale)
    {
        if(fzlogscale == _fzlogscale)
            return set_zlogscale (base);
        else
            return unset_zlogscale ();
    }

    /// @brief Seta intervalo da palette (autoscale por padrao).
    Gnuplot & set_cbrange (const int iFrom, const int iTo);

```

```

/// @brief Seta intervalo da palette (autoscale por padrao).
Gnuplot & CBRange(const int iFrom, const int iTo)
{ return set_cbrange(iFrom, iTo); }



---


/// @brief Plota dados de um arquivo de disco.
Gnuplot & plotfile_x (const std::string & filename,
                     const int column = 1, const std::string & title = "");

/// @brief Plota dados de um arquivo de disco.
Gnuplot & PlotFile (const std::string & filename,
                   const int column = 1, const std::string & title = "")
{ return plotfile_x(filename, column, title); }

/// @brief Plota dados de um vector.
Gnuplot & plot_x (const std::vector < double >&x, const std::string & title)

/// @brief Plota dados de um vector.
Gnuplot & PlotVector (const std::vector < double >&x, const std::string & title)
{ return plot_x(x, title); }

/// @brief Plota pares x,y a partir de um arquivo de disco.
Gnuplot & plotfile_xy (const std::string & filename,
                     const int column_x = 1,
                     const int column_y = 2, const std::string & title = "");

/// @brief Plota pares x,y a partir de um arquivo de disco.
Gnuplot & PlotFile (const std::string & filename,
                   const int column_x = 1,
                   const int column_y = 2, const std::string & title = "")
{
    return plotfile_xy(filename, column_x, column_y, title);
}

/// @brief Plota pares x,y a partir de vetores.
Gnuplot & plot_xy (const std::vector < double >&x,
                  const std::vector < double >&y, const std::string & title = "")

/// @brief Plota pares x,y a partir de vetores.
Gnuplot & PlotVector (const std::vector < double >&x,
                     const std::vector < double >&y, const std::string & title = "")

```

```

{ return plot_xy ( x, y, t ) }

/// @brief Plota pares x,y com barra de erro dy a partir de um arquivo.
Gnuplot & plotfile_xy_err ( const std::string & filename ,
    const int column_x = 1 ,
    const int column_y = 2 ,
    const int column_dy = 3, const std::string & title = "" )

/// @brief Plota pares x,y com barra de erro dy a partir de um arquivo.
Gnuplot & PlotFileXYErrorBar( const std::string & filename ,
    const int column_x = 1 ,
    const int column_y = 2 ,
    const int column_dy = 3, const std::string & title = "" )
    { return plotfile_xy_err ( filename , column_x, column_y, column_dy, title ) }

/// @brief Plota pares x,y com barra de erro dy a partir de vetores.
Gnuplot & plot_xy_err ( const std::vector < double > &x,
    const std::vector < double > &y,
    const std::vector < double > &dy,
    const std::string & title = "" );

/// @brief Plota pares x,y com barra de erro dy a partir de vetores.
Gnuplot & PlotVectorXYErrorBar( const std::vector < double > &x,
    const std::vector < double > &y,
    const std::vector < double > &dy,
    const std::string & title = "" )
    { return plot_xy_err ( x, y, dy, title ) }

/// @brief Plota valores de x,y,z a partir de um arquivo de disco.
Gnuplot & plotfile_xyz ( const std::string & filename ,
    const int column_x = 1 ,
    const int column_y = 2 ,
    const int column_z = 3, const std::string & title = "" );

/// @brief Plota valores de x,y,z a partir de um arquivo de disco.
Gnuplot & PlotFile ( const std::string & filename ,
    const int column_x = 1 ,
    const int column_y = 2 ,
    const int column_z = 3, const std::string & title = "" )
    { return plotfile_xyz ( filename , column_x, column_y, column_z, title ) }

```

```

column_y, column_z);

/// @brief Plota valores de x,y,z a partir de vetores.
Gnuplot & plot_xyz (const std::vector< double >&x,
                    const std::vector< double >&y,
                    const std::vector< double >&z, const std::string & title = "

/// @brief Plota valores de x,y,z a partir de vetores.
Gnuplot & PlotVector(const std::vector< double >&x,
                    const std::vector< double >&y,
                    const std::vector< double >&z, const std::string & title = "
                    { return plot_xyz(x,

/// @brief Plota uma equacao da forma y = ax + b, voce fornece os coef
Gnuplot & plot_slope (const double a, const double b, const std::string

/// @brief Plota uma equacao da forma y = ax + b, voce fornece os coef
Gnuplot & PlotSlope (const double a, const double b, const std::string
                    { return plot_slope(a

/// @brief Plota uma equacao fornecida como uma std::string y=f(x).
/// Escrever somente a funcao f(x) e nao y=
/// A variavel independente deve ser x
/// Os operadores binarios aceitos sao:
/// ** exponenciacao,
/// * multiplicacao,
/// / divisao,
/// + adicao,
/// - subtracao,
/// % modulo
/// Os operadores unarios aceitos sao:
/// - menos,
/// ! fatorial
/// Funcoes elementares:
/// rand(x), abs(x), sgn(x), ceil(x), floor(x), int(x), imag(x), real(x)
/// sqrt(x), exp(x), log(x), log10(x), sin(x), cos(x), tan(x), asin(x),
/// atan(x), atan2(y,x), sinh(x), cosh(x), tanh(x), asinh(x), acosh(x),
/// Funcoes especiais:
/// erf(x), erfc(x), inverf(x), gamma(x), igamma(a,x), lgamma(x), ibeta
/// besj0(x), besj1(x), besy0(x), besy1(x), lambertw(x)

```

```

/// Funcoes estatisticas:
/// norm(x), invnorm(x)
Gnuplot & plot_equation (const std::string & equation ,
                        const std::string & title = "");

/// @brief Plota uma equacao fornecida como uma std::string y=f(x).
/// Escrever somente a funcao f(x) e nao y=
/// A variavel independente deve ser x.
/// Exemplo: gnuplot->PlotEquation(CFuncao& obj);
// Deve receber um CFuncao, que tem cast para string.
Gnuplot & PlotEquation(const std::string & equation ,
                    const std::string & title = "")
    { return plot_equation( equation , title ); }

/// @brief Plota uma equacao fornecida na forma de uma std::string z=f(x,y)
/// Escrever somente a funcao f(x,y) e nao z=, as variaveis independentes
Gnuplot & plot_equation3d (const std::string & equation , const std::string & title = "")
    { return plot_equation3d( equation , title ); }

/// @brief Plota uma equacao fornecida na forma de uma std::string z=f(x,y)
/// Escrever somente a funcao f(x,y) e nao z=, as variaveis independentes
// gnuplot->PlotEquation3d(CPolinomio());
Gnuplot & PlotEquation3d (const std::string & equation ,
                        const std::string & title = "")
    { return plot_equation3d( equation , title ); }

/// @brief Plota uma imagem.
Gnuplot & plot_image (const unsigned char *ucPicBuf,
                    const int iWidth, const int iHeight, const std::string & title = "")
    { return plot_image( ucPicBuf, iWidth, iHeight, title ); }

/// @brief Plota uma imagem.
Gnuplot & PlotImage (const unsigned char *ucPicBuf,
                    const int iWidth, const int iHeight, const std::string & title = "")
    { return plot_image( ucPicBuf, iWidth, iHeight, title ); }

///
// Repete o ultimo comando de plotagem, seja plot (2D) ou splot (3D)
// Usado para visualizar plotagens, apos mudar algumas opcoes de plotagem
// ou quando gerando o mesmo grafico para diferentes dispositivos (show)
Gnuplot & replot ();

```



```

// Repete o ultimo comando de plotagem, seja plot (2D) ou splot (3D)
// Usado para visualizar plotagens, apos mudar algumas opcoes de plotag
// ou quando gerando o mesmo grafico para diferentes dispositivos (show
Gnuplot & Replot() { return replot(); }

// Reseta uma sessao do gnuplot (proxima plotagem apaga definicoes pre
Gnuplot & reset_plot ();

// Reseta uma sessao do gnuplot (proxima plotagem apaga definicoes pre
Gnuplot & ResetPlot() { return reset_plot(); }

// Chama funcao reset do gnuplot
Gnuplot & Reset() { this->cmd("reset"); return *this; }

// Reseta uma sessao do gnuplot e seta todas as variaveis para o defau
Gnuplot & reset_all ();

// Reseta uma sessao do gnuplot e seta todas as variaveis para o defau
Gnuplot & ResetAll () { return reset_all(); }

// Verifica se a sessao esta valida
bool is_valid ();

// Verifica se a sessao esta valida
bool IsValid () { return is_valid (); };

};
typedef Gnuplot CGnuplot;
#endif

```

Fonte: produzido pelo autor.

Apresenta-se na listagem 7.16 a implementação da classe CGnuplot.

Listing 7.16: Arquivo CGnuplot.cpp

```

////////////////////////////////////
//
// A C++ interface to gnuplot.
//
// This is a direct translation from the C interface
// written by Nicolas Devillard (ndevilla@free.fr) which
// is available from http://ndevilla.free.fr/gnuplot/.

```

```

//
// As in the C interface this uses pipes and so wont
// run on a system that doesn't have POSIX pipe support
//
// Rajarshi Guha
// e-mail: rguha@indiana.edu, rajarshi@presidency.com
// http://cheminfo.informatics.indiana.edu/rguha/code/cc++/
//
// 07/03/03
//
////////////////////////////////////
//
// A little correction for Win32 compatibility
// and MS VC 6.0 done by V.Chyzhdenka
//
// Notes:
// 1. Added private method Gnuplot::init().
// 2. Temporary file is created in the current
//    folder but not in /tmp.
// 3. Added #ifdef WIN32 e.t.c. where is needed.
// 4. Added private member m_sGNUPlotFileName is
//    a name of executed GNUPlot file.
//
// Viktor Chyzhdenka
// e-mail: chyzhdenka@mail.ru
//
// 20/05/03
//
////////////////////////////////////
//
// corrections for Win32 and Linux compatibility
//
// some member functions added:
// set_GNUPlotPath, set_terminal_std,
// create_tmpfile, get_program_path, file_exists,
// operator<<, replot, reset_all, savetops, showonscreen,
// plotfile_*, plot_xy_err, plot_equation3d
// set, unset: pointsize, grid, *logscale, *autoscale,
// smooth, title, legend, samples, isosamples,
// hidden3d, cbrange, contour

```

```

//
// Markus Burgis
// e-mail: mail@burgis.info
//
// 10/03/08
//
////////////////////////////////////
//
// Modificacoes:
// Traducao para o portugues
// Adicao de novos nomes para os metodos(funcoes)
// Uso de documentacao no formato javadoc/doxygen
// Bueno A.D.
// e-mail: bueno@lenep.uenf.br
// 20/07/08
//
////////////////////////////////////
#include <fstream>           // for std::ifstream
#include <sstream>           // for std::ostringstream
#include <list>              // for std::list
#include <cstdio>            // for FILE, fputs(), fflush(), popen()
#include <cstdlib>           // for getenv()
#include "CGnuplot.h"

// Se estamos no windows // defined for 32 and 64-bit environments
#if defined(WIN32) || defined(_WIN32) || defined(__WIN32__) || defined(__
    #include <io.h>          // for _access(), _mktemp()
    #define GP_MAX_TMP_FILES 27 // 27 temporary files it's Microsoft res
// Se estamos no unix, GNU/Linux, Mac Os X
#elif defined(unix) || defined(__unix) || defined(__unix__) || defined(__
    #include <unistd.h>      // for access(), mkstemp()
    #define GP_MAX_TMP_FILES 64
#else
    #error unsupported or unknown operating system
#endif

//-----
//
// initialize static data
//

```

```

int Gnuplot::tmpfile_num = 0;

// Se estamos no windows
#if defined(WIN32) || defined(_WIN32) || defined(__WIN32__) || defined(__
std::string Gnuplot::m_sGNUPlotFileName = "gnuplot.exe";
std::string Gnuplot::m_sGNUPlotPath = "C:/gnuplot/bin/";
// Se estamos no unix, GNU/Linux, Mac Os X
#elif defined(unix) || defined(__unix) || defined(__unix__) || defined(__
std::string Gnuplot::m_sGNUPlotFileName = "gnuplot";
std::string Gnuplot::m_sGNUPlotPath = "/usr/bin/";
#endif

// Se estamos no windows
#if defined(WIN32) || defined(_WIN32) || defined(__WIN32__) || defined(__
std::string Gnuplot::terminal_std = "windows";
// Se estamos no unix, GNU/Linux
#elif ( defined(unix) || defined(__unix) || defined(__unix__) ) && !defin
std::string Gnuplot::terminal_std = "x11";
// Se estamos Mac Os X
#elif defined(__APPLE__)
std::string Gnuplot::terminal_std = "aqua";
#endif

//-----
//
// define static member function: set Gnuplot path manual
// for windows: path with slash '/' not backslash '\ '
//
bool Gnuplot::set_GNUPlotPath(const std::string &path)
{
    std::string tmp = path + "/" + Gnuplot::m_sGNUPlotFileName;
#if defined(WIN32) || defined(_WIN32) || defined(__WIN32__) || defined(__
        if ( Gnuplot::file_exists(tmp,0) ) // check existence
#elif defined(unix) || defined(__unix) || defined(__unix__) || defined(__
        if ( Gnuplot::file_exists(tmp,1) ) // check existence and execution p
#endif
    {
        Gnuplot::m_sGNUPlotPath = path;
        return true;
    }
}

```

```

        else
        {
            Gnuplot::m_sGNUPlotPath.clear();
            return false;
        }
    }

//-----
// define static member function: set standart terminal, used by showons
// defaults: Windows - win, Linux - x11, Mac - aqua
void Gnuplot::set_terminal_std(const std::string &type)
{
    #if defined(unix) || defined(__unix) || defined(__unix__) || defined(__APPLE__)
        if (type.find("x11") != std::string::npos && getenv("DISPLAY") == NULL)
        {
            throw GnuplotException("Can't find _DISPLAY_ variable");
        }
    #endif

    Gnuplot::terminal_std = type;
    return;
}

//-----
// A string tokenizer taken from http://www.sunsite.ualberta.ca/Document
// /Gnu/libstdc++-2.90.8/html/21_strings/stringtok_std_h.txt
template <typename Container>
void stringtok (Container &container,
                std::string const &in,
                const char * const delimiters = "_\\t\\n")
{
    const std::string::size_type len = in.length();
    std::string::size_type i = 0;

    while ( i < len )
    {
        // eat leading whitespace
        i = in.find_first_not_of (delimiters, i);
    }
}

```

```

    if (i == std::string::npos)
        return;    // nothing left but white space

    // find the end of the token
    std::string::size_type j = in.find_first_of (delimiters , i);

    // push token
    if (j == std::string::npos)
    {
        container.push_back (in.substr(i));
        return;
    }
    else
        container.push_back (in.substr(i , j-i));

    // set up for next loop
    i = j + 1;
}

return;
}

//-----
//
// constructor: set a style during construction
//
Gnuplot::Gnuplot(const std::string &style)
{
    this->init();
    this->set_style(style);
}

//-----
// constructor: open a new session, plot a signal (x)
Gnuplot::Gnuplot(const std::vector<double> &x,
                 const std::string &title ,
                 const std::string &style ,
                 const std::string &labelx ,
                 const std::string &labely)

```

```

{
    this→init ();

    this→set_style ( style );
    this→set_xlabel ( labelx );
    this→set_ylabel ( labely );

    this→plot_x ( x , title );
}

//-----
// constructor: open a new session , plot a signal ( x , y )
Gnuplot::Gnuplot ( const std::vector<double> &x ,
                  const std::vector<double> &y ,
                  const std::string &title ,
                  const std::string &style ,
                  const std::string &labelx ,
                  const std::string &labely )
{
    this→init ();

    this→set_style ( style );
    this→set_xlabel ( labelx );
    this→set_ylabel ( labely );

    this→plot_xy ( x , y , title );
}

//-----
// constructor: open a new session , plot a signal ( x , y , z )
Gnuplot::Gnuplot ( const std::vector<double> &x ,
                  const std::vector<double> &y ,
                  const std::vector<double> &z ,
                  const std::string &title ,
                  const std::string &style ,
                  const std::string &labelx ,
                  const std::string &labely ,
                  const std::string &labelz )
{
    this→init ();

```

```

    this→set_style( style );
    this→set_xlabel( labelx );
    this→set_ylabel( labely );
    this→set_zlabel( labelz );

    this→plot_xyz( x, y, z, title );
}

//-----
// Destructur: needed to delete temporary files
Gnuplot::~Gnuplot()
{
    if ((this→tmpfile_list).size() > 0)
    {
        for (unsigned int i = 0; i < this→tmpfile_list.size(); i++)
            remove( this→tmpfile_list[i].c_str() );

        Gnuplot::tmpfile_num -= this→tmpfile_list.size();
    }

    // A stream opened by popen() should be closed by pclose()
    #if defined(WIN32) || defined(_WIN32) || defined(__WIN32__) || defined(__
        if (_pclose(this→gnucmd) == -1)
    #elif defined(unix) || defined(__unix) || defined(__unix__) || defined(__
        if (pclose(this→gnucmd) == -1)
    #endif
        true; //throw GnuplotException("Problem closing communication to g
}

//-----
// Resets a gnuplot session (next plot will erase previous ones)
Gnuplot& Gnuplot::reset_plot()
{
    if (this→tmpfile_list.size() > 0)
    {
        for (unsigned int i = 0; i < this→tmpfile_list.size(); i++)
            remove(this→tmpfile_list[i].c_str());

        Gnuplot::tmpfile_num -= this→tmpfile_list.size();
    }
}

```



```

        this→tmpfile_list.clear();
    }

    this→nplots = 0;

    return *this;
}

//-----
// resets a gnuplot session and sets all variables to default
Gnuplot& Gnuplot::reset_all()
{
    if (this→tmpfile_list.size() > 0)
    {
        for (unsigned int i = 0; i < this→tmpfile_list.size(); i++)
            remove(this→tmpfile_list[i].c_str());

        Gnuplot::tmpfile_num -= this→tmpfile_list.size();
        this→tmpfile_list.clear();
    }

    this→nplots = 0;
    this→cmd("reset");
    this→cmd("clear");
    this→pstyle = "points";
    this→smooth = "";
    this→showonscreen();

    return *this;
}

//-----
// Find out if valid is true
bool Gnuplot::is_valid()
{
    return(this→valid);
}

//-----
// replot repeats the last plot or splot command

```

```

Gnuplot& Gnuplot::replot()
{
    if (this->nplots > 0)
    {
        this->cmd("replot");
    }

    return *this;
}

```

---

```

//-----
// Change the plotting style of a gnuplot session
Gnuplot& Gnuplot::set_style(const std::string &stylestr)
{
    if (stylestr.find("lines") == std::string::npos &&
        stylestr.find("points") == std::string::npos &&
        stylestr.find("linespoints") == std::string::npos &&
        stylestr.find("impulses") == std::string::npos &&
        stylestr.find("dots") == std::string::npos &&
        stylestr.find("steps") == std::string::npos &&
        stylestr.find("fsteps") == std::string::npos &&
        stylestr.find("histeps") == std::string::npos &&
        stylestr.find("boxes") == std::string::npos &&
        // 1-4 columns of data are required
        stylestr.find("filledcurves") == std::string::npos &&
        stylestr.find("histograms") == std::string::npos )
    // only for one data column
    //          stylestr.find("labels") == std::string::npos &&
    // 3 columns of data are required
    //          stylestr.find("xerrorbars") == std::string::npos &&
    // 3-4 columns of data are required
    //          stylestr.find("xerrorlines") == std::string::npos &&
    // 3-4 columns of data are required
    //          stylestr.find("errorbars") == std::string::npos &&
    // 3-4 columns of data are required
    //          stylestr.find("errorlines") == std::string::npos &&
    // 3-4 columns of data are required
    //          stylestr.find("yerrorbars") == std::string::npos &&
    // 3-4 columns of data are required

```

```

//          stylestr.find("yerrorlines")    == std::string::npos    @@
// 3-4 columns of data are required
//          stylestr.find("boxerrorbars")    == std::string::npos    @@
// 3-5 columns of data are required
//          stylestr.find("xyerrorbars")     == std::string::npos    @@
// 4,6,7 columns of data are required
//          stylestr.find("xyerrorlines")    == std::string::npos    @@
// 4,6,7 columns of data are required
//          stylestr.find("boxxyerrorbars")  == std::string::npos    @@
// 4,6,7 columns of data are required
//          stylestr.find("financebars")     == std::string::npos    @@
// 5 columns of data are required
//          stylestr.find("candlesticks")    == std::string::npos    @@
// 5 columns of data are required
//          stylestr.find("vectors")         == std::string::npos    @@
//          stylestr.find("image")           == std::string::npos    @@
//          stylestr.find("rgbimage")        == std::string::npos    @@
//          stylestr.find("pm3d")            == std::string::npos    )
{
    this→pstyle = std::string("points");
}
else
{
    this→pstyle = stylestr;
}

return *this;
}

//-----
// smooth: interpolation and approximation of data
Gnuplot& Gnuplot::set_smooth(const std::string &stylestr)
{
    if (stylestr.find("unique")    == std::string::npos    &&
        stylestr.find("frequency") == std::string::npos    &&
        stylestr.find("csplines") == std::string::npos    &&
        stylestr.find("acsplines") == std::string::npos    &&
        stylestr.find("bezier")    == std::string::npos    &&
        stylestr.find("sbezier")   == std::string::npos    )
    {

```

```

        this->smooth = "";
    }
    else
    {
        this->smooth = stylestr;
    }

    return *this;
}

//-----
// unset smooth
Gnuplot& Gnuplot::unset_smooth()
{
    this->smooth = "";

    return *this;
}

//-----
// sets terminal type to windows / x11
Gnuplot& Gnuplot::showonscreen()
{
    this->cmd("set_output");
    this->cmd("set_terminal_" + Gnuplot::terminal_std);

    return *this;
}

//-----
// saves a gnuplot session to a postscript file
Gnuplot& Gnuplot::savetops(const std::string &filename)
{
    //      this->cmd("set terminal postscript color");          // Tipo de terminal
    //
    //      std::ostream cmdstr;                                // Muda o nome do arquivo
    //      cmdstr << "set output \"\" << filename << ".ps\"\"; // Nome do arquivo
    //      this->cmd(cmdstr.str());
    //      this->replot();                                         // Replota o gráfico
    //

```

```

//      ShowOnScreen ();                                     // Volta para

    this->cmd("set term png size 1800,1200");

    std::ostream cmdstr;
    cmdstr << "set output \"./Src/" << filename << ".png\"";
    this->cmd(cmdstr.str());
    this->Replot();

    return *this;
}

//-----
// saves a gnuplot session to a png file and return do on screen termina
Gnuplot& Gnuplot::savetopng(const std::string &filename)
{
//                                     // Muda o termo
//      this->cmd("set term png enhanced size 1280,960"); // Tipo de termo
//
//      std::ostream cmdstr;                                     // Muda o nome
//      cmdstr << "set output \"" << filename << ".png\""; // Nome do arquivo
//      this->cmd(cmdstr.str());
//      this->replot();                                           // Replota o gráfico
//                                     // Retorna o termo
//      ShowOnScreen ();                                         // Volta para
//      this->replot();                                           // Replota o gráfico
    SaveTo(filename,"png", "enhanced_size_1280,960");

    return *this;
}

//-----
// saves a gnuplot session to a jpeg file and return do on screen termina
Gnuplot& Gnuplot::savetojpeg(const std::string &filename)
{
//                                     // Muda o termo
//      this->cmd("set term jpeg enhanced size 1280,960"); // Tipo de termo
//
//      std::ostream cmdstr;                                     // Muda o nome
//      cmdstr << "set output \"" << filename << ".jpeg\""; // Nome do arquivo
//      this->cmd(cmdstr.str());

```

```

//      this->replot();
//
//      ShowOnScreen ();
//      this->replot();
SaveTo(filename,"jpeg", "enhanced_size_1280,960");

    return *this;
}

```

---

```

//-----
// saves a gnuplot session to spectific terminal and output file
// @filename: name of disc file
// @terminal_type: type of terminal
// @flags: additional information specitif to terminal type
// Ex:
// grafico.SaveTo("pressao_X_temperatura","png", "enhanced size 1280,960
// grafico.TerminalType("png").SaveFile(pressao_X_temperatura); pense nis
Gnuplot& Gnuplot::SaveTo(const std::string &filename,const std::string &t
{
    // Muda o termino
    this->cmd("set_term_" + terminal_type + "_" + flags); // Tipo de t
    std::ostream cmdstr; // Muda o nome do
    cmdstr << "set_output_" << filename << "." << terminal_type << "\"
    this->cmd(cmdstr.str());
    this->replot(); // Replota o graf
    // Retorna o term

    ShowOnScreen ();
    this->replot(); // Replota o graf

    return *this;
}

```

---

```

//-----
// Switches legend on
Gnuplot& Gnuplot::set_legend(const std::string &position)
{
    std::ostream cmdstr;
    cmdstr << "set_key_" << position;

    this->cmd(cmdstr.str());
}

```

```
        return *this;
    }

//-----
// Switches legend off
Gnuplot& Gnuplot::unset_legend()
{
    this->cmd("unset_key");

    return *this;
}

//-----
// Turns grid on
Gnuplot& Gnuplot::set_grid()
{
    this->cmd("set_grid");

    return *this;
}

//-----
// Turns grid off
Gnuplot& Gnuplot::unset_grid()
{
    this->cmd("unset_grid");

    return *this;
}

//-----
// turns on log scaling for the x axis
Gnuplot& Gnuplot::set_xlogscale(const double base)
{
    std::ostringstream cmdstr;

    cmdstr << "set_logscale_x_" << base;
    this->cmd(cmdstr.str());
}
```

```
        return *this;
    }

//-----
// turns on log scaling for the y axis
Gnuplot& Gnuplot::set_ylogscale(const double base)
{
    std::ostream cmdstr;

    cmdstr << "set_logscale_y_" << base;
    this->cmd(cmdstr.str());

    return *this;
}

//-----
// turns on log scaling for the z axis
Gnuplot& Gnuplot::set_zlogscale(const double base)
{
    std::ostream cmdstr;

    cmdstr << "set_logscale_z_" << base;
    this->cmd(cmdstr.str());

    return *this;
}

//-----
// turns off log scaling for the x axis
Gnuplot& Gnuplot::unset_xlogscale()
{
    this->cmd("unset_logscale_x");
    return *this;
}

//-----
// turns off log scaling for the y axis
Gnuplot& Gnuplot::unset_ylogscale()
{
    this->cmd("unset_logscale_y");
```



```
        return *this;
    }

//-----
// turns off log scaling for the z axis
Gnuplot& Gnuplot::unset_zlogscale()
{
    this->cmd("unset_logscale_z");
    return *this;
}

//-----
// scales the size of the points used in plots
Gnuplot& Gnuplot::set_pointsize(const double pointsize)
{
    std::ostringstream cmdstr;
    cmdstr << "set_pointsize_" << pointsize;
    this->cmd(cmdstr.str());

    return *this;
}

//-----
// set isoline density (grid) for plotting functions as surfaces
Gnuplot& Gnuplot::set_samples(const int samples)
{
    std::ostringstream cmdstr;
    cmdstr << "set_samples_" << samples;
    this->cmd(cmdstr.str());

    return *this;
}

//-----
// set isoline density (grid) for plotting functions as surfaces
Gnuplot& Gnuplot::set_isosamples(const int isolines)
{
    std::ostringstream cmdstr;
    cmdstr << "set_isosamples_" << isolines;
```

```

        this→cmd(cmdstr.str());

        return *this;
    }

//-----
// enables hidden line removal for surface plotting
Gnuplot& Gnuplot::set_hidden3d()
{
    this→cmd("set_hidden3d");

    return *this;
}

//-----
// disables hidden line removal for surface plotting
Gnuplot& Gnuplot::unset_hidden3d()
{
    this→cmd("unset_hidden3d");

    return *this;
}

//-----
// enables contour drawing for surfaces set contour {base | surface | both}
Gnuplot& Gnuplot::set_contour(const std::string &position)
{
    if (position.find("base") == std::string::npos &&
        position.find("surface") == std::string::npos &&
        position.find("both") == std::string::npos )
    {
        this→cmd("set_contour_base");
    }
    else
    {
        this→cmd("set_contour_" + position);
    }

    return *this;
}

```

---

```
//-----  
// disables contour drawing for surfaces  
Gnuplot& Gnuplot::unset_contour()  
{  
    this→cmd("unset_contour");  
  
    return *this;  
}  
  
//-----  
// enables the display of surfaces (for 3d plot)  
Gnuplot& Gnuplot::set_surface()  
{  
    this→cmd("set_surface");  
  
    return *this;  
}  
  
//-----  
// disables the display of surfaces (for 3d plot)  
Gnuplot& Gnuplot::unset_surface()  
{  
    this→cmd("unset_surface");  
  
    return *this;  
}  
  
//-----  
// Sets the title of a gnuplot session  
Gnuplot& Gnuplot::set_title(const std::string &title)  
{  
    std::ostringstream cmdstr;  
  
    cmdstr << "set_title \" " << title << " \"";  
    this→cmd(cmdstr.str());  
  
    return *this;  
}
```

---

```
//-----  
// Clears the title of a gnuplot session  
Gnuplot& Gnuplot::unset_title()  
{  
    this→set_title("");  
  
    return *this;  
}  
  
//-----  
// set labels  
// set the xlabel  
Gnuplot& Gnuplot::set_xlabel(const std::string &label)  
{  
    std::ostringstream cmdstr;  
  
    cmdstr << "set_xlabel\" << label << "\"";  
    this→cmd(cmdstr.str());  
  
    return *this;  
}  
  
//-----  
// set the ylabel  
Gnuplot& Gnuplot::set_ylabel(const std::string &label)  
{  
    std::ostringstream cmdstr;  
  
    cmdstr << "set_ylabel\" << label << "\"";  
    this→cmd(cmdstr.str());  
  
    return *this;  
}  
  
//-----  
// set the zlabel  
Gnuplot& Gnuplot::set_zlabel(const std::string &label)  
{  
    std::ostringstream cmdstr;
```

```

    cmdstr << "set_zlabel\" << label << "\"";
    this->cmd(cmdstr.str());

    return *this;
}

//-----
// set range
// set the xrange
Gnuplot& Gnuplot::set_xrange(const int iFrom,
                             const int iTo)
{
    std::ostringstream cmdstr;

    cmdstr << "set_xrange[" << iFrom << ":" << iTo << "]";
    this->cmd(cmdstr.str());

    return *this;
}

//-----
// set autoscale x
Gnuplot& Gnuplot::set_xautoscale()
{
    this->cmd("set_xrange_restore");
    this->cmd("set_autoscale_x");

    return *this;
}

//-----
// set the yrange
Gnuplot& Gnuplot::set_yrange(const int iFrom, const int iTo)
{
    std::ostringstream cmdstr;

    cmdstr << "set_yrange[" << iFrom << ":" << iTo << "]";
    this->cmd(cmdstr.str());

    return *this;
}

```

```
}

//-----
// set autoscale y
Gnuplot& Gnuplot::set_yautoscale()
{
    this→cmd("set_yrange_restore");
    this→cmd("set_autoscale_y");

    return *this;
}

//-----
// set the zrange
Gnuplot& Gnuplot::set_zrange(const int iFrom,
                             const int iTo)
{
    std::ostringstream cmdstr;

    cmdstr << "set_zrange[" << iFrom << ":" << iTo << "]" ;
    this→cmd(cmdstr.str());

    return *this;
}

//-----
// set autoscale z
Gnuplot& Gnuplot::set_zautoscale()
{
    this→cmd("set_zrange_restore");
    this→cmd("set_autoscale_z");

    return *this;
}

//-----
// set the palette range
Gnuplot& Gnuplot::set_cbrange(const int iFrom,
                              const int iTo)
{
```

```

    std::ostream cmdstr;

    cmdstr << "set_cbrange[" << iFrom << ":" << iTo << "];";
    this->cmd(cmdstr.str());

    return *this;
}

//-----
// Plots a linear equation  $y=ax+b$  (where you supply the
// slope  $a$  and intercept  $b$ )
Gnuplot& Gnuplot::plot_slope(const double a,
                             const double b,
                             const std::string &title)
{
    std::ostream cmdstr;

    // command to be sent to gnuplot
    if (this->nplots > 0 && this->two_dim == true)
        cmdstr << "replot_";
    else
        cmdstr << "plot_";

    cmdstr << a << "_*_x+_ " << b << "_title_\n";

    if (title == "")
        cmdstr << "f(x)_=" << a << "_*_x+_ " << b;
    else
        cmdstr << title;

    cmdstr << "\n_with_" << this->pstyle;

    // Do the actual plot
    this->cmd(cmdstr.str());

    return *this;
}

//-----
// Plot an equation which is supplied as a std::string  $y=f(x)$  (only  $f(x)$ )

```

```

Gnuplot& Gnuplot::plot_equation(const std::string &equation ,
                                const std::string &title)
{
    std::ostringstream cmdstr;

    // command to be sent to gnuplot
    if (this->nplots > 0 && this->two_dim == true)
        cmdstr << "replot_";
    else
        cmdstr << "plot_";

    cmdstr << equation << "_title_\\";

    if (title == "")
        cmdstr << "f(x)_=_ " << equation;
    else
        cmdstr << title;

    cmdstr << "\\_with_" << this->pstyle;

    // Do the actual plot
    this->cmd(cmdstr.str());

    return *this;
}

//-----
// plot an equation supplied as a std::string y=(x)
Gnuplot& Gnuplot::plot_equation3d(const std::string &equation ,
                                const std::string &title)
{
    std::ostringstream cmdstr;

    // command to be sent to gnuplot
    if (this->nplots > 0 && this->two_dim == false)
        cmdstr << "replot_";
    else
        cmdstr << "splot_";

    cmdstr << equation << "_title_\\";

```



```

    if ( title == "" )
        cmdstr << "f(x,y)_" << equation;
    else
        cmdstr << title;

    cmdstr << "\"_with_" << this->pstyle;

    // Do the actual plot
    this->cmd(cmdstr.str());

    return *this;
}

//-----
// Plots a 2d graph from a list of doubles (x) saved in a file
Gnuplot& Gnuplot::plotfile_x(const std::string &filename ,
                             const int column ,
                             const std::string &title)
{
    // check if file exists
    if( !(Gnuplot::file_exists(filename,4)) ) // check existence and read
    {
        std::ostream except;
        if( !(Gnuplot::file_exists(filename,0)) ) // check existence
            except << "File_" << filename << "\"_does_not_exist";
        else
            except << "No_read_permission_for_File_" << filename << "\"";
        throw GnuplotException( except.str() );
        return *this;
    }

    std::ostream cmdstr;

    // command to be sent to gnuplot
    if (this->nplots > 0 && this->two_dim == true)
        cmdstr << "replot_";
    else
        cmdstr << "plot_";

```

```

    cmdstr << "\" << filename << "\"_using_" << column;

    if (title == "")
        cmdstr << "_notitle_";
    else
        cmdstr << "_title_" << title << "\"_";

    if(smooth == "")
        cmdstr << "with_" << this->pstyle;
    else
        cmdstr << "smooth_" << this->smooth;

    // Do the actual plot
    this->cmd(cmdstr.str()); //nplots++; two_dim = true;  already in this

    return *this;
}

//-----
// Plots a 2d graph from a list of doubles: x
Gnuplot& Gnuplot::plot_x(const std::vector<double> &x,
                        const std::string &title)
{
    if (x.size() == 0)
    {
        throw GnuplotException("std::vector_too_small");
        return *this;
    }

    std::ofstream tmp;
    std::string name = create_tmpfile(tmp);
    if (name == "")
        return *this;

    // write the data to file
    for (unsigned int i = 0; i < x.size(); i++)
        tmp << x[i] << std::endl;

    tmp.flush();
    tmp.close();

```

```

        this→plotfile_x(name, 1, title);

        return *this;
    }

//-----
// Plots a 2d graph from a list of doubles (x y) saved in a file
Gnuplot& Gnuplot::plotfile_xy(const std::string &filename,
                               const int column_x,
                               const int column_y,
                               const std::string &title)
{
    // check if file exists
    if( !(Gnuplot::file_exists(filename,4)) ) // check existence and read
    {
        std::ostringstream except;
        if( !(Gnuplot::file_exists(filename,0)) ) // check existence
            except << "File_\\"" << filename << "\"_does_not_exist";
        else
            except << "No_read_permission_for_File_\\"" << filename << "\"";
        throw GnuplotException( except.str() );
        return *this;
    }

    std::ostringstream cmdstr;

    // command to be sent to gnuplot
    if (this→nplots > 0 && this→two_dim == true)
        cmdstr << "replot_";
    else
        cmdstr << "plot_";

    cmdstr << "\"\" << filename << "\"_using_" << column_x << ":" << column_y;

    if (title == "")
        cmdstr << "_notitle_";
    else
        cmdstr << "_title_\\"" << title << "\"_";

```

```

    if(smooth == "")
        cmdstr << "with_" << this->pstyle;
    else
        cmdstr << "smooth_" << this->smooth;

    // Do the actual plot
    this->cmd(cmdstr.str());

    return *this;
}

//-----
// Plots a 2d graph from a list of doubles: x y
Gnuplot& Gnuplot::plot_xy(const std::vector<double> &x,
                        const std::vector<double> &y,
                        const std::string &title)
{
    if (x.size() == 0 || y.size() == 0)
    {
        throw GnuplotException("std::vectors_too_small");
        return *this;
    }

    if (x.size() != y.size())
    {
        throw GnuplotException("Length_of_the_std::vectors_differs");
        return *this;
    }

    std::ofstream tmp;
    std::string name = create_tmpfile(tmp);
    if (name == "")
        return *this;

    // write the data to file
    for (unsigned int i = 0; i < x.size(); i++)
        tmp << x[i] << "_" << y[i] << std::endl;

    tmp.flush();
    tmp.close();

```

```

        this→plotfile_xy(name, 1, 2, title);

        return *this;
    }

//-----
// Plots a 2d graph with errorbars from a list of doubles (x y dy) saved
Gnuplot& Gnuplot::plotfile_xy_err(const std::string &filename ,
                                const int column_x,
                                const int column_y,
                                const int column_dy,
                                const std::string &title)
{
    // check if file exists
    if( !(Gnuplot::file_exists(filename,4)) ) // check existence and read
    {
        std::ostreamstream except;
        if( !(Gnuplot::file_exists(filename,0)) ) // check existence
            except << "File_\\"" << filename << "\"_does_not_exist";
        else
            except << "No_read_permission_for_File_\\"" << filename << "\"";
        throw GnuplotException( except.str() );
        return *this;
    }

    std::ostreamstream cmdstr;

    // command to be sent to gnuplot
    if (this→nplots > 0 && this→two_dim == true)
        cmdstr << "replot_";
    else
        cmdstr << "plot_";

    cmdstr << "\"\" << filename << "\"_using_" << column_x << ":" << column_y << " using _";

    if (title == "")
        cmdstr << "_notitle_";
    else
        cmdstr << "_title_\\"" << title << "\"_";
}

```

```

cmdstr << "with_" << this->pstyle << ",_" << filename << "\"_using_
    << column_x << ":" << column_y << ":" << column_dy << "_notitl

// Do the actual plot
this->cmd(cmdstr.str());

return *this;
}

//-----
// plot x,y pairs with dy errorbars
Gnuplot& Gnuplot::plot_xy_err(const std::vector<double> &x,
                                const std::vector<double> &y,
                                const std::vector<double> &dy,
                                const std::string &title)
{
    if (x.size() == 0 || y.size() == 0 || dy.size() == 0)
    {
        throw GnuplotException("std::vectors_too_small");
        return *this;
    }

    if (x.size() != y.size() || y.size() != dy.size())
    {
        throw GnuplotException("Length_of_the_std::vectors_differs");
        return *this;
    }

    std::ofstream tmp;
    std::string name = create_tmpfile(tmp);
    if (name == "")
        return *this;

    // write the data to file
    for (unsigned int i = 0; i < x.size(); i++)
        tmp << x[i] << "_" << y[i] << "_" << dy[i] << std::endl;

    tmp.flush();
    tmp.close();
}

```

```

    // Do the actual plot
    this->plotfile_xy_err(name, 1, 2, 3, title);

    return *this;
}

//-----
// Plots a 3d graph from a list of doubles (x y z) saved in a file
Gnuplot& Gnuplot::plotfile_xyz(const std::string &filename,
                                const int column_x,
                                const int column_y,
                                const int column_z,
                                const std::string &title)
{
    // check if file exists
    if( !(Gnuplot::file_exists(filename,4)) ) // check existence and read
    {
        std::ostringstream except;
        if( !(Gnuplot::file_exists(filename,0)) ) // check existence
            except << "File_\\"" << filename << "\"_does_not_exist";
        else
            except << "No_read_permission_for_File_\\"" << filename << "\"";
        throw GnuplotException( except.str() );
        return *this;
    }

    std::ostringstream cmdstr;

    // command to be sent to gnuplot
    if (this->nplots > 0 && this->two_dim == false)
        cmdstr << "replot_";
    else
        cmdstr << "splot_";

    cmdstr << "\"\" << filename << "\"_using_" << column_x << ":" << column_y << " using _";

    if (title == "")
        cmdstr << "_notitle_with_" << this->pstyle;

```

```

    else
        cmdstr << "_title_" << title << "\"_with_" << this->pstyle;

        // Do the actual plot
        this->cmd(cmdstr.str());

    return *this;
}

//-----
// Plots a 3d graph from a list of doubles: x y z
Gnuplot& Gnuplot::plot_xyz(const std::vector<double> &x,
                           const std::vector<double> &y,
                           const std::vector<double> &z,
                           const std::string &title)
{
    if (x.size() == 0 || y.size() == 0 || z.size() == 0)
    {
        throw GnuplotException("std::vectors_too_small");
        return *this;
    }

    if (x.size() != y.size() || x.size() != z.size())
    {
        throw GnuplotException("Length_of_the_std::vectors_differs");
        return *this;
    }

    std::ofstream tmp;
    std::string name = create_tmpfile(tmp);
    if (name == "")
        return *this;

    // write the data to file
    for (unsigned int i = 0; i < x.size(); i++)
    {
        tmp << x[i] << "_" << y[i] << "_" << z[i] << std::endl;
    }
}

```



```

    tmp.flush ();
    tmp.close ();

    this->plotfile_xyz(name, 1, 2, 3, title );

    return *this ;
}

//-----
/// * note that this function is not valid for versions of GNUPlot below
Gnuplot& Gnuplot::plot_image(const unsigned char * ucPicBuf,
                             const int iWidth,
                             const int iHeight,
                             const std::string &title)
{
    std::ofstream tmp;
    std::string name = create_tmpfile(tmp);
    if (name == "")
        return *this;

    // write the data to file
    int iIndex = 0;
    for(int iRow = 0; iRow < iHeight; iRow++)
    {
        for(int iColumn = 0; iColumn < iWidth; iColumn++)
        {
            tmp << iColumn << "_" << iRow << "_" << static_cast<float>(u
        }
    }

    tmp.flush ();
    tmp.close ();

    std::ostringstream cmdstr;

    // command to be sent to gnuplot

```

```

    if (this->nplots > 0 && this->two_dim == true)
        cmdstr << "replot_";
    else
        cmdstr << "plot_";

    if (title == "")
        cmdstr << "\"_with_image";
    else
        cmdstr << "\"_title_\"_<< title << "\"_with_image";

    // Do the actual plot
    this->cmd(cmdstr.str());

    return *this;
}

//-----
// Sends a command to an active gnuplot session
Gnuplot& Gnuplot::cmd(const std::string &cmdstr)
{
    if( !(this->valid) )
    {
        return *this;
    }

    // int fputs ( const char * str, FILE * stream );
    // writes the string str to the stream.
    // The function begins copying from the address specified (str) until
    // terminating null character ('\0'). This final null-character is not
    fputs( (cmdstr+"\n").c_str(), this->gnucmd );

    // int fflush ( FILE * stream );
    // If the given stream was open for writing and the last i/o operation
    // any unwritten data in the output buffer is written to the file.
    // If the argument is a null pointer, all open files are flushed.
    // The stream remains open after this call.
    fflush( this->gnucmd );

    if( cmdstr.find("replot") != std::string::npos )

```

```

    {
        return *this;
    }
    else if( cmdstr.find("splot") != std::string::npos )
    {
        this->two_dim = false;
        this->nplots++;
    }
    else if( cmdstr.find("plot") != std::string::npos )
    {
        this->two_dim = true;
        this->nplots++;
    }
    return *this;
}

//-----
// Sends a command to an active gnuplot session, identical to cmd()
Gnuplot& Gnuplot::operator<<(const std::string &cmdstr)
{
    this->cmd(cmdstr);
    return *this;
}

//-----
// Opens up a gnuplot session, ready to receive commands
void Gnuplot::init()
{
    // char * getenv ( const char * name ); get value of an environment
    // Retrieves a C string containing the value of the environment variable
    // name is specified as argument.
    // If the requested variable is not part of the environment list, the
    #if ( defined(unix) || defined(__unix) || defined(__unix__) ) && !defined(_WIN32)
        if (getenv("DISPLAY") == NULL)
        {
            this->valid = false;
            throw GnuplotException("Can't find_DISPLAY_variable");
        }
    #endif

```

```

// if gnuplot not available
if (!Gnuplot::get_program_path())
{
    this→valid = false;
    throw GnuplotException("Can't find gnuplot");
}

// open pipe
std::string tmp = Gnuplot::m_sGNUPlotPath + "/" + Gnuplot::m_sGNUPlotPath;

// FILE *popen(const char *command, const char *mode);
// The popen() function shall execute the command specified by the string command
// create a pipe between the calling program and the executed command
// return a pointer to a stream that can be used to either read from or write to the command
#if defined(WIN32) || defined(_WIN32) || defined(__WIN32__) || defined(_WIN32_WINNT)
    this→gncmd = _popen(tmp.c_str(), "w");
#elif defined(unix) || defined(__unix) || defined(__unix__) || defined(_XOPEN_SOURCE)
    this→gncmd = popen(tmp.c_str(), "w");
#endif

// popen() shall return a pointer to an open stream that can be used to read from or write to the command
// Otherwise, it shall return a null pointer and may set errno to indicate the error
if (!this→gncmd)
{
    this→valid = false;
    throw GnuplotException("Couldn't open connection to gnuplot");
}

this→nplots = 0;
this→valid = true;
this→smooth = "";

//set terminal type
this→showonscreen();

return;
}

//-----
// Find out if a command lives in m_sGNUPlotPath or in PATH

```

```

bool Gnuplot::get_program_path()
{
    // first look in m_sGNUPlotPath for Gnuplot
    std::string tmp = Gnuplot::m_sGNUPlotPath + "/" + Gnuplot::m_sGNUPlotFile;

    #if defined(WIN32) || defined(_WIN32) || defined(__WIN32__) || defined(_WIN64)
        if ( Gnuplot::file_exists(tmp,0) ) // check existence
    #elif defined(unix) || defined(__unix) || defined(__unix__) || defined(_POSIX_C_SOURCE)
        if ( Gnuplot::file_exists(tmp,1) ) // check existence and execution permission
    #endif
    {
        return true;
    }

    // second look in PATH for Gnuplot
    char *path;
    // Retrieves a C string containing the value of the environment variable PATH
    path = getenv("PATH");

    if (path == NULL)
    {
        throw GnuplotException("Path_is_not_set");
        return false;
    }
    else
    {
        std::list<std::string> ls;
        // split path (one long string) into list ls of strings
    #if defined(WIN32) || defined(_WIN32) || defined(__WIN32__) || defined(_WIN64)
        strtok(path, ";");
    #elif defined(unix) || defined(__unix) || defined(__unix__) || defined(_POSIX_C_SOURCE)
        strtok(path, ":");
    #endif

        // scan list for Gnuplot program files
        for (std::list<std::string>::const_iterator i = ls.begin(); i != ls.end(); i++)
        {
            tmp = (*i) + "/" + Gnuplot::m_sGNUPlotFileName;
        }
    #if defined(WIN32) || defined(_WIN32) || defined(__WIN32__) || defined(_WIN64)
        if ( Gnuplot::file_exists(tmp,0) ) // check existence
    #elif defined(unix) || defined(__unix) || defined(__unix__) || defined(_POSIX_C_SOURCE)
        if ( Gnuplot::file_exists(tmp,1) ) // check existence and execution permission
    #endif
    {
        return true;
    }
    return false;
}

```

```

        if ( Gnuplot::file_exists(tmp,1) ) // check existence and execute
#endif
        {
            Gnuplot::m_sGNUPlotPath = *i; // set m_sGNUPlotPath
            return true;
        }
    }

    tmp = "Can't find gnuplot neither in PATH nor in \" + Gnuplot::m_sGNUPlotPath;
    throw GnuplotException(tmp);

    Gnuplot::m_sGNUPlotPath = "";
    return false;
}
}

//-----
// check if file exists
bool Gnuplot::file_exists(const std::string &filename, int mode)
{
    if ( mode < 0 || mode > 7)
    {
        throw std::runtime_error("In function \"Gnuplot::file_exists\": mode is not valid");
        return false;
    }

    // int _access(const char *path, int mode);
    // returns 0 if the file has the given mode,
    // it returns -1 if the named file does not exist or is not accessible
    // mode = 0 (F_OK) (default): checks file for existence only
    // mode = 1 (X_OK): execution permission
    // mode = 2 (W_OK): write permission
    // mode = 4 (R_OK): read permission
    // mode = 6 : read and write permission
    // mode = 7 : read, write and execution permission
    #if defined(WIN32) || defined(_WIN32) || defined(__WIN32__) || defined(__NT__)
        if (_access(filename.c_str(), mode) == 0)
    #elif defined(unix) || defined(__unix) || defined(__unix__) || defined(__posix)
        if (access(filename.c_str(), mode) == 0)
    #endif
}

```

```

    {
        return true;
    }
    else
    {
        return false;
    }
}

//-----
// Opens a temporary file
std::string Gnuplot::create_tmpfile(std::ofstream &tmp)
{
    #if defined(WIN32) || defined(_WIN32) || defined(__WIN32__) || defined(__
        char name[] = "gnuplotiXXXXXX"; //tmp file in working directory
    #elif defined(unix) || defined(__unix) || defined(__unix__) || defined(__
        char name[] = "/tmp/gnuplotiXXXXXX"; // tmp file in /tmp
    #endif

    // check if maximum number of temporary files reached
    if (Gnuplot::tmpfile_num == GP_MAX_TMP_FILES - 1)
    {
        std::ostringstream except;
        except << "Maximum_number_of_temporary_files_reached_" << GP_MAX
            << "):_cannot_open_more_files" << std::endl;

        throw GnuplotException( except.str() );
        return "";
    }

    //int mkstemp(char *name);
    // shall replace the contents of the string pointed to by "name" by a
    // and return a file descriptor for the file open for reading and wr
    // Otherwise, -1 shall be returned if no suitable file could be creat
    // The string in template should look like a filename with six traili
    // mkstemp() replaces each 'X' with a character from the portable file
    // The characters are chosen such that the resulting name does not d

```

```

    // open temporary files for output
#if defined(WIN32) || defined(_WIN32) || defined(__WIN32__) || defined(__
    if (_mktemp(name) == NULL)
#elif defined(unix) || defined(__unix) || defined(__unix__) || defined(__
    if (mkstemp(name) == -1)
#endif
    {
        std::ostringstream except;
        except << "Cannot_create_temporary_file_" << name << "\"";
        throw GnuplotException(except.str());
        return "";
    }

    tmp.open(name);
    if (tmp.bad())
    {
        std::ostringstream except;
        except << "Cannot_create_temporary_file_" << name << "\"";
        throw GnuplotException(except.str());
        return "";
    }

    // Save the temporary filename
    this->tmpfile_list.push_back(name);
    Gnuplot::tmpfile_num++;

    return name;
}

```

Fonte: produzido pelo autor.

Apresenta-se na listagem 7.17 a implementação da classe CPlotter.

Listing 7.17: Arquivo CPlotter.h

```

#ifndef CLOTTER_H
#define CLOTTER_H

#include "../Core/CWellLog.h"
#include "../Interpretation/CInterpreter.h"
#include "CGnuplot.h"
#include <vector>
#include <string>

```



```

class CPlotter {
public:
    void PlotCurve(const CWellLog& rWell, const std::string& strCurveName)
    void PlotMultiple(const CWellLog& rWell, const std::vector<std::string>& curves)
    void PlotSelectedTracks(const CWellLog& rWell, const std::vector<std::string>& tracks)

    void PlotCompareCurves(const CWellLog& well,
                           const std::vector<std::string>& curves,
                           bool overlay = true,
                           bool normalize = true);

    void PlotInterpretationBasic(const CWellLog& well,
                                const VshParams& vp,
                                const SepParams& sp,
                                const PayParams& pp);

    void PlotInterpretationBasic(const CWellLog& well,
                                const VshParams& vp,
                                const SepParams& sp);

    void PlotWithIntervals(const CWellLog& rWell,
                          const std::vector<std::string>& curvesToPlot,
                          const std::vector<Interval>& ivals);

};

#endif

```

Fonte: produzido pelo autor.

Apresenta-se na listagem 7.18 a implementação da classe CPlotter.

Listing 7.18: Arquivo CPlotter.cpp

```

// CPlotter.cpp
#include <algorithm>
#include <filesystem>
#include <fstream>
#include <iostream>
#include <optional>
#include <sstream>
#include <string>

```

```

#include <vector>
#include <cctype>
#include <cmath>          // std::isfinite, std::isnan, std::fabs

#include "CPlotter.h"
#include "CWellLog.h"
#include "CLogCurve.h"
#include "CGnuplot.h"
#include "CLogCurveUtilities.h"

// =====
// Helpers internos
// =====
// Pasta padrao para saida dos arquivos .dat
static std::string s_OutDir = "../out";

// ===== Config do OVERLAY =====
constexpr int    kOverlayTermW   = 420;    // largura da janela do over
[ajuste aqui]
constexpr int    kOverlayTermH   = 900;    // altura da janela do over
constexpr double kOverlayRulerPx = 84.0;    // largura da regua no over
[mais grossa]
constexpr double kOverlayGapPx   = 10.0;    // gap entre regua e track

// ————— SIDE-BY-SIDE (terminal do tamanho exato) —————
// Knobs (px) – ajuste ao gosto
constexpr int    kSideTermH      = 900;    // altura da janela
constexpr double kRulerPx        = 96.0;    // largura da regua
constexpr double kTrackPxTarget  = 180.0;    // largura desejada por tr
constexpr double kGapPx          = 12.0;    // gap entre paineis (regua
constexpr double kRightPadPx     = 16.0;    // respiro a direita (opc

// Tolerancia para comparar com o valor NULL do LAS
constexpr double kNullTol = 1e-4;

// Paleta simples de cores para overlay (hex RGB)
static const char* kColors[] = {
    "#1f77b4", "#ff7f0e", "#2ca02c", "#d62728",
    "#9467bd", "#8c564b", "#e377c2", "#7f7f7f",
    "#bcbd22", "#17becf"

```

```

};

// Filtra pontos invalidos (NULL) e devolve xs/ys filtrados
static void FilterNulls(const std::vector<double>& xin,
                        const std::vector<double>& yin,
                        std::optional<double> nullVal,
                        std::vector<double>& xout,
                        std::vector<double>& yout)
{
    xout.clear(); yout.clear();
    const size_t n = std::min(xin.size(), yin.size());
    if (!nullVal) {
        xout.assign(xin.begin(), xin.begin() + n);
        yout.assign(yin.begin(), yin.begin() + n);
        return;
    }
    const double nv = *nullVal;
    for (size_t i = 0; i < n; ++i) {
        double vx = xin[i];
        if (std::isfinite(nv) && std::isfinite(vx) && std::fabs(vx -
        xout.push_back(vx);
        yout.push_back(yin[i]);
    }
}

// Gera um .dat (x y) para uma serie e retorna o caminho do arquivo
static std::string WriteDat(const std::string& name,
                            const std::vector<double>& x,
                            const std::vector<double>& y)
{
    std::error_code ec;
    std::filesystem::create_directories(s_OutDir, ec);

    // sanitiza o nome do arquivo
    std::string safe = name;
    for (char& c : safe) {
        if (!(std::isalnum((unsigned char)c) || c == '_' || c == '-'))
    }

    std::string path = s_OutDir + "/track_" + safe + ".dat";

```

```

        std::ofstream f(path);
        f.setf(std::ios::fixed);
        f.precision(8);

        const size_t n = std::min(x.size(), y.size());
        for (size_t i = 0; i < n; ++i)
            f << x[i] << "_" << y[i] << "\n";

        return path;
    }

    // Escreve a regua de profundidade (coluna cinza) e retorna o caminho
    static std::string WriteDepthRuler(const std::vector<double>& depths)
    {
        std::error_code ec;
        std::filesystem::create_directories(s_OutDir, ec);

        std::string path = s_OutDir + "/track_DEPTH.dat";
        std::ofstream depthFile(path);
        depthFile.setf(std::ios::fixed);
        depthFile.precision(6);

        for (double d : depths) {
            depthFile << 0.0 << "_" << d << "\n";
            depthFile << 1.0 << "_" << d << "\n\n";
        }
        return path;
    }

    static void PreparePanel(Gnuplot& gp,
                             double originX, double width,
                             double xmin, double xmax,
                             double ymin, double ymax,
                             const std::string& xlabel,
                             bool wantLogX)
    {
        gp.Cmd("unset_lmargin");
        gp.Cmd("unset_rmargin");

        gp.Cmd(("set_size_" + std::to_string(width) + ",1").c_str());
    }

```

```

gp.Cmd(("set_origin_" + std::to_string(originX) + ",0").c_str());

// Limpa qualquer estado persistente
gp.Cmd("unset_key");
gp.Cmd("unset_label");
gp.Cmd("unset_arrow");
gp.Cmd("unset_object");
gp.Cmd("unset_logscale_x");

// Eixos e grades
gp.Cmd(("set_xlabel_" + xlabel + "'").c_str());
gp.Cmd("set_ylabel_'"); // nada de ylabel nas tracks
gp.Cmd(("set_xrange_" + std::to_string(xmin) + ":" + std::to_string(xmax)).c_str());
gp.Cmd(("set_yrange_" + std::to_string(ymax) + ":" + std::to_string(ymax)).c_str());
gp.Cmd("set_grid");

if (wantLogX)
    gp.Cmd("set_logscale_x");
}

static bool NameLooksLikeResistivity(const std::string& curve)
{
    if (curve == "RT" || curve == "RPD2" || curve == "RPM2" || curve == "RPM3")
        return true;
    return curve.rfind("RP", 0) == 0; // "RP*"
}

void CPlotter::PlotCurve(const CWellLog& rWell, const std::string& curveName)
{
    const auto& depth = rWell.GetDepths();
    auto it = rWell.GetCurves().find(curveName);
    if (it == rWell.GetCurves().end() || depth.empty()) {
        std::cerr << "[PlotCurve]_Curva_nao_encontrada_ou_profundidade_vazia\n";
        return;
    }

    const auto& vF = it->second.GetData();
    if (vF.size() != depth.size()) {
        std::cerr << "[PlotCurve]_Tamanho_incompativel_em_" << curveName << "\n";
        return;
    }
}

```

```

}

std::vector<double> xs(vF.begin(), vF.end());
std::vector<double> ys(depth.begin(), depth.end());

// Filtro de NULL
const auto nullOpt = (std::isnan(rWell.Info().null) ? std::optional<double>() : std::optional<double>());

std::vector<double> xsf, ysf;
FilterNulls(xs, ys, nullOpt, xsf, ysf);
if (xsf.empty()) {
    std::cerr << "[PlotCurve]_Sem_pontos_validos_apos_filtrar_NULL_em\n";
    return;
}

auto [mn, mx] = std::minmax_element(xsf.begin(), xsf.end());
double xmin = *mn, xmax = *mx;
if (xmin == xmax) { xmin -= 1.0; xmax += 1.0; }

double ymin = *std::min_element(ysf.begin(), ysf.end());
double ymax = *std::max_element(ysf.begin(), ysf.end());

// Decide log X: resistividade OU unidade contendo "OHM", mas so se >
bool wantLogX = NameLooksLikeResistivity(curveName) ||
    (it->second.Unit().find("OHM") != std::string::npos);
if (wantLogX) {
    for (double v : xsf) { if (!(v > 0.0)) { wantLogX = false; break; } }
}

const std::string labelX =
    it->second.Unit().empty() ? curveName : (curveName + "_" + it->second.Unit().str());

std::string dat = WriteDat(curveName, xsf, ysf);

Gnuplot gp;
gp.Cmd("reset");
gp.Cmd("set_terminal_windows_size_700,900");
PreparePanel(gp, /*originX*/0.0, /*width*/1.0, xmin, xmax, ymin, ymax);

gp.Cmd(("plot_" + dat + "'_using_1:2_with_lines_title_" + curveName

```

```

        gp.Cmd("pause_mouse_close");
    }

    void CPlotter::PlotMultiple(const CWellLog& rWell, const std::vector<std::string>& curvesToPlot)
    {
        for (const auto& c : curvesToPlot)
            PlotCurve(rWell, c);
    }

    void CPlotter::PlotSelectedTracks(const CWellLog& rWell,
                                      const std::vector<std::string>& curvesToPlot)
    {
        // 0) Checagem
        const auto& vecDepth = rWell.GetDepths();
        if (vecDepth.empty()) {
            std::cerr << "[PlotSelectedTracks]_Sem_profundidades_para_plotar.\n";
            return;
        }

        // 1) Lista de curvas (se vazio, usa todas as curvas do poço)
        std::vector<std::string> curves = curvesToPlot;
        if (curves.empty()) {
            for (const auto& kv : rWell.GetCurves())
                curves.push_back(kv.first);
        }
        if (curves.empty()) {
            std::cerr << "[PlotSelectedTracks]_Nenhuma_curva_selecionada.\n";
            return;
        }

        // 2) Profundidades (ys) e estatísticas
        std::vector<double> ys(vecDepth.begin(), vecDepth.end());
        const double fMinDepth = *std::min_element(ys.begin(), ys.end());
        const double fMaxDepth = *std::max_element(ys.begin(), ys.end());

        // 3) Arquivo da regua de profundidade
        const std::string depthDat = WriteDepthRuler(ys);

        // 4) Layout do multiplot
    }

```

```

const int totalTracks = static_cast<int>(curves.size());
const double trackWidth = 1.0 / static_cast<double>(totalTracks + 1);
double currentOrigin = 0.0;

Gnuplot gp;
gp.Cmd("reset");
gp.Cmd("set terminal windows size 1920,900");
gp.Cmd("set multiplot");
gp.Cmd("set tmargin at screen 0.95");
gp.Cmd("set bmargin at screen 0.05");

// 5) Coluna 0: regua de profundidade
{
    gp.Cmd(("set size " + std::to_string(trackWidth) + ",1").c_str());
    gp.Cmd(("set origin " + std::to_string(currentOrigin) + ",0").c_str());
    gp.Cmd("unset key");
    gp.Cmd("unset label");
    gp.Cmd("unset arrow");
    gp.Cmd("unset object");
    gp.Cmd("unset xtics");
    gp.Cmd("set ytics nomirror");
    gp.Cmd("set ylabel 'Profundidade (m)'");
    gp.Cmd("set xrange [0:1]");
    gp.Cmd(("set yrange [" + std::to_string(fMaxDepth) + ":" + std::to_string(fMinDepth) + "]");
    gp.Cmd("set style fill solid 1.0");
    gp.Cmd(("plot '" + depthDat + "' using 1:2 with filledcurves xl 1");
    currentOrigin += trackWidth;
}

// 6) Uma curva por painel
for (const auto& curveName : curves) {
    auto it = rWell.GetCurves().find(curveName);
    if (it == rWell.GetCurves().end()) {
        std::cerr << "[PlotSelectedTracks]_Curva_nao_encontrada:_ " << curveName << "\n";
        continue;
    }

    const auto& vF = it->second.GetData();
    if (vF.size() != ys.size()) {
        std::cerr << "[PlotSelectedTracks]_Tamanho_incompativel:_ " << curveName << "\n";
    }
}

```



```

        << "_" << vF.size() << "_vs_" << ys.size() << ")\n"
        continue;
    }

    std::vector<double> xs(vF.begin(), vF.end());

    // Filtro de NULL
    const auto nullOpt = (std::isnan(rWell.Info().null) ? std::optional<double>() : std::optional<double>());

    std::vector<double> xsf, ysf;
    FilterNulls(xs, ys, nullOpt, xsf, ysf);
    if (xsf.empty()) {
        std::cerr << "[PlotSelectedTracks]_Sem_pontos_validos_apos_filtro\n";
        currentOrigin += trackWidth;
        continue;
    }

    // X-range com dados filtrados
    auto [mn, mx] = std::minmax_element(xsf.begin(), xsf.end());
    double xmin = *mn, xmax = *mx;
    if (xmin == xmax) { xmin -= 1.0; xmax += 1.0; }

    // Decide se queremos log X (nome ou unidade) e se PODE logar (>0)
    bool wantLogX = NameLooksLikeResistivity(curveName) ||
        (it->second.Unit().find("OHM") != std::string::npos);
    if (wantLogX) {
        for (double v : xsf) { if (!(v > 0.0)) { wantLogX = false; break; } }
    }

    // label com unidade
    const std::string labelX =
        it->second.Unit().empty() ? curveName : (curveName + "_" + it->second.Unit());

    // Painel
    PreparePanel(gp, currentOrigin, trackWidth, xmin, xmax, fMinDepth, labelX, wantLogX);

    // .dat explicito e plot unico
    const std::string dat = WriteDat(curveName, xsf, ysf);
    gp.Cmd(("plot_" + dat + "'_using_1:2_with_lines_notitle").c_str());

```

```

        // avanca para o proximo painel
        currentOrigin += trackWidth;
    }

    // 7) Fecha multiplot
    gp.Cmd("unset_multiplot");

    gp.Cmd("pause_mouse_close");
}

void CPlotter::PlotCompareCurves(const CWellLog& well,
                                   const std::vector<std::string>& curvesIn,
                                   bool overlay,
                                   bool normalize)
{
    // ----- dados base -----
    const auto& d = well.GetDepths();
    if (d.empty()) {
        std::cerr << "[PlotCompareCurves]_Sem_profundidades.\n";
        return;
    }
    std::vector<double> depthY(d.begin(), d.end());
    const auto nullOpt = (std::isnan(well.Info().null) ? std::optional<double>()
                                                         : std::optional<double>(well.Info().null));

    // Selecao das curvas (mantem ordem pedida; se vazio, usa todas)
    std::vector<std::string> curves = curvesIn;
    if (curves.empty()) {
        for (const auto& kv : well.GetCurves())
            curves.push_back(kv.first);
    }
    if (curves.empty()) {
        std::cerr << "[PlotCompareCurves]_Nenhuma_curva_selecionada.\n";
        return;
    }

    struct Serie {
        std::string name;
        std::string unit;
    };

```

```

        std::string datPath;
        std::vector<double> x;
        std::vector<double> y;
    };

    std::vector<Serie> S;
    S.reserve(curves.size());

    // monta as series
    for (const auto& nm : curves) {
        auto it = well.GetCurves().find(nm);
        if (it == well.GetCurves().end()) {
            std::cerr << "[PlotCompareCurves]_Curva_nao_encontrada:_ " <<
                continue;
        }

        const auto& data = it->second.GetData();
        if (data.size() != depthY.size()) {
            std::cerr << "[PlotCompareCurves]_Tamanho_incompativel:_ " <<
                continue;
        }

        std::vector<double> xs; xs.reserve(data.size());
        std::vector<double> ys; ys.reserve(data.size());
        for (size_t i = 0; i < data.size(); ++i) {
            double v = data[i];
            double y = depthY[i];
            if (!std::isfinite(v)) continue;
            if (nullptr &&
                std::fabs(v - *nullptr) < 1e-6) continue;
            xs.push_back(v);
            ys.push_back(y);
        }
        if (xs.empty()) {
            std::cerr << "[PlotCompareCurves]_Sem_pontos_validos_em_" <<
                continue;
        }

        std::vector<double> xsf = xs;
    }

```

```

        std::vector<double> ysf = ys;
        S.push_back({ nm, it->second.Unit(), /*dat=*/"", std::move(xsf),
    }

// ----- OVERLAY -----
if (overlay) {
    // Normalizacao opcional (0-1) por serie
    if (normalize) {
        for (auto& s : S) {
            std::vector<double> nx;
            CLogCurveUtilities::Normalize01(s.x, nx);
            s.x.swap(nx);
        }
    }

    // Ranges globais
    double xmin = +1e300, xmax = -1e300;
    for (const auto& s : S) {
        auto [mn, mx] = std::minmax_element(s.x.begin(), s.x.end());
        xmin = std::min(xmin, *mn);
        xmax = std::max(xmax, *mx);
    }
    if (xmin == xmax) { xmin -= 1.0; xmax += 1.0; }
    double ymin = *std::min_element(depthY.begin(), depthY.end());
    double ymax = *std::max_element(depthY.begin(), depthY.end());

    // .dat das series
    for (auto& s : S)
        s.datPath = WriteDat(s.name, s.x, s.y);

    // Layout em px
    constexpr int kOverlayTermW = 900;
    constexpr int kOverlayTermH = 800;
    constexpr double kOverlayRulerPx = 96.0;
    constexpr double kOverlayGapPx = 16.0;

    const double rfrac = kOverlayRulerPx / static_cast<double>(kOverlayTermW);
    const double gfrac = kOverlayGapPx / static_cast<double>(kOverlayTermW);
    const double tfrac = std::max(0.0, 1.0 - rfrac - gfrac);

```

```

Gnuplot gp;
gp.Cmd("reset");
gp.Cmd("set_terminal_wxt");
gp.Cmd(("set_terminal_windows_size_" + std::to_string(kOverlayTerm));
gp.Cmd("set_multiplot");
gp.Cmd("set_tmargin_at_screen_0.95");
gp.Cmd("set_bmargin_at_screen_0.05");

// regua
{
    const std::string depthDat = WriteDepthRuler(depthY);
    gp.Cmd(("set_size_" + std::to_string(rfrac) + ",1").c_str());
    gp.Cmd("set_origin_0,0");
    gp.Cmd("unset_key"); gp.Cmd("unset_label"); gp.Cmd("unset_arrow");
    gp.Cmd("set_ytics_nomirror");
    gp.Cmd("set_ylabel_'Profundidade_(m)'");
    gp.Cmd("set_xrange_[0:1]");
    gp.Cmd(("set_yrange_" + std::to_string(ymax) + ":" + std::to_string(ymax)));
    gp.Cmd(("plot_" + depthDat + "'_using_1:2_with_lines_lc_rgb"));
}

// track overlay
{
    gp.Cmd(("set_size_" + std::to_string(tfrac) + ",1").c_str());
    gp.Cmd(("set_origin_" + std::to_string(rfrac + gfrac) + ",0"));

    gp.Cmd("unset_key");
    gp.Cmd(("set_xrange_" + std::to_string(xmin) + ":" + std::to_string(xmax)));
    gp.Cmd(("set_yrange_" + std::to_string(ymax) + ":" + std::to_string(ymax)));
    gp.Cmd("set_xlabel_'Curvas'");
    gp.Cmd("set_ylabel_'Profundidade_(m)'");

    std::ostream oss;
    oss << "plot_";
    bool first = true;
    int colorIdx = 0;
    for (const auto& s : S) {
        if (!first) oss << ",_";
        const char* col = kColors[colorIdx % (sizeof(kColors)/sizeof(kColors[0]))];
        oss << " '" << s.datPath << "'_using_1:2_with_lines_lc_rgb";
    }
}

```

```

        first = false;
        ++colorIdx;
    }
    if ( first )
        oss << "plot_1/0_notitle";
    gp.Cmd( oss.str() );
}

gp.Cmd("unset_multiplot");
gp.Cmd("pause_mouse_close");
return;
}

// ----- SIDE-BY-SIDE -----
// Ranges de profundidade
double fMinDepth = *std::min_element(depthY.begin(), depthY.end());
double fMaxDepth = *std::max_element(depthY.begin(), depthY.end());

const std::string depthDat = WriteDepthRuler(depthY);
const int n = static_cast<int>(S.size());

// 1) Define o tamanho do terminal a partir de n
//      regua + n*track + n*gap + padding
int termW = static_cast<int>(
    std::ceil(kRulerPx + n * kTrackPxTarget + n * kGapPx + kRightPadP
);
// (se quiser um teto, habilite a proxima linha)
// termW = std::min(termW, 1920);

// 2) Converte px -> fracoes do terminal (para set size / set origin)
const double rfrac = kRulerPx / static_cast<double>(termW);
const double tfrac = kTrackPxTarget / static_cast<double>(termW);
const double gfrac = kGapPx / static_cast<double>(termW);
// (o pequeno padding fica para fora, nao precisa fracao)

// 3) Desenho
double origin = 0.0;

Gnuplot gp;
gp.Cmd("reset");

```

```

gp.Cmd(("set_terminal_windows_size_" + std::to_string(termW) + "," +
gp.Cmd("set_multiplot");
gp.Cmd("set_tmargin_at_screen_0.95");
gp.Cmd("set_bmargin_at_screen_0.05");

// — regua com preenchimento —
{
    gp.Cmd(("set_size_" + std::to_string(rfrac) + ",1").c_str());
    gp.Cmd(("set_origin_" + std::to_string(origin) + ",0").c_str());

    gp.Cmd("unset_key"); gp.Cmd("unset_label"); gp.Cmd("unset_arrow");
    gp.Cmd("set_ytics_nomirror");
    gp.Cmd("set_ylabel_'Profundidade_(m)'");
    gp.Cmd("set_xrange_[0:1]");
    gp.Cmd(("set_yrange_" + std::to_string(fMaxDepth) + ":" + std::to_string(fMinDepth)));

    // pinta o painel todo antes de plotar (independente do estilo de plotagem)
    gp.Cmd("set_object_1_rectangle_from_graph_0,0_to_graph_1,1_fc_rgb_1,0,0");

    // um traco qualquer para materializar o eixo e a moldura
    // (pode usar o mesmo arquivo da regua, mas agora so com 'lines')
    gp.Cmd(("plot_" + depthDat + "'_using_1:2_with_lines_lc_rgb_#666666"));

    // limpa o objeto para nao vazar pro proximo painel
    gp.Cmd("unset_object_1");

    origin += rfrac + gfrac;
}

// — sem series validas? mostra mensagem e sai mantendo janela aberta
if (S.empty()) {
    gp.Cmd(("set_size_" + std::to_string(tfrac) + ",1").c_str());
    gp.Cmd(("set_origin_" + std::to_string(origin) + ",0").c_str());
    gp.Cmd("unset_key"); gp.Cmd("unset_label"); gp.Cmd("unset_arrow");
    gp.Cmd("set_xlabel_'Sem_curvas_validas_para_comparar'");
    gp.Cmd("set_ylabel_''");
    gp.Cmd("set_xrange_[0:1]");
    gp.Cmd(("set_yrange_" + std::to_string(fMaxDepth) + ":" + std::to_string(fMinDepth)));
    gp.Cmd("plot_1/0_notitle");
    gp.Cmd("unset_multiplot");
}

```

```

        gp.Cmd("pause_mouse_close");
        return;
    }

    // — uma track por painel (todas do mesmo tamanho) —
    for (auto& s : S) {
        auto [mn, mx] = std::minmax_element(s.x.begin(), s.x.end());
        double xmin = *mn, xmax = *mx; if (xmin == xmax) { xmin -= 1.0; xmax += 1.0; }

        bool wantLogX = NameLooksLikeResistivity(s.name) || (s.unit.find("log") != s.unit.end());
        if (wantLogX) for (double v : s.x) { if (!(v > 0.0)) { wantLogX = false; } }

        const std::string labelX = s.unit.empty() ? s.name : (s.name + " " + s.unit);

        s.datPath = WriteDat(s.name, s.x, s.y);

        // painel da track
        PreparePanel(gp, origin, tfrac, xmin, xmax, fMinDepth, fMaxDepth, labelX);
        gp.Cmd(("plot_" + s.datPath + "_using_1:2_with_lines_notitle"));

        origin += tfrac + gfrac;
    }

    gp.Cmd("unset_multiplot");
    gp.Cmd("pause_mouse_close");
    return;
}

void CPlotter::PlotInterpretationBasic(const CWellLog& well,
                                       const VshParams& vp,
                                       const SepParams& sp,
                                       const PayParams& pp)
{
    // == 0) Derivados e intervalos ==
    auto D = CInterpreter::ComputeDerived(well, vp, sp);
    auto picks = CInterpreter::PickReservoirs(D, pp);

    if (D.depth.empty()) {
        std::cerr << "[PlotInterpretationBasic]_Sem_profundidade.\n";
        return;
    }
}

```



```

}

// Depth (eixo Y, invertido)
std::vector<double> depth(D.depth.begin(), D.depth.end());
const double zMin = *std::min_element(depth.begin(), depth.end());
const double zMax = *std::max_element(depth.begin(), depth.end());

// === 1) GR bruto ===
std::vector<double> grx, gry;
{
    auto it = well.GetCurves().find(vp.gr_curve);
    if (it != well.GetCurves().end()) {
        const auto& vF = it->second.GetData();
        const size_t n = std::min(vF.size(), depth.size());

        // 2.1) Coleta valores validos, ja filtrando NULL do LAS
        std::vector<double> vValid;
        vValid.reserve(n);

        const bool hasNull = !std::isnan(well.Info().null);
        const double nullVal = well.Info().null;

        for (size_t i = 0; i < n; ++i) {
            double v = vF[i];
            if (!std::isfinite(v)) continue;
            if (hasNull && std::fabs(v - nullVal) < 1e-6) continue;
            vValid.push_back(v);
        }

        if (!vValid.empty()) {
            // 2.2) Percentil
            auto Percentil = [](std::vector<double> v, double p) -> double {
                if (v.empty()) return std::numeric_limits<double>::quiet_NaN();
                if (p <= 0.0) return *std::min_element(v.begin(), v.end());
                if (p >= 100.) return *std::max_element(v.begin(), v.end());
                std::sort(v.begin(), v.end());
                const double pos = p / 100.0 * (v.size() - 1);
                const size_t i0 = static_cast<size_t>(pos);
                const double frac = pos - i0;
                if (i0 + 1 < v.size())

```

```

        return v[i0] * (1.0 - frac) + v[i0 + 1] * frac;
    else
        return v[i0];
};

const double p1 = Percentil(vValid, 1.0);
const double p99 = Percentil(vValid, 99.0);

// 2.3) Monta GRX/GRY so dentro de [P1, P99]
grx.reserve(n);
gry.reserve(n);
for (size_t i = 0; i < n; ++i) {
    double v = vF[i];
    if (!std::isfinite(v)) continue;
    if (hasNull && std::fabs(v - nullVal) < 1e-6) continue;
    if (v < p1 || v > p99) continue; // corta spikes
    grx.push_back(v);
    gry.push_back(depth[i]);
}
}
}

// === 2) Resistividades profundas e rasas (ja de D) ===
std::vector<double> rdx, rdy, rsx, rsy;
{
    const size_t n = depth.size();
    rdx.reserve(n); rdy.reserve(n);
    rsx.reserve(n); rsy.reserve(n);

    for (size_t i = 0; i < n; ++i) {
        if (i < D.rdeep.size() && std::isfinite(D.rdeep[i]) && D.rdeep[i] > 0)
            rdx.push_back(D.rdeep[i]);
        rdy.push_back(depth[i]);
    }
    if (i < D.rshal.size() && std::isfinite(D.rshal[i]) && D.rshal[i] > 0)
        rsx.push_back(D.rshal[i]);
    rsy.push_back(depth[i]);
}
}

```

```

    }
}

if (grx.empty() && rdx.empty() && rsx.empty()) {
    std::cerr << "[PlotInterpretationBasic]_Nao_ha_dados_validos_de_C
    return;
}

// === 3) Ranges deduzidos dos dados ===
auto mm_lin = [] (const std::vector<double>& v) {
    double mn = +1e300, mx = -1e300;
    bool ok = false;
    for (double x : v) {
        if (std::isfinite(x)) { ok = true; mn = std::min(mn, x); mx =
    }
    if (!ok) { mn = 0.0; mx = 1.0; }
    if (mn == mx) {
        mn -= 1.0;
        mx += 1.0;
    } else {
        // da um "respiro" de 5% nas bordas
        double pad = 0.05 * (mx - mn);
        mn -= pad;
        mx += pad;
    }
    return std::pair<double, double>(mn, mx);
};

auto mm_pos = [] (const std::vector<double>& v) {
    double mn = +1e300, mx = -1e300;
    bool ok = false;
    for (double x : v) {
        if (std::isfinite(x) && x > 0.0) { ok = true; mn = std::min(mn, x);
    }
    if (!ok) { mn = 0.1; mx = 10.0; }
    if (mn == mx) {
        mn /= 10.0;
        mx *= 10.0;
    } else {
        double factor = std::pow(10.0, 0.1); // 1.26

```

```

        mn /= factor;
        mx *= factor;
    }
    return std::pair<double, double>(mn, mx);
};

auto [grMin, grMax] = mm_lin(grx);
auto [rdMin, rdMax] = mm_pos(rdx);
auto [rsMin, rsMax] = mm_pos(rsx);
double rxMin = std::min(rdMin, rsMin);
double rxMax = std::max(rdMax, rsMax);

// === 4) Arquivos .dat ===
const std::string depthDat = WriteDepthRuler(depth);
const std::string grDat     = WriteDat("GR",      grx, gry);
const std::string rdeepDat  = WriteDat("RDEEP",   rdx, rdy);
const std::string rshalDat  = WriteDat("RSHAL",   rsx, rsy);

// === 5) Layout do multiplot ===
const int termH = 800;
const double kRulerPx      = 80.0;
const double kTrackPx      = 260.0; // largura de cada track
const double kGapPx        = 16.0;
const double kRightPadPx   = 20.0;

const int nTracks = 2; // GR + Resistividade
int termW = static_cast<int>(std::ceil(kRulerPx + nTracks * kTrackPx

const double rulerFrac = kRulerPx / termW;
const double trackFrac = kTrackPx / termW;
const double gapFrac   = kGapPx   / termW;

Gnuplot gp;
gp.Cmd("reset");
gp.Cmd(("set_terminal_windows_size_" + std::to_string(termW) + "," +
gp.Cmd("set_multiplot");
gp.Cmd("set_tmargin_at_screen_0.96");
gp.Cmd("set_bmargin_at_screen_0.08");

// === 6) Regua de profundidade ===

```

```

{
    gp.Cmd(("set_size_" + std::to_string(rulerFrac) + ",1").c_str());
    gp.Cmd("set_origin_0,0");

    gp.Cmd("unset_key;_unset_label;_unset_arrow;_unset_object");
    gp.Cmd("unset_xtics;_set_ytics_nomirror");

    gp.Cmd("set_xrange_[0:1]");
    gp.Cmd(("set_yrange_" + std::to_string(zMax) + ":" + std::to_string(zMin)));

    // fundo cinza
    gp.Cmd("set_object_900_rect_from_graph_0,0_to_graph_1,1_behind_fc");
    gp.Cmd(("plot_" + depthDat + "'_using_1:2_with_lines_lc_rgb_#ddd"));

    gp.Cmd("unset_object_900");
}

// === 7) Track GR (apenas GR + sombras de pay) ===
{
    const double originX = rulerFrac + gapFrac;
    gp.Cmd(("set_size_" + std::to_string(trackFrac) + ",1").c_str());
    gp.Cmd(("set_origin_" + std::to_string(originX) + ",0").c_str());

    gp.Cmd("unset_key;_unset_arrow;_unset_object");
    gp.Cmd("set_grid_xtics_ytics");
    gp.Cmd(("set_yrange_" + std::to_string(zMax) + ":" + std::to_string(zMin)));
    gp.Cmd(("set_xrange_" + std::to_string(grMin) + ":" + std::to_string(grMax)));
    gp.Cmd("unset_logscale_x");
    gp.Cmd("set_xlabel_'GR_(GAPI)')");

    // sombras de pay
    int objId = 1000;
    for (const auto& iv : picks) {
        std::ostringstream oss;
        oss << "set_object_" << objId++
            << "_rect_from_graph_0,_first_" << iv.top
            << "_to_graph_1,_first_" << iv.base
            << "_behind_fc_rgb_#fff7c2'_fs_solid_1.0_noborder";
        gp.Cmd(oss.str());
    }
}

```

```

// curva GR fina (linha padrao)
std::ostream cmd;
cmd << "plot_";
if (!grx.empty()) {
    cmd << " ' " << grDat << " ' _using_1:2_with_lines_lc_rgb_'#7b68e
} else {
    cmd << "1/0_notitle";
}
gp.Cmd(cmd.str());

// opcional: limpar objetos se quiser
// gp.Cmd("unset object");
}

// === 8) Track Resistividades (Rdeep/Rshal) ===
{
    const double originX = rulerFrac + gapFrac + trackFrac + gapFrac;
    gp.Cmd(("set_size_" + std::to_string(trackFrac) + ",1").c_str());
    gp.Cmd(("set_origin_" + std::to_string(originX) + ",0").c_str());

    gp.Cmd("unset_key;_unset_arrow;_unset_object");
    gp.Cmd("set_grid_xtics_ytics");
    gp.Cmd(("set_yrange_" + std::to_string(zMax) + ":" + std::to_string(zMin)).c_str());
    gp.Cmd(("set_xrange_" + std::to_string(rxMin) + ":" + std::to_string(rxMax)).c_str());
    gp.Cmd("set_logscale_x");
    gp.Cmd("set_xlabel_'Resistividades_(ohm.m)");

    // sombras de pay
    int objId = 2000;
    for (const auto& iv : picks) {
        std::ostream oss;
        oss << "set_object_" << objId++
            << "_rect_from_graph_0,_first_" << iv.top
            << "_to_graph_1,_first_" << iv.base
            << "_behind_fc_rgb_'#fff7c2'_fs_solid_1.0_noborder";
        gp.Cmd(oss.str());
    }

    std::ostream cmd;

```

```

    cmd << "plot_";
    bool first = true;

    if (!rdx.empty()) {
        cmd << "' ' << rdeepDat << "'_using_1:2_with_lines_lc_rgb_'#ff
        first = false;
    }
    if (!rsx.empty()) {
        if (!first) cmd << ",_";
        cmd << "' ' << rshalDat << "'_using_1:2_with_lines_lc_rgb_'#ff
        first = false;
    }
    if (first)
        cmd << "1/0_notitle";

    gp.Cmd(cmd.str());
}

gp.Cmd("unset_multiplot");
gp.Cmd("pause_mouse_close");
}

// overload curto: usa cortes de pay sensatos
void CPlotter::PlotInterpretationBasic(const CWellLog& well,
                                         const VshParams& vp,
                                         const SepParams& sp)
{
    PayParams pp;
    pp.vsh_max      = 0.35;    // Vsh maximo
    pp.RR_min       = 3.0;     // minimo de Rdeep/Rshal
    pp.dR_min       = 1.0;     // separacao minima
    pp.min_thickness = 3.0;    // espessura minima (m)

    PlotInterpretationBasic(well, vp, sp, pp);
}

void CPlotter::PlotWithIntervals(const CWellLog& rWell,
                                  const std::vector<std::string>& curvesTo
                                  const std::vector<Interval>& ivals)

```

```

{
    const auto& vecDepth = rWell.GetDepths();
    if (vecDepth.empty()) { std::cerr << "[PlotWithIntervals]_Sem_profundidade\n"; }

    // 1) Curvas
    std::vector<std::string> curves = curvesToPlot;
    if (curves.empty()) {
        for (const auto& kv : rWell.GetCurves()) curves.push_back(kv.first);
    }
    if (curves.empty()) { std::cerr << "[PlotWithIntervals]_Nenhuma_curva\n"; }

    // 2) Depth e ranges
    std::vector<double> ys(vecDepth.begin(), vecDepth.end());
    const double fMinDepth = *std::min_element(ys.begin(), ys.end());
    const double fMaxDepth = *std::max_element(ys.begin(), ys.end());

    // Escreve regua (garante que pelo menos isso plote)
    const std::string depthDat = WriteDepthRuler(ys);

    // 3) Layout compacto px
    const int totalTracks = static_cast<int>(curves.size());
    constexpr int kTermH = 900;
    constexpr double kRulerPx = 96.0;
    constexpr double kTrackPx = 200.0;
    constexpr double kGapPx = 14.0;
    constexpr double kRightPadPx = 12.0;

    int termW = static_cast<int>(std::ceil(kRulerPx + totalTracks*kTrackPx));
    const double rfrac = kRulerPx / termW;
    const double tfrac = kTrackPx / termW;
    const double gfrac = kGapPx / termW;

    Gnuplot gp;
    gp.Cmd("reset");
    gp.Cmd(("set_terminal_windows_size_" + std::to_string(termW) + ", " +
    gp.Cmd("set_multiplot");
    gp.Cmd("set_tmargin_at_screen_0.95");
    gp.Cmd("set_bmargin_at_screen_0.05");

    double origin = 0.0;

```



```
// 4) Regua (preenchida) – SEMPRE PLOTA
```

```
{
    gp.Cmd(("set_size_" + std::to_string(rfrac) + ",1").c_str());
    gp.Cmd(("set_origin_" + std::to_string(origin) + ",0").c_str());
    gp.Cmd("unset_key"); gp.Cmd("unset_label"); gp.Cmd("unset_arrow");
    gp.Cmd("set_ytics_nomirror");
    gp.Cmd("set_ylabel_'Profundidade_(m)'");
    gp.Cmd("set_xrange_[0:1]");
    gp.Cmd(("set_yrange_" + std::to_string(fMaxDepth) + ":" + std::to_string(fMinDepth)).c_str());
    gp.Cmd("set_object_1_rectangle_from_graph_0,0_to_graph_1,1_fc_rgb");
    gp.Cmd(("plot_" + depthDat + "'_using_1:2_with_lines_lc_rgb'#606060").c_str());
    gp.Cmd("unset_object_1");
    origin += rfrac + gfrac;
}
```

```
// 5) Tracks
```

```
const auto nullOpt = (std::isnan(rWell.Info().null) ? std::optional<double>() : std::optional<double>(0));

int plotted_tracks = 0;
int object_id_base = 1000;

for (const auto& curveName : curves) {
    auto it = rWell.GetCurves().find(curveName);
    if (it == rWell.GetCurves().end()) {
        std::cerr << "[PlotWithIntervals]_Curva_nao_encontrada:_ " << curveName << "\n";
        origin += tfrac + gfrac; continue;
    }

    const auto& vF = it->second.GetData();
    if (vF.size() != ys.size()) {
        std::cerr << "[PlotWithIntervals]_Size_mismatch_" << curveName << "\n";
        origin += tfrac + gfrac; continue;
    }

    std::vector<double> xs(vF.begin(), vF.end());
    std::vector<double> xsf, ysf;
    FilterNulls(xs, ys, nullOpt, xsf, ysf);

    if (xsf.empty()) {
```

```

        std::cerr << "[PlotWithIntervals]_Sem_pontos_validos_em_" <<
        origin += tfrac + gfrac; continue;
    }

    auto [mn, mx] = std::minmax_element(xsf.begin(), xsf.end());
    double xmin = *mn, xmax = *mx;
    if (xmin == xmax) { xmin -= 1.0; xmax += 1.0; }

    bool wantLogX = NameLooksLikeResistivity(curveName) ||
        (it->second.Unit().find("OHM") != std::string::npos);
    if (wantLogX) for (double v : xsf) { if (!(v > 0.0)) { wantLogX =

    const std::string labelX = it->second.Unit().empty()
        ? curveName : (curveName + "_" + it->second.Unit() + "");

    PreparePanel(gp, origin, tfrac, xmin, xmax, fMinDepth, fMaxDepth,

    // sombreia intervalos (toda largura)
    int oid = object_id_base;
    for (const auto& I : ivals) {
        std::ostringstream os;
        os << "set_object_" << oid++ << "_rectangle_from_graph_0,_fir
            << I.top << "_to_graph_1,_first_" << I.base
            << "_fc_rgb_'#fff59d'_fs_solid_0.35_noborder_behind";
        gp.Cmd(os.str());
    }

    const std::string dat = WriteDat(curveName, xsf, ysf);
    gp.Cmd(("plot_" + dat + "'_using_1:2_with_lines_lc_rgb_'#6a1b9a'
    plotted_tracks++;

    // limpa objetos do painel
    for (int id = object_id_base; id < oid; ++id)
        gp.Cmd(("unset_object_" + std::to_string(id)).c_str());
    object_id_base += 1000;

    origin += tfrac + gfrac;
}

gp.Cmd("unset_multiplot");

```

```

    // Segura a janela mesmo se nada foi plotado (alem da regua)
    gp.Cmd("pause_mouse_close");
}

```

Fonte: produzido pelo autor.

Apresenta-se na listagem 7.19 a implementação da classe CLogCurveUtilities.

Listing 7.19: Arquivo CLogCurveUtilities.h

```

#ifndef CLOGCURVEUTILITIES_H
#define CLOGCURVEUTILITIES_H

#include <vector>
#include <cstdint>

namespace CLogCurveUtilities {

    // estatisticas ignorando NaN
    bool MinMax(const std::vector<double>& v, double& vmin, double& vmax)
    double Mean(const std::vector<double>& v);

    // media movel simples (janela w amostras; NaN-safe)
    void SmoothMA(std::vector<double>& v, int w);

    // normaliza para [0,1] ignorando NaN
    void Normalize01(const std::vector<double>& in, std::vector<double>& out);

    // alinha dois vetores ao menor tamanho
    template<typename T> void TruncToSameSize(std::vector<T>& a, std::vector<T>& b)
    {
        size_t n = std::min(a.size(), b.size());
        a.resize(n); b.resize(n);
    }
}

#endif

```

Fonte: produzido pelo autor.

Apresenta-se na listagem 7.20 a implementação da classe CLogCurveUtilities.

Listing 7.20: Arquivo CLogCurveUtilities.cpp

```

#include "CLogCurveUtilities.h"
#include <algorithm>

```

```

#include <cmath>

namespace CLogCurveUtilities {

bool MinMax(const std::vector<double>& v, double& vmin, double& vmax)
{
    bool ok = false;
    vmin = +1e300;
    vmax = -1e300;
    for (double x : v) {
        if (std::isfinite(x)) {
            ok = true;
            vmin = std::min(vmin, x);
            vmax = std::max(vmax, x);
        }
    }
    if (!ok) {
        vmin = 0.0;
        vmax = 0.0;
    }
    return ok;
}

double Mean(const std::vector<double>& v)
{
    double acc = 0.0;
    int n = 0;
    for (double x : v) {
        if (std::isfinite(x)) {
            acc += x;
            ++n;
        }
    }
    return (n > 0) ? acc / n : NAN;
}

void SmoothMA(std::vector<double>& v, int w)
{
    if (w <= 1 || v.empty())
        return;

```

```

    const int half = w / 2;
    const size_t N = v.size();
    std::vector<double> y(N, NAN);

    for (size_t i = 0; i < N; ++i) {
        int i0 = static_cast<int>(i) - half;
        int i1 = static_cast<int>(i) + half;
        if (i0 < 0) i0 = 0;
        if (i1 >= static_cast<int>(N)) i1 = static_cast<int>(N) - 1;

        double acc = 0.0;
        int n = 0;
        for (int k = i0; k <= i1; ++k) {
            if (std::isfinite(v[k])) {
                acc += v[k];
                ++n;
            }
        }
        if (n > 0)
            y[i] = acc / n;
    }

    v.swap(y);
}

void Normalize01(const std::vector<double>& in, std::vector<double>& out)
{
    out.assign(in.begin(), in.end());

    double mn, mx;
    if (!MinMax(out, mn, mx) || mx == mn) {
        std::fill(out.begin(), out.end(), 0.0);
        return;
    }

    const double inv = 1.0 / (mx - mn);
    for (double& x : out) {
        if (std::isfinite(x))
            x = (x - mn) * inv;
    }
}

```

```
        else
            x = NAN;
    }
}
```

Fonte: produzido pelo autor.

# Capítulo 8

## Teste

Todo projeto de engenharia passa por uma etapa de testes. Neste capítulo, apresentamos alguns testes do software GeoLogViewer. Esses testes têm o objetivo de validar as funcionalidades do sistema, verificando se os resultados obtidos estão de acordo com os requisitos definidos nos diagramas de caso de uso. Cada teste foi estruturado com dados de entrada, resultados esperados e análise de comportamento da aplicação.

### 8.1 Teste 1: Leitura do arquivo LAS e verificação dos perfis

O primeiro teste teve por objetivo validar o procedimento de carregamento e parsing de um arquivo LAS de entrada e confirmar que o parser extrai corretamente metadados e curvas para posterior processamento. A Figura 8.1 apresenta a execução do binário do GeoLogViewer em ambiente Windows, exibindo o menu principal do sistema. A sequência demonstrada inclui a seleção da opção "1) Carregar LAS", a indicação do caminho relativo do arquivo de teste (`../data/1-KPGL-1D-SPS_GR-RES.las`) e a mensagem de confirmação apresentada pelo programa: "Arquivo lido com sucesso!". Essa mensagem sinaliza que o CLASReader completou o parsing sem erros críticos e que os dados foram aceitos pela estrutura interna (CWellLog) para as etapas seguintes.

A Figura 8.2 complementa a validação ao mostrar o resultado da operação de listagem de curvas (opção "2) Listar curvas"). O console apresenta a lista de mnemonic codes das curvas detectadas no LAS — por exemplo: AP, ATMP, ECD, GR, ROP, RPD2, RPM2, RPS2, TVAXAVG, TVD, entre outras — demonstrando que o leitor identificou corretamente os nomes, preservou a ordem e organizou as curvas em estruturas consultáveis. A exibição dessas curvas confirma que o sistema extraiu tanto as curvas quanto seus vetores de amostras, o que é condição necessária para operações posteriores como normalização, comparação, interpretação e plotagem.

As duas figuras evidenciam o comportamento esperado do subsistema de I/O: o GeoLogViewer é capaz de localizar e abrir um arquivo LAS, realizar parse das seções relevantes

e disponibilizar ao usuário o inventário de curvas encontradas. A mensagem de sucesso e a listagem das curvas são indicadores de que o núcleo de leitura (CLASReader → CWellLog → CLogCurve) está operacional e robusto para arquivos com a estrutura testada

```
PS C:\Rafael\codigo\CODIGO_Final\build>
PS C:\Rafael\codigo\CODIGO_Final\build> . "C:/Rafael/codigo/CODIGO_Final/build/WellLogVisualizer.exe"

==== GeoLogViewer ====
1) Carregar LAS
2) Listar curvas
3) Plotar tracks selecionadas (multiplot)
4) Comparar curvas (overlay/side-by-side)
5) Interpretacao basica (GR | VSH | dR/RR) + shading
6) Exportar intervalos (CSV)
0) Sair
> 1
Caminho do arquivo LAS: ../data/1-KPGL-1D-SPS_GR-RES.las
Arquivo lido com sucesso!

==== GeoLogViewer ====
1) Carregar LAS
2) Listar curvas
3) Plotar tracks selecionadas (multiplot)
4) Comparar curvas (overlay/side-by-side)
5) Interpretacao basica (GR | VSH | dR/RR) + shading
6) Exportar intervalos (CSV)
0) Sair
> 
```

Figura 8.1: Tela do prompt executando a leitura do arquivo LAS



```
==== GeoLogViewer ====
1) Carregar LAS
2) Listar curvas
3) Plotar tracks selecionadas (multiplot)
4) Comparar curvas (overlay/side-by-side)
5) Interpretacao basica (GR | VSH | dR/RR) + shading
6) Exportar intervalos (CSV)
0) Sair
> 2
Curvas:
- AP
- ATMP
- ECD
- GR
- ROP
- RPD2
- RPD2_X
- RPM2
- RPS2
- TVAXAVG
- TVD
- TVLATAVG
- TVTOTAVG

==== GeoLogViewer ====
1) Carregar LAS
2) Listar curvas
3) Plotar tracks selecionadas (multiplot)
4) Comparar curvas (overlay/side-by-side)
5) Interpretacao basica (GR | VSH | dR/RR) + shading
6) Exportar intervalos (CSV)
0) Sair
> █
```

Figura 8.2: Tela do prompt listando curvas do arquivo LAS

## 8.2 Teste 2: Plotagem dos perfis do poço

O segundo teste teve como objetivo validar o módulo de plotagem do GeoLogViewer, responsável por converter os dados lidos do arquivo LAS em perfis visuais interpretáveis. A Figura 8.3. mostra a execução da funcionalidade “3) Plotar tracks selecionadas (multiplot)”, na qual o usuário pode optar por visualizar todas as curvas simultaneamente ou selecionar apenas algumas delas. Neste caso, a plotagem de todos os perfis foi realizada utilizando o backend gráfico do Gnuplot, gerando uma interface visual com múltiplas faixas verticais, cada uma correspondente a uma curva do arquivo.

O resultado exibido confirma que o sistema organizou corretamente cada curva em seu respectivo track, preservando escalas individuais conforme os valores originais registrados no LAS. Observa-se também que o eixo vertical representa a profundidade em metros, enquanto cada track possui seu próprio eixo horizontal de amplitude, variando de acordo

com a unidade da curva (como GAPI, PSI, OHMM, LB/G, entre outras). A consistência na exibição dos valores, a estabilidade na renderização das curvas e a ausência de artefatos visuais indicam que o módulo CPlotter está convertendo corretamente os vetores numéricos armazenados em CLogCurve para vetores gráficos.

As Figuras 8.3, 8.3 e 8.3 complementarão este teste, demonstrando cenários específicos de visualização respectivamente:

- Plotagem com overlay e normalização, onde duas curvas são ajustadas para a mesma escala e sobrepostas no mesmo track, facilitando análises comparativas;
- Plotagem com overlay sem normalização, na qual as curvas são sobrepostas mantendo suas escalas originais, destacando diferenças absolutas entre magnitudes;
- Plotagem sem overlay, exibindo as curvas selecionadas em tracks individuais para inspeção isolada.

Esses modos de plotagem asseguram flexibilidade para análises de perfis geofísicos, atendendo tanto a interpretações rápidas quanto a estudos comparativos mais detalhados. O teste confirma que o módulo gráfico do GeoLogViewer está funcional, responsivo e adequado para o fluxo de trabalho de visualização de dados de poços.

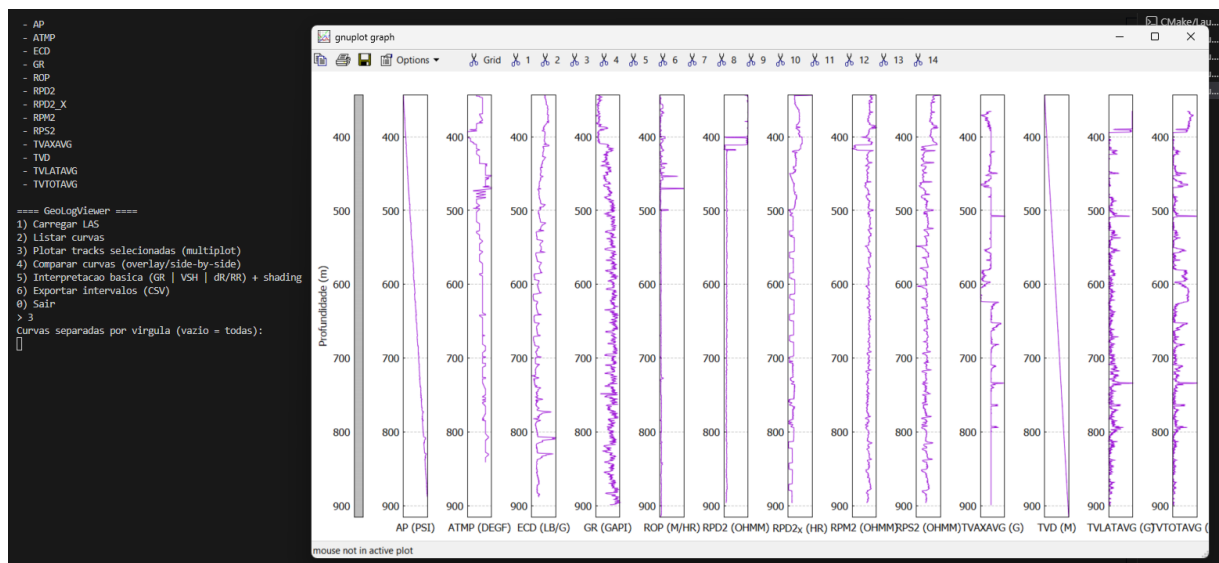


Figura 8.3: Tela do prompt ilustrando os perfis do arquivo

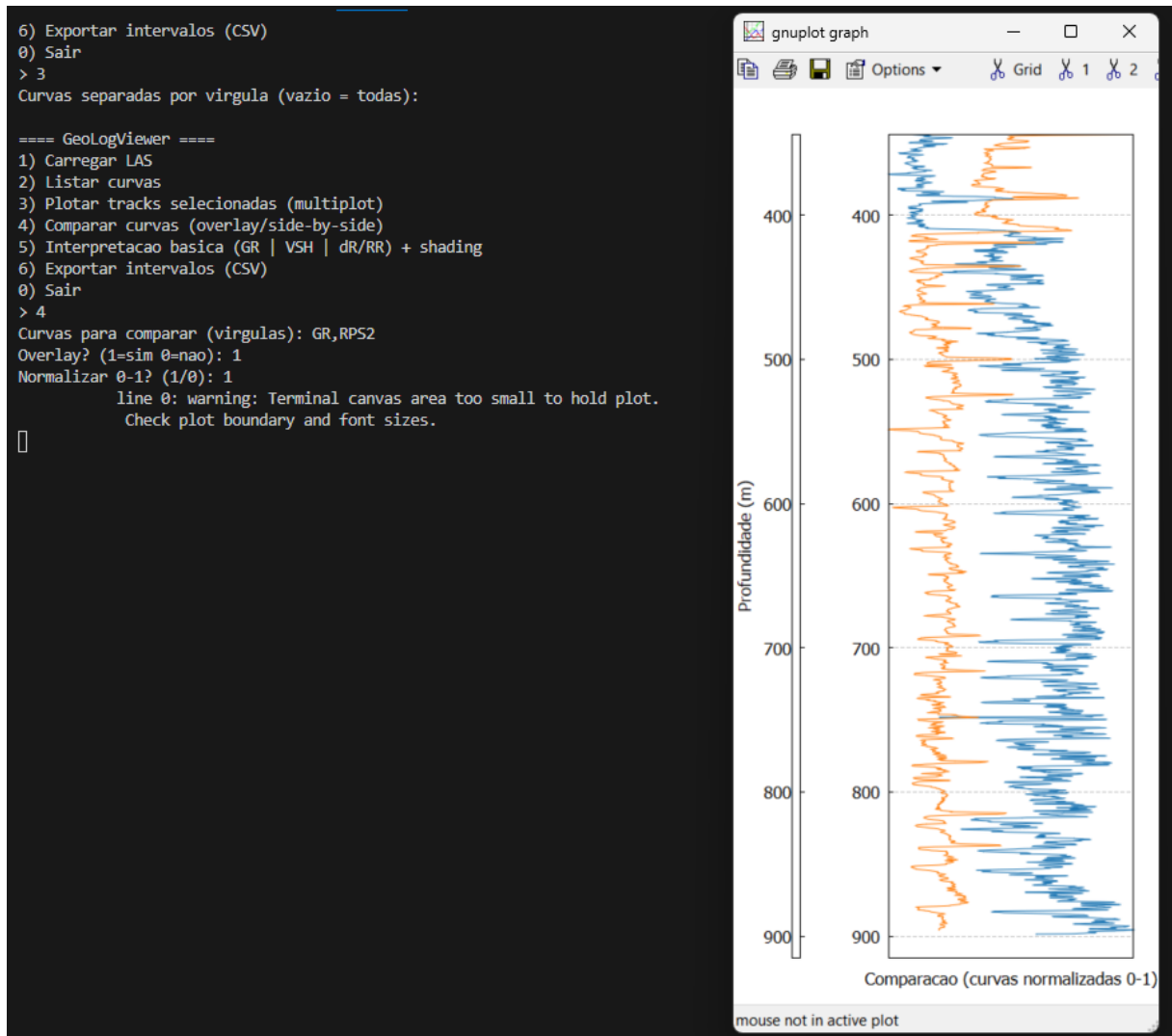


Figura 8.4: Tela do prompt ilustrando os perfis com overlay e normalização

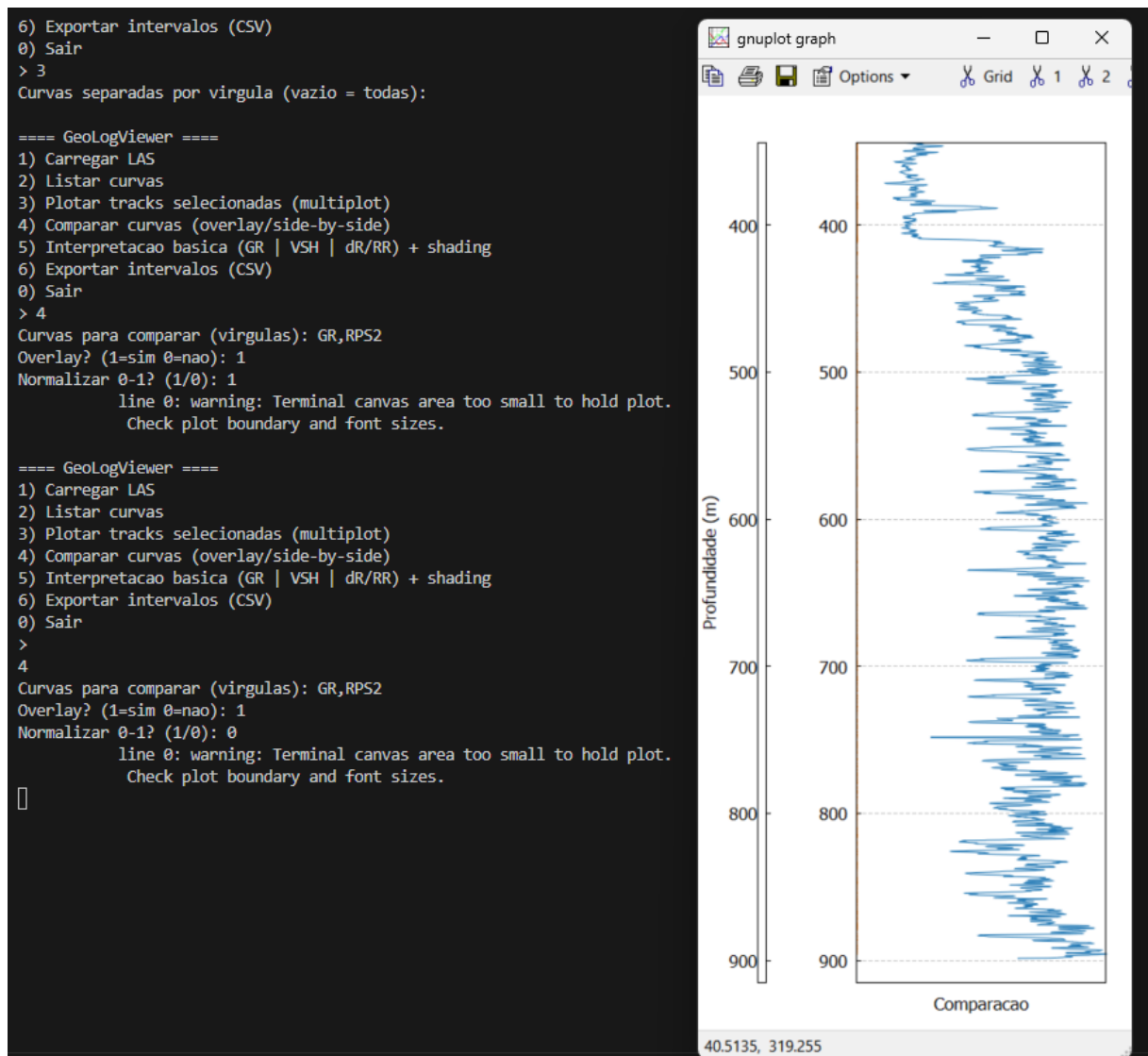


Figura 8.5: Tela do prompt ilustrando perfis com overlay e sem normalização

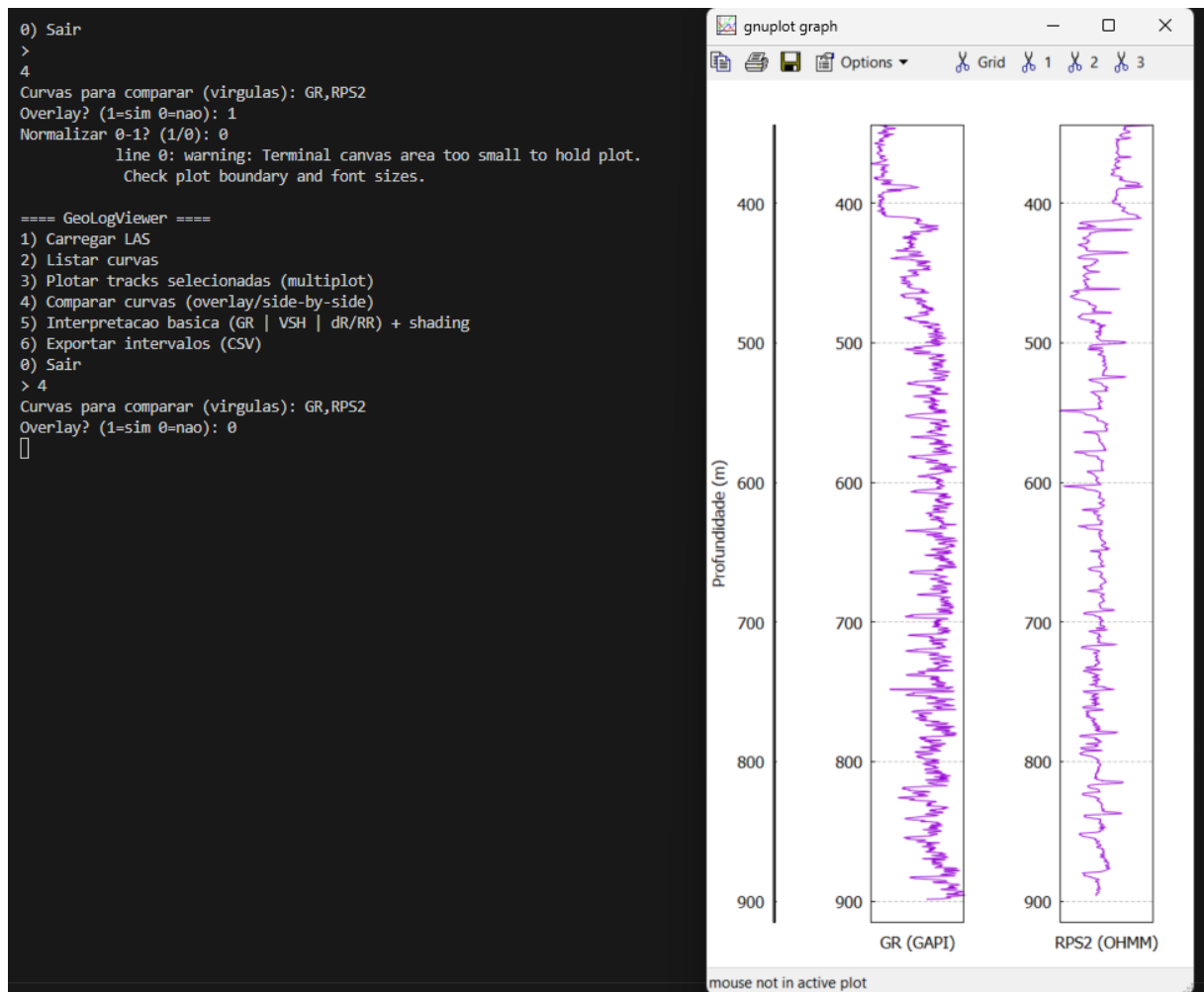


Figura 8.6: Tela do prompt ilustrando perfis sem overlay

### 8.3 Teste 3: Interpretação automática, exportação de dados e encerramento do aplicativo

O terceiro teste tem como finalidade validar o módulo de interpretação petrofísica, a exportação de intervalos interpretados e o comportamento do sistema durante o encerramento. A primeira imagem 8.7. demonstra a execução da funcionalidade “5) Interpretação básica (GR | VSH | dR/RR) + shading”, responsável por realizar automaticamente uma análise preliminar dos perfis geofísicos. Nesse processo, o usuário seleciona curvas como GR e RPS2 e o programa aplica algoritmos de interpretação — como cálculo de volume de argila, contraste resistivo e análises simples de possíveis zonas de reservatório. O gráfico exibido mostra a curva interpretada sobreposta ao shading gerado pelo cálculo automático, destacando trechos onde há maior probabilidade de presença de rocha reservatório. A integração entre os módulos CInterpreter, CPlotter e CPetrophysicsCalculator é evidenciada pela correta renderização dos resultados e pela coerência visual entre dados brutos e parâmetros interpretados.

A Figura 8.8 apresenta o funcionamento do módulo “6) Exportar intervalos (CSV)”, no qual o usuário solicita que os intervalos interpretados sejam gravados em um arquivo .csv. Esse recurso permite que os resultados do GeoLogViewer sejam utilizados em análises externas, relatórios acadêmicos ou importados para outros softwares de geologia e petrofísica. O teste validará se o arquivo é gerado corretamente, se os dados seguem o formato padronizado e se os valores exportados são consistentes com aqueles exibidos na interpretação gráfica.

A 8.9 mostrará o procedimento de encerramento do sistema através da opção “0) Sair”. Esse teste garante que o aplicativo finaliza suas operações de forma limpa, liberando corretamente memória, fechando arquivos utilizados e evitando erros ou travamentos. O comportamento esperado é que, após o comando de saída, o GeoLogViewer retorne ao terminal sem mensagens de erro, confirmando que todos os módulos foram encerrados adequadamente.

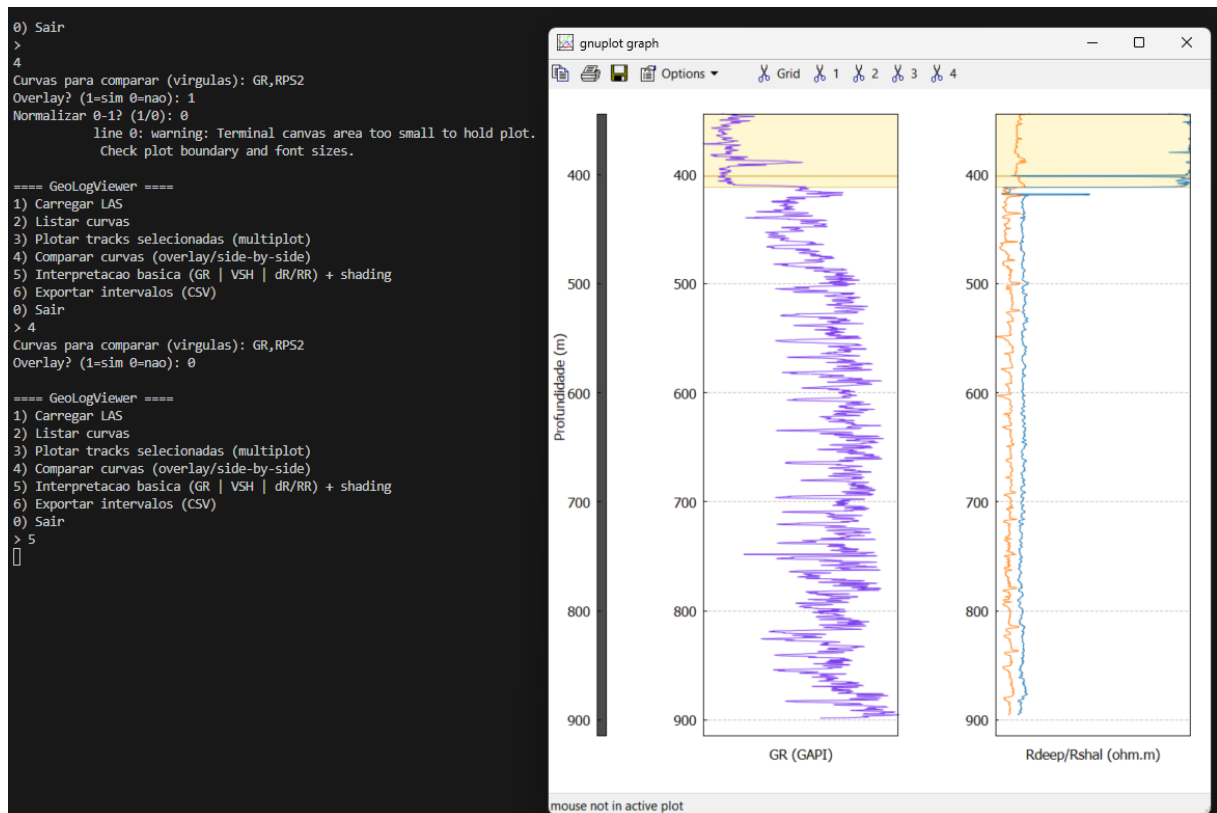


Figura 8.7: Tela do prompt ilustrando a interação do reservatório

```
out >  intervalos.csv
1  Top_m,Base_m,Thick_m,VSH_mean,Rdeep_mean,dR_mean,RR_mean
2  343.967,400.355,56.388,0.010,3832.025,3831.474,7083.487
3  401.422,411.175,9.754,0.027,2698.741,2698.097,4218.860
4    

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

> 5

==== GeoLogViewer ====
1) Carregar LAS
2) Listar curvas
3) Plotar tracks selecionadas (multiplot)
4) Comparar curvas (overlay/side-by-side)
5) Interpretacao basica (GR | VSH | dR/RR) + shading
6) Exportar intervalos (CSV)
0) Sair
> 6
intervalos.csv gerado em ./out

==== GeoLogViewer ====
1) Carregar LAS
2) Listar curvas
3) Plotar tracks selecionadas (multiplot)
4) Comparar curvas (overlay/side-by-side)
5) Interpretacao basica (GR | VSH | dR/RR) + shading
6) Exportar intervalos (CSV)
0) Sair
> 
```

Figura 8.8: Tela do prompt ilustrando o arquivo CSV gerado

```
==== GeoLogViewer ====  
1) Carregar LAS  
2) Listar curvas  
3) Plotar tracks selecionadas (multiplot)  
4) Comparar curvas (overlay/side-by-side)  
5) Interpretacao basica (GR | VSH | dR/RR) + shading  
6) Exportar intervalos (CSV)  
0) Sair  
> 0  
PS C:\Rafael\codigo\CODIGO_Final\build> █
```

Figura 8.9: Tela do prompt ilustrando a saída do aplicativo



# Capítulo 9

## Documentação para o Desenvolvedor

Nesta parte do projeto apresentamos alguns exemplos de uso do software.

### 9.1 Dependências para compilar o software

Para compilar o software é necessário atender as seguintes dependências:

- Instalar o compilador `g++` da GNU disponível em <http://gcc.gnu.org>. Para instalar no GNU/Linux use o comando `yum install gcc`.
- Biblioteca `CGnuplot`; os arquivos para acesso a biblioteca `CGnuplot` devem estar no diretório com os códigos do software;
- O software `gnuplot`, disponível no endereço <http://www.gnuplot.info/>, deve estar instalado. É possível que haja necessidade de setar o caminho para execução do `gnuplot`.



## Referências Bibliográficas